Tel-Aviv University
The Raymond and Beverly Sackler Faculty of Exact Sciences
The Blavatnik School of Computer Science

Algorithms for NP-Hard problems related to strings and for approximate pattern matching and repeats

Thesis submitted for the degree of "Doctor of Philosophy"

by Nira Shafrir

This work was carried out under the supervision of

Prof. Haim Kaplan

Submitted to the Senate of Tel-Aviv University
December 2009

To my parents Drora and Shlomo.
To my children Dana and Dror.

Abstract

This thesis has two main topics. The first one is design and analysis of approximation algorithms for NP-hard problems that are related to strings. The second topic is algorithms that are related to approximate pattern matching.

The first part is about the traveling salesperson problem on directed graphs (ATSP) and the shortest common superstring problem. We give approximation algorithms for several variants of the traveling salesperson problem on directed graphs, with better approximation ratios than what was previously known. We assume that we have a complete directed graph with non-negative weights $w(u, v)$ assigned to the arcs (u, v). We say that the edges satisfy the triangle inequality, if for every u, v and z, $w(u, v) \leq w(u, z) + w(z, v)$. In the minimum ATSP, we assume that the edge weights satisfy the triangle inequality, and the problem is to find a closed tour, visiting each vertex exactly once, of minimum weight. We give a $0.842 \log_2 n$-approximation algorithm for this problem. In maximum asymmetric TSP, the problem is to find a closed tour visiting each vertex exactly once of maximum weight. We give a 2/3-approximation algorithm for maximum ATSP with general non-negative edge weights, and a 10/13-approximation algorithm for maximum ATSP when the edge weights satisfy the triangle inequality.

The shortest common superstring problem is defined as follows. The input is a set $S = s_1, \cdots, s_n$ of strings and we seek the shortest possible string s such that every string in S is a (contiguous) substring of s. This problem is known to be NP-hard and even MAX-SNP hard. Using a known reduction from the shortest superstring problem to maximum asymmetric TSP, our approximation algorithm for maximum ATSP yields a 2.5-approximation algorithm for the shortest common superstring problem, which matches the best approximation ratio known for the problem with a simpler algorithm. We also analyze the simple greedy algorithm for this problem and show that its approximation ratio is 3.5. This improves upon the previously known approximation ratio of 4.

Another NP-Hard problem that we address is edit distance with block moves. The problem is defined as follows. Given two strings S and T, where $|S| + |T| = n$, find the minimum number of insert character, delete character, and move substring operations required to transform the string S into the string T. We give a lower bound of $\Omega(n^{0.46})$ on the approximation ratio of a greedy algorithm for the problem.

The second topic of this thesis is approximate pattern matching and approximate re-

peats. We consider the following approximate pattern matching problem. Let P be a pattern of length m, and let T be a text of length n. Let $k \leq m$ be an integer. For every position j of T we want to compute the position $s(j)$ in P, of the k-mismatch when we align P against T starting at position j of T. If P matches T starting at position j with at least one but less than k mismatches, we compute the position in P of the last mismatch. Otherwise, we report that there is an exact match at position j.

We give exact and approximation algorithms for this problem. Our approximation algorithms have an additional accuracy parameter ϵ. They compute for every position j in T, a position $s(j)$ in P, such that if we align P with T starting at position j of T, the number of mismatches up to position $s(j)$ of P is between $(1-\epsilon)k$ and $(1+\epsilon)k$.

Finally, in the last part of the thesis, we apply our algorithms for finding the position of the k-mismatch to efficiently solve an approximate version of the k-mismatch tandem repeats problem. In the k-mismatch tandem repeats problem, we are given a string S, and we look for all substrings uv of S, such that $|u| = |v| > k$, and the number of mismatches between u and v is at most k. In our relaxation we allow to report also substrings uv such that the number of mismatches between u and v is no larger than $(1+\epsilon)k$ for some accuracy parameter ϵ.

We also apply our exact algorithm for finding the position of the k-mismatch to efficiently find other kinds of approximate repeats with k-mismatches. A repeat is a substring of the form $x^i x'$, where $i \geq 2$ and x' is a prefix of x. A repeat is maximal if it can be extended neither to the right nor to the left and remain a repeat of the same kind and the same period. We consider algorithms for finding two kinds of maximal repeats that allow k-mismatches which were defined by Kolpakov and Kucherov [57].

We show that by allowing to algorithm to report also maximal repeats with $(1+\epsilon)k$ mismatches (but it does not have to report such repeats) we can substantially improve the dependency of the running time on k.

Acknowledgments

First of all, I am deeply grateful to my advisor Haim Kaplan for his guidance and help during this long period of my Ph.D studies and my M.Sc. studies. Working with him was both a pleasure and a learning experience.

Furthermore, I thank my co-authours to the papers containing the results of this thesis and to other papers with results not included in this thesis: Haim Kaplan, Loukas Georgiadis, Moshe Lewenstein, Eli Porat, Maxim Sviridenko, Robert Tarjan and Renato Werneck.

I also want to thank Uri Zwick for useful remarks in the k-mismatch result of Chapter 6, and to Gregory Kucherov for bringing paper [57] into our attention.

Finally, I want to thank my husband Arik for his love and support.

Contents

Chapter 1

Introduction

1.1 Overview

This thesis focuses on problems in two fields of stringology (algorithms to solve problems on strings). The first is approximation algorithms for NP-hard problems related to strings. The second is approximate pattern matching and approximate repeats. In this section, we give an overview of these topics.

1.1.1 NP-Hard problems related to strings

Many problems in stringology are motivated by problems that arise in computational biology and many of these problems are NP-hard problems.

Here are few examples. The closest string problem is defined as follows. Given a set of strings $s_1, \cdots, s_m$ and a parameter d, find a string s such that the Hamming distance of each s_i for $i = 1 \cdots m$ from s is at most d, (as an optimization problem, the goal is to minimize d). This problem has applications in computational biology to find similarity between DNA or RNA sequences. Li, Ma and Wang [68], gave a $(1 + \epsilon)$-approximation algorithm for the problem.

Another problem is the longest common subsequence (LCS) problem which is defined as follows. Given a set of strings $s_1, \cdots, s_n$, find the longest possible string x such that x is a subsequence of s_i for $i = 1 \cdots n$. A string x is a *subsequence* of a string y, if we can get x out of y, by deleting arbitrary number of characters from y. Since the longest common subsequences of strings gives us information about their similarity, this problem has applications both in molecular biology and in file comparison. The problem is NP-Hard [49]. However, if the given number of strings is constant, the problem can be solved in polynomial time, using dynamic programming.

A related problem is the shortest common supersequence problem which is defined as follows. Given a set of strings $s_1, \cdots, s_n$, find the shortest string x such that x is a

supersequence of each s_i for $i = 1 \cdots n$. A string x is a *supersequence* of a string y if y is a subsequence of x. As the LCS problem, the shortest common supersequence problem is NP-Hard [49] but can be solved in polynomial time using dynamic programming, if the given number of strings is constant.

Our Results. We now review the NP-hard problems related to strings that we deal with in this thesis. The first such problem is the *shortest superstring problem.* In the shortest superstring problem the input is a set $S = s_1, \cdots, s_n$ of strings and we seek the shortest possible string s such that every string in S is a (contiguous) substring of s. This problem is known to be NP-hard and even MAX-SNP hard [18]. For a more detailed introduction to this problem see Chapter 3. The shortest superstring problem is strongly related to the minimum and maximum asymmetric TSP problem.

Let G be a complete directed graph and let w be a non-negative weight function on the edges of G. We say that the edge weights satisfy the triangle inequality, if for every three vertices a, b and c, $w(a, c) \leq w(a, b) + w(b, c)$. In the maximum asymmetric TSP we need to find a closed tour of maximum weight. The maximum asymmetric TSP problem has two variants, one where the edge weights satisfy the triangle inequality, and the other with general (non-negative) edge weights. In the minimum asymmetric TSP problem, we are given a complete directed graph whose edge weights satisfy the triangle inequality, and we look for a closed tour of minimum weight visiting each vertex exactly once. Notice that there is no good approximation algorithm for the minimum asymmetric TSP problem with arbitrary edge weights, unless $P = NP$, see Section 2.1. In Chapter 2 we give approximation algorithms for these problems which are of self interest.

Before describing our results, we give the following definitions:

Definition 1.1.1 *A 2-cycle is a directed cycle of length* 2*.*

Definition 1.1.2 *A cycle cover in a graph is a spanning subgraph of disjoint simple cycles.*

Note that in a cycle cover of a directed graph each node belongs to exactly one simple directed cycle.

We obtain the approximation algorithms by introducing a new technique for decomposing directed regular multigraphs. A directed multigraph is said to be d-regular if the indegree and outdegree of every vertex is exactly d. By Hall's theorem one can represent such a multigraph as a combination of at most n^2 cycle covers each taken with an appropriate multiplicity.

We prove that if the d-regular multigraph does not contain more than $\lfloor d/2 \rfloor$ copies of any 2-cycle, then we can find a similar decomposition into n^2 *pairs of cycle covers* where each 2-cycle occurs in at most one component of each pair. Our proof is constructive and gives a polynomial time algorithm to find such a decomposition. Since our applications only need one such a pair of cycle covers whose weight is at least the average weight of all pairs, we also give an alternative, simpler algorithm to extract a single such pair.

This combinatorial theorem then comes handy in rounding a fractional solution of a linear program relaxation of the maximum Traveling Salesman Problem (TSP) problem. The first stage of the rounding procedure obtains two cycle covers that do not share a 2-cycle with weight at least twice the weight of the optimal solution. Then we show how to extract a tour from the 2 cycle covers, whose weight is at least 2/3 of the weight of the longest tour. This improves upon the previous 5/8 approximation with a simpler algorithm.

For minimum asymmetric TSP the same technique gives two cycle covers, not sharing a 2-cycle, with weight at most twice the weight of the optimum. Assuming triangle inequality, we then show how to obtain from this pair of cycle covers a tour whose weight is at most $0.842 \log_2 n$ larger than optimal. This improves upon a previous approximation algorithm with an approximation guarantee of $0.999 \log_2 n$. Other applications of the rounding procedure are approximation algorithms for maximum 3-cycle cover (approximation factor of 2/3, previously 3/5) and maximum asymmetric TSP with triangle inequality (approximation factor of 10/13, previously 3/4). The results of Chapter 2 appeared in [50].

In Chapter 3 we get back to the shortest superstring problem. It is known that a c-approximation to the minimum asymmetric TSP yields a $2c$-approximation to the shortest superstring problem, (see Chapter 3). But since there is no known constant approximation algorithm to the minimum asymmetric TSP problem (see chapter 2), this does not give a good approximation algorithm to the shortest superstring problem. However, Breslauer Jiang and Jiang [20] gave a reduction from the shortest common superstring problem to the maximum asymmetric TSP problem, which we describe in Section 3.4. Using this reduction together with our approximation algorithm for maximum asymmetric TSP we get a 2.5-approximation algorithm for the shortest common superstring problem which matches the best known bound of Sweedyk [80] but is simpler.

Still about the shortest superstring problem, we also analyze the simple greedy algorithm for the problem and prove that its approximation ratio is 3.5, (See Section 3.1). This improves the bound of 4 that was proven in [18]. The results of Chapter 3 appeared in [52].

Another NP-hard problem that we consider is the *edit distance with block moves*. In this problem, given two strings S and T, we want to find the minimum number of "insert character", "delete character", and "move substring" operations required to transform the string S into the string T. We give a lower bound on the approximation ratio of a greedy algorithm for this problem, and show that it is $\Omega(n^{0.46})$, where $n = |S| + |T|$. For a more detailed review of this problem and other related problems, see Section 4.1. The results of Chapter 4 appeared in [53].

1.1.2 Approximate Pattern Matching

The second part of the thesis is related to approximate pattern matching. The problem of exact pattern matching is as follows. Given a text T of length n and a pattern P of length m, find all occurrences of P in T. There are several classical linear time algorithms for this problem, see [56], [19].

There are also other more complex types of pattern matching. Another famous type is *pattern matching with don't cares*. In pattern matching with don't cares, the pattern and the text may contain don't care symbols. We mark a don't care symbol by the letter ϕ. We say that a pattern P occurs in position i of T, if for each $1 \leq j \leq m$, the character $T[i+j-1]$ matches the character $P[j]$, where $P[j]$ is the character at position j of P, and $T[\ell]$ is the character at position ℓ of T. We say that $P[j]$ matches $T[i+j-1]$ if $P[j] = T[i+j-1]$, $T[i+j-1] = \phi$, or $P[j] = \phi$. Recently, Peter Clifford and Raphal Clifford [24] gave the fastest known algorithm for the problem that runs in $O(n \log m)$.

There are many other types of pattern matching, we will mention only some of them. In subset matching each location in the pattern and in the text is a set of characters, and the problem is to find all occurrences of the pattern in the text. The pattern occur in position i of T, if the set $P[j]$ is a subset of the set $T[i+j-1]$ for $j = 1 \cdots m$. Cole, Hariharan and Indyk [28] gave a randomized algorithm that solves this problem with high probability. Their algorithm runs in $O(n \log^3 n / \log \log n)$ time. Cole, Hariharan and Indyk [28], also gave a deterministic algorithm for this problem, that runs in $O(n \log^3 n)$ time.

The subset matching problem is helpful in solving the tree pattern matching problem. The tree pattern matching problem is defined as follows. Given a pattern tree P and a text tree T, where both are ordered trees, we need to find all occurrences of P in T. The pattern P occurs at a node v in T if there is a one to one mapping from the nodes of P to nodes of T such that the following holds

1. The root of P is mapped to v.
2. If $x \in P$ is mapped into $y \in T$ and x is not a leaf, then the i-th child of x is mapped to the i-th child of y.

Cole and Hariharan [26] reduced the tree pattern matching problem to the subset matching problem, and thereby got a deterministic algorithm and a randomized algorithm for the problem that run in $O(n \log^3 n)$ time and in $O(n \log^3 n / \log \log n)$ time, respectively.

In two dimensional pattern matching[1], the pattern and the text are both two dimensional arrays, and we need to find all occurrences of the pattern within the text. It has applications in image processing. Crochemore *et al.* [30] gave a linear time algorithm for the problem that uses small space.

In parameterized pattern matching we have a pattern and a text that consist of parameter characters and "real" characters. The goal is to find all locations in the text that match the pattern. The pattern occurs in position i of T if

1. For each j either both $T[i+j-1]$ and $P[j]$ are parameter characters or both $T[i+j-1]$ and $P[j]$ are "real" characters.
2. If $T[i+j-1]$ and $P[j]$ are real characters, then $T[i+j-1] = P[j]$.

[1] There are also pattern matching algorithms for higher dimensions.

3. There is a bijection f that maps each parameter character in T to a parameter character in P, and $f(T[i+j-1]) = P[j]$ for each $1 \leq j \leq m$, such that both $T[i+j-1]$ and $P[j]$ are parameter characters. Notice that the bijection is not fixed. Thus for each i there may be a different bijection.

The parameterized pattern matching is used to find fragments of code that are identical (up to variables renaming). There are algorithms that solve this problem in $O(n \log(\min(m,p)))$ time, where p is the number of distinct parameter symbols. See [4, 10].

Another type of pattern matching is pattern matching with swaps. We say that P' is a swapped version of the pattern P if P' can be derived from P by a sequence of swaps between two consecutive positions in P, (P' has a swap at position j, if $P'[j+1] = P[j]$ and $P'[j] = P[j+1]$), such that each position participates in at most one swap. The pattern P matches T at position i if there is a swapped version P' of P such that $T[i+j-1] = P'[j]$ for all $1 \leq j \leq m$. Amir *et al.* [3] gave an algorithm that solves this problem in $O(nm^{1/3} \log m \log \min\{m, |\Sigma|\})$ time.

Definition 1.1.3 *Let $T(i,\ell)$ denote the substring of T of length ℓ starting at position i.*

A natural extension to the pattern matching problem is the problem of approximate pattern matching. In this problem we define some distance function, and say that a text position i matches the pattern if the distance function between $T(i,m)$ to P is bounded by some parameter. The definition of the distance function may vary, the simplest of which is the Hamming distance. That is, the number of mismatches at position $1 \leq i \leq n-m+1$ of the text is the Hamming distance between P and $T(i,m)$. Specifically, the Hamming distance between P and $T(i,m)$ is the number of positions j, such that $P[j] \neq T[i+j-1]$ for $1 \leq j \leq m$.

The main results on approximate pattern matching are an algorithm of Abrahamson [1] that finds the Hamming distance between $T(j,m)$ and P for each $j \leq n-m+1$, in $O(n\sqrt{m \log m})$ time. Amir, Lewenstein and Porat [6], found for a given k, all positions of the text that match the pattern with at most k-mismatches, in $O(n\sqrt{k \log k})$ time. Clifford *et al.* [25] found all the positions of the text that match the pattern with at most k-mismatches when both the pattern and the text contain don't care symbols. They give a deterministic algorithm that runs in $O(nk^2 \log^3 m)$ time and a randomized algorithm that runs in $O(n(k + \log n \log\log n) \log m)$ time and finds the answer with high probability.

Our results. We give algorithms for finding the position of the k-mismatch. Given a pattern P, a text T, and a parameter k, we give an algorithm that finds for each text position j the position in the pattern of the k-mismatch between P and $T(j,m)$. If $T(j,m)$ matches P with less than k-mismatches, we report the position of the last mismatch or that there is a perfect match. We refer to this problem as the *k-mismatch problem*. We give a deterministic algorithm for this problem that runs in $O(nk^{\frac{2}{3}} \log^{\frac{1}{3}} m \log k)$ time and linear space. For large values of k, our algorithm is faster than the algorithm that is often

called the *kangaroo method*, of Landau and Vishkin [65], and of Galil and Giancarlo [39] that runs in $O(nk)$ time and $O(n)$ space, see Section 5.1.

We also give a deterministic algorithm and a randomized algorithm for the following approximate version of the problem. The *approximate k-mismatch problem*, has an additional accuracy parameter ϵ. The task is to determine for every $1 \leq j \leq n - m + 1$, a position $s(j)$ in P such that the number of mismatches between $T(j, s(j))$ and $P(1, s(j))$ is at least $(1 - \epsilon)k$ and at most $(1 + \epsilon)k$, or report that there is no such position.

We give a deterministic algorithm for the *approximate k-mismatch problem*. The running time of this algorithm is $O(\frac{1}{\epsilon^2} n\sqrt{k} \log^3 m)$. We also give a randomized algorithm with running time of $O(\frac{1}{\epsilon^2} n \log n \log^2 m \log k)$. The randomized algorithm guarantees that for each j the number of mismatches between $T(j, s(j))$ and $P(1, s(j))$ is at least $(1 - \epsilon)k$ and at most $(1 + \epsilon)k$ with high probability. For more on these results, see Chapters 5 and 6. The results of Chapter 6 appeared in [51].

Other algorithms for approximate pattern matching. The special types of exact pattern matching mentioned above also have approximation algorithms. Amir, Lewenstein and Porat [7] handled the special case of subset matching with don't cares in the pattern and in the text. In this problem, each position of the pattern or the text is either a subset of characters or a don't care symbol. When we compare P and $T(i, m)$ we say that there is a match at position j, if $P[j]$ is a subset of $T[i+j-1]$ or at least one of $P[j]$ and $T[i+j-1]$ is a don't care. Amir, Lewenstein and Porat count the number of mismatches between P and $T(i, m)$ for all $1 \leq i \leq n - m + 1$ in $O((n + s)\sqrt{s' \log m})$ time, where s and s' denote the total number of elements in all subsets of T and P respectively.

Hazay, Lewenstein and Sokol [46] gave an algorithm for approximate parameterized pattern matching. Given a text T, a pattern P and a parameter k, their algorithm finds all locations i is the text such that $T(i, m)$ matches the pattern P with at most k mismatches. The running time of the algorithm is $O(nk^{1.5} + mk \log m)$.

Amir, Lewenstein and Porat [5] also gave an algorithm for approximate pattern matching with swaps. Their algorithm counts for each text position i that matches the pattern with swaps, the number of swaps needed to create a swapped version P', such that $T(i, m)$ matches P'. The running time of the algorithm is $O(f(n, m) + n \log m \log \min\{m, |\Sigma|\})$, where $f(m, n)$ is the time needed to find all text positions that match the pattern with swaps.

Another important distance function is the edit distance. There are various edit operations, that can be defined, see Chapter 4.1. In the most fundamental version of edit distance, the edit operations are character insertion, character deletion, and a substitution of a character by another character. The problem of approximate pattern matching under the edit distance with parameter k is defined as follows. Find all text positions i for which there exists a substring of the text $S = T[i \cdots j]$, such that the number of edit operations that are needed to convert S into P is at most k. Landau and Vishkin [66] gave an algorithm for this problem based on dynamic programming that runs in $O(nk)$ time. Cole and

Hariharn [27] gave an algorithm that runs in $O(\frac{nk^4}{m} + n + m)$ time.

1.1.3 Approximate repeats

The last part of the thesis is essentially an application of our algorithms for finding the position of the k-mismatch. We give algorithms that find several kinds of approximate repeats. The algorithms are presented in Chapter 7. Here we give some overview of this area and of our results.

Given a string S a *tandem repeat* in S is a substring of the form uu, $|u| \geq 1$. Checking whether a string contains no tandem repeats can be done in linear time [72]. Notice that there can be $O(n^2)$ tandem repeats in a string (for example in the string a^n). Main and Lorentz [71] gave an algorithm that finds all tandem repeats in S in $O(n \log n + z)$ time, where z is the number of tandem repeats in S. A *primitive* tandem repeat is a substring of the form uu, where u cannot be written as $u = w^i$ for an integer i, $i \geq 2$. There are at most $O(n \log n)$ primitive tandem repeats in a string (this is a tight bound), and they can be found in $O(n + z)$ time, where z is the number of primitive tandem repeats in S. See [58], [42], and [60].

repeats are strings of the form $u^i v$, where $i \geq 2$ and v is a prefix of u. A repeat is maximal if it can not be extended to the right or to the left by adding a character. Analyzing the structure of the string and finding its repeats has applications in speeding up pattern matching algorithms. For example, the algorithm of Cole and Hariharan [27] and the algorithm of Amir, Lewenstein and Porat [6] for approximate pattern matching under the edit distance and the Hamming distance respectively, partition the pattern into periodic and aperiodic segments. We also use a similar technique in our algorithm for finding the position of the k-mismatch, see Chapter 6. repeats also have applications in text compression algorithms.

repeats occur frequently in DNA and in protein sequences. Some repeats are associated with genetic diseases. The repeats are often not exact and there is a need to find approximate repeats.

Kolpakov and Kucherov [59] showed that the number of maximal repeats in a string of length n is cn for some constant c that they didn't specify. A bound of $1.6n$ was proved in [31] and was later improved to $1.029n$ in [32]. Kolpakov and Kucherov [58] also gave an algorithm that finds all maximal repeats in a string in linear time.

1.1.3.1 Approximate tandem repeats

A string uv is a *k-mismatch tandem repeat* if $|u| = |v|$, and the Hamming distance between u and v is at most k. Landau, Schmidt and Sokol [64] and Kolpakov and Kucherov [57] gave algorithms for finding all k-mismatch tandem repeats in a string S of length n that run in $O(nk \log(n/k) + z)$ time and in $O(nk \log k + z)$ time respectively, where z is the number of k-mismatch tandem repeats in S.

Landau, Schmidt and Sokol [64] also gave an algorithm that finds all k-mismatch tandem repeats for the edit distance. That is, it finds all substrings uv, where u and v are not necessarily of the same length, such that the edit distance between u and v is at most k. The running time of this algorithm is $O(nk \log k \log(n/k) + z)$, where z is the number of such approximate tandem repeats.

Our results. In Chapter 7.1, we define the *approximate k-mismatch tandem repeats* problem for the Hamming distance. In this problem, we have an additional parameter ϵ, and we want to find all k-mismatch tandem repeats but we are also allowed to report tandem repeats with at most $(1+\epsilon)k$-mismatches. We give algorithms for approximate k-mismatch tandem repeats that run faster than the algorithms for exact k-mismatch tandem repeats for large k. By combining the algorithm of [64] with our exact algorithm for the k-mismatch problem we get an algorithm for approximate k-mismatch tandem repeats that runs in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z)$ time where z is the number of approximate k-mismatch tandem repeats that we report. Similarly, using our deterministic algorithm for the approximate k-mismatch problem we get an algorithm for approximate k-mismatch tandem repeats that runs in $O(\frac{1}{\epsilon^3}n\sqrt{k} \log^3 n \log(n/k) + z)$ time. We can also use the randomized algorithm of Section 6.5 and get an algorithm that reports all k-mismatch tandem repeats with high probability, and possibly tandem repeats with up to $(1+\epsilon)k$ mismatches in $O(\frac{1}{\epsilon^3}n \log^3 n \log k \log(n/k) + z)$ time. The results of Chapter 7.1 appeared in [51].

1.1.3.2 Approximate general repeats in the Hamming Distance

There can be many definitions for k-mismatch repeats. Kolpakovand Kucherov [57] defined k-mismatch globally-defined repeat as follows. A string R of length n is called a *k-mismatch globally-defined repeat (gd-repeat) of period length p*, if $p \leq n/2$ and the number of times that $R[i] \neq R[i+p]$ is at most k.

Another type of approximate repeats that they defined is a run of k-mismatch tandem repeats of period length p (runs of repeats). A string R is a *run of k-mismatch tandem repeats of period length p*, if $p \leq n/2$ and all substrings of R of length $2p$ are k-mismatch tandem repeats.

A repeat is *maximal* if it can be extended neither to the right nor to the left and remain a repeat of the same kind and the same period. Kolpakov and Kucherov [57] gave an algorithm that finds all occurrences of maximal k-mismatch gd-repeats in a string S in $O(nk \log k + z)$ time, where z is the number of maximal k-mismatch gd-repeats in S. They also gave an algorithm that finds all occurrences of maximal runs of k-mismatch tandem repeats in a string S with the same running time.

Gd-repeats are very strict in the sense that the total number of allowed mismatches is at most k. On the other hand, runs of k-mismatch tandem repeats of period length p are very weak, since they only require that every substring of length $2p$ has at most k-mismatches. With this definition, the prefix of length p of a run R can be completely different from some

other substring of length p at position $R[1+jp], j \geq 1$, of the run. For example, the string 001011 is run of 1-mismatch tandem repeat of period length 2, whose prefix of length 2 is 00 and whose suffix of length 2 is 11.

Our results. We define the *approximate k-mismatch gd-repeat problem* as follows. Given a string S and $\epsilon > 0$, we want to find a set of substrings of S which are maximal $(1+\epsilon)k$-mismatch gd-repeats, such that each maximal k-mismatch gd-repeat is a substring of a string from the set. Using our exact algorithm for the k-mismatch problem, we construct two algorithms for the approximate k-mismatch gd-repeat problem that run in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log^2 k + z)$ and in $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log k\log(n/k) + z\right)$ time, respectively, where z is the number of maximal $(1+\epsilon)k$-mismatch gd-repeats that we report. We also use our first algorithm for the approximate k-mismatch gd-repeat problem to get another algorithm for approximate k-mismatch tandem repeats that runs in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log^2 k + z)$ time, where z is the number of approximate k-mismatch tandem repeats that we report.

Similarly, we define the *approximate run of k-mismatch tandem repeats problem* as follows. Given a string S and $\epsilon > 0$, we want to find a set of substrings which are runs of $(1+2\epsilon)k$-mismatch tandem repeats, not necessarily maximal, such that each maximal run of k-mismatch tandem repeats is a substring of a string that belongs to this set. We also require that the number of strings that we report is at most the number of maximal runs of $(1+\epsilon)k$-mismatch tandem repeats in S. Using our exact algorithm for the k-mismatch problem, we get an algorithm for the approximate runs of k-mismatch tandem repeats problem that runs in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log k\log(n/k)+z)$ time, where z is the number of maximal runs $(1+\epsilon)k$-mismatch tandem repeats in S. For more about this result, see Chapter 7.

Here are few other kinds of repeats that were defined in the literature. Extending our definitions and algorithms to these repeats is an open problem.

Other types of approximate repeats. Other types of approximate repeats mentioned in [57] are:

1. Uniform k-mismatch repeats. A string S of length $n \geq 2p$ is a uniform k-mismatch repeat of period length p, if for every substring $S(i,j), j \leq p$ the number of mismatches with $S(i+qp, j), q \geq 1$, is at most k. (See definition 1.1.3 of $S(i,j)$). For example, the string 001011 is a run of 1-mismatch tandem repeats of period length 2 but it is not a uniform 1-mismatch repeat of period length 2, since the number of mismatches between $S(1,2) = 00$ to $S(1+2*2,2) = 11$ is 2. The String 001000 is both a run of 1 mismatch tandem repeats and a uniform 1-mismatch repeat of period length 2, but it is not a 1-mismatch globally-defined repeat of period length 2.

2. A string S of length $\ell \geq 2p$ is a *k-mismatch consensus repeat* of period length p, if there is an exact repeat u of period length p, such that $|u| = \ell$ and the number of mismatches between $u[i\cdots j]$ to $S[i\cdots j]$ is at most k, for every i and j such that $j - i < p$. The string 001011 mentioned above, is both a run of 1-mismatch tandem

repeats and a 1-mismatch consensus repeat of period length 2, using the exact repeat $u = 101010$. The string 000010010110111 is a run of 1-mismatch tandem repeats of period length 3 but it is not a 1-mismatch consensus repeat of period length 3, since there isn't any exact repeat that can match both 111 and 000 with at most 1-mismatch.

We do not address these repeats here and leave the question of how to extend our techniques to handle them open.

Landau, Schmidt and Sokol [64] defined another type of approximate k-mismatch repeat. The definition at high level is as follows. Let $S = u_1 u_2 \cdots u_j u'$, where $|u_i| = p$, for $1 \leq i \leq j$, $j \geq 2$, and $0 \leq |u'| < p$. The string S is an approximate multiple repeat with k-mismatches for period length p, if there exists a string u of length p, that contains at most k don't care symbols such that each u_i for $1 \leq i \leq j$, matches u, and u' matches the prefix of u of length $|u'|$. They also defined the notion of "primitive" and "maximal" for this kind of approximate repeat. They gave an algorithm that finds all occurrences of such maximal and primitive k-mismatch repeats that runs in $O(nka \log(n/k))$ time, where a is the maximal number of periods in any reported repeat. That is, any approximate repeat of period length p that they find is of length smaller than p^{a+1}.

1.2 Algorithms and data structures that we use

We now review some of the techniques and data structures that we use.

1.2.1 Rounding of Linear Programming

Many approximation algorithms for NP-hard problems use the technique of rounding a solution to a linear program that describes a fractional version of the problem. Our approximation algorithms for asymmetric TSP in Chapter 2 round a solution to a linear program. For an overview of our rounding technique see Section 2.1.1.

The general idea is as follows. We formulate the problem as an integer program. We then relax the conditions that the variables are integers and get a linear program. We use a polynomial time algorithm to get an optimal solution to the linear program. Since an integer solution is a possible solution to the linear program, the optimal solution gives a bound, (an upper bound in case of a maximization problem, and a lower bound in case of a minimization problem), on the optimal solution for our original problem. Then we round the fractional solution to an integral solution without losing too much in the value of the objective function.

We demonstrate this technique using the set cover problem. In the set cover problem we are given a collection I of sets whose elements are taken from a universe U, $|U| = n$. The problem is to find a smallest subcollection of the sets such that each $x \in U$ is contained in

one of the sets of the subcollection. The following is an integer programming formulation of the set cover problem, where the variable x_S corresponds to the set S.

Min $\sum_{S \in I} x_S$ subject to

$\sum_{S \in I | j \in S} x_S \geq 1, \quad \forall j \in U$

$x_S \in \{0,1\} \quad \forall S \in I$ (Integer constraints)

We then replace the constraints $x_S \in \{0,1\}$ by the constraints $0 \leq x_S \leq 1$, and solve the resulting linear program and get a fractional solution x^*_S.

We round the fractional solution by picking S to our cover with probability x^*_S. We get a partial cover whose expected cost is the cost of the solution to the LP, (which is at most the cost of the optimal solution to the set cover problem). It can be proven that by repeating this $O(\log n)$ times and adding a set S to the subcollection if it was picked in at least one of the rounds, we get a set cover with high probability. That is, the probability that there exists an element from U which is not covered by the sets that we chose is at most $\frac{1}{poly(n)}$.

1.2.2 Greedy approximation algorithms

Greedy algorithms are sometimes used to approximate NP-hard problems. Greedy algorithms work by taking the step that improves our current local solution most. For example, the greedy algorithm for the set cover problem always chooses the set that covers the most items that were not covered yet. The greedy algorithm for set cover achieves an approximation ratio of $O(\log n)$. Inapproximability results show that this approximation ratio is best-possible for set cover.

In this thesis we prove an upper bound of 3.5 on the approximation ratio of a greedy algorithm for the shortest superstring problem.

However, greedy approximation algorithms do not always achieve good approximation ratios. For example we show a lower bound of $\Omega(n^{0.46})$ on the approximation ratio of a natural greedy algorithm for edit distance with block moves. On the other hand an approximation ratio of $O(\log n \log^* n)$ is achieved by a different algorithm.

1.2.3 Suffix Trees

A suffix tree is a data structure that helps to perform efficiently many operations on strings. To define a suffix tree, we first define a *trie*. Let $S_1, \cdots S_m$, be a set of strings over an alphabet Σ, such that no string is a prefix of another.[2] The trie of $S_1, \cdots, S_m$ is a tree with the following properties.

[2]Otherwise, we can add a unique character which does not belong to Σ to the end of each string.

1. The tree has m leaves, each corresponds to a string from the set. Let the leaf v_i represent the string S_i.

2. Each edge is labeled with a non empty substring. We get the string S_i by concatenating the strings of the edges on the path from the root to the leaf v_i.

3. Each internal node (that is a node that is not the root and not a leaf), has at least two children.

4. Let v be a node in the tree which is not a leaf. Let $e_1 = (v, u_1)$ and $e_2 = (v, u_2)$ be two outgoing edges of v, and let t_1 and t_2 be the labels on e_1 and on e_2 respectively. Then, t_1 and t_2 have no common prefix. That is, the longest common prefix of t_1 and of t_2 is of length 0.

For an example see Figure 1.1. We can construct the trie in a straightforward way in $O(\sum_{i=1}^{m} |S_i|)$ time.

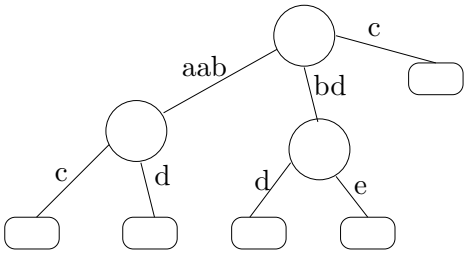

Figure 1.1: A trie for the strings *aabc*, *aabd*, *bdd*, *bde* and *c*.

We are now ready to define a suffix tree. A suffix tree for a string S of length n is a trie of the suffixes of S. To ensure that no suffix is a prefix of another we add to S the character \$ which is not in Σ. To ensure that the tree uses linear space, instead of explicitly storing a string with each edge we use the fact that all these strings are substring of S and store the indices of the first and the last character of each such string, (that is, instead of explicitly storing the string $S[i \cdots j]$ with an edge we store the pair (i, j)). This together with Properties 3 and 4 ensures that the size of the tree is $O(n)$. Notice that a straightforward algorithm to build a suffix tree runs in $O(n^2)$ time.[3] Weiner [87], gave an algorithm that builds a suffix tree in $O(n)$ time, (see also [73] and [85] for other algorithms).

We denote by v_i for $i = 1 \cdots n$, the leaf that corresponds to the suffix $S[i \cdots n]$. With suffix trees one can perform many operations on strings efficiently. Here are some examples.

1. Exact pattern matching. Using the suffix tree of a text T, given any pattern P, we can find all occurrences of P in T in $O(|P| + k)$ time, where k is the number of

[3]The total length of all suffixes of S is $O(n^2)$.

occurrences of P in T. This is done by traversing the tree T according to the pattern P until we reach a vertex v such that P is the prefix of the string from the root to v, (in case of a match), or until we get stuck (in case of a mismatch). If we find such a vertex v, then each leaf in the subtree of v corresponds to a different occurrence of P in T, (each such leaf v_i corresponds to a suffix $S[i \cdots n]$, such that P is a prefix of this suffix).

2. We can build a generalized suffix tree, see Figure 1.2. A *generalized suffix tree* of two strings S_1 and S_2,[4] is a trie over all the suffixes of S_1 and all the suffixes of S_2. Each leaf of the tree corresponds to a suffix of one of the strings. We add to S_1 a character $\$ \notin \Sigma$ and to S_2 a character $\# \notin \Sigma$, to ensure that all suffixes are unique. We can build a generalized suffix tree of S_1 and S_2 in $O(|S_1| + |S_2|)$ time by building a suffix tree for the string $S_1\$S_2\#$.

   A generalized suffix tree for a set of strings can be used to match a pattern against a database of strings, and to find all strings in the database in which the pattern occurs. (In a similar manner to the implementation of exact pattern matching that is described above).

3. Using a generalized suffix tree for S_1 and S_2, we can find the longest common substring of S_1 and S_2 in $O(|S_1| + |S_2|)$ time. A common substring is represented by a node v that has in its subtree leaves representing suffixes of both S_1 and S_2. The common substring is the string that is formed by concatenating the substrings of the edges on the path from the root to v. The longest common substring can be found by identifying the longest string associated with such a node v. We can do it by a careful scan of the suffix tree.

   We use a suffix tree to find the longest common substring of two strings, in the greedy algorithm for edit distance with block moves, see Section 4.3.

We can improve the functionality of the suffix tree by adding to it a data structure for answering lowest common ancestor (LCA) queries. Given two nodes u an v, the query $\mathrm{LCA}(u, v)$ returns the lowest node in the tree which is a common ancestor of u and v, see Figure 1.2. There are linear size data structures that one can construct in linear time supporting lowest common ancestor queries in constant worst case time. Harel and Tarjan gave the first such algorithm in [43]. Schieber and Vishkin [77], and Bender and Farach [12] gave simpler algorithms.

Given a generalized suffix tree for S_1 and S_2 with a data structure for LCA queries, we can find in constant time the longest common prefix of any substring of S_1 and a substring of S_2, by performing an LCA query on the vertices that represent the corresponding suffixes. In Chapter 5, we describe the *kangaroo method* of Landau and Vishkin [65], and Galil and

[4] We can define a generalized suffix tree for more than two strings as well.

Giancarlo [39]. This method uses a suffix tree with a data structure for LCA queries, to find all occurrences in the text that match a pattern with at most k mismatches in $O(nk + m)$ time where n is the length of the text and m is the length of the pattern.

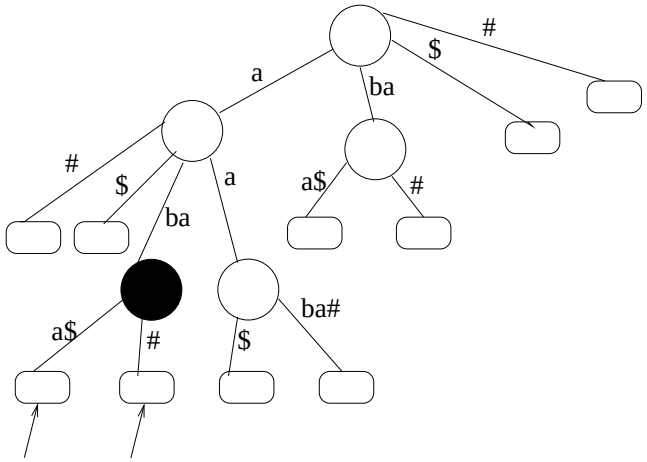

Figure 1.2: A generalized suffix tree for the strings *aaba* and *abaa*. To make all the suffixes different so that no suffix is a prefix of another, we added the character # to *aaba* and the character \$ to *abaa*. The black node is the lowest common ancestor of *aba*# and of *abaa*\$. Using a suffix tree that supports constant time lowest common ancestor queries, we can find that node in constant time. By saving with each node in the suffix tree the length of the string from the root to that node, we can find in constant time the length of the longest common prefix of the two suffixes. In this example, the longest common prefix of *aba*# and *abaa*\$ is *aba* of length 3.

Part I

Asymmetric TSP and the shortest superstring problem

Chapter 2

Approximation Algorithms for Asymmetric TSP

2.1 Introduction

The ubiquitous traveling salesman problem is one of the most researched problems in computer science and has many practical applications. The problem is well known to be NP-Complete and many approximation algorithms have been suggested to solve variants of the problem.

For general graphs it is NP-hard to approximate minimum TSP within any factor (to see this reduce from Hamiltonian cycle with non-edges receiving an extremely heavy weight). However, the metric version can be approximated. The celebrated $\frac{3}{2}$-approximation algorithm of Christofides [22] is the best approximation for the symmetric version of the problem. The *minimum asymmetric TSP (ATSP) problem* is the metric version of the problem on directed graphs. The first nontrivial approximation algorithm is due to Frieze, Galbiati and Mafioli [38]. The performance guarantee of their algorithm is $\log_2 n$. Bläser [13] improved their approach and obtained an approximation algorithm with performance guarantee $0.999 \log_2 n$. In this thesis we give a $4/3 \log_3 n$-approximation algorithm. Note that $4/3 \log_3 n \approx 0.841 \log_2 n$.

The *Maximum TSP Problem* is defined as follows. Given a complete weighted directed graph $G = (V, E, w)$ with nonnegative edge weights $w_{uv} \geq 0, (u, v) \in E$, find a closed tour of maximum weight visiting all vertices exactly once. Notice that the weights are non-negative. If we allow negative weights then the minimum and maximum variants essentially boil down to the same problem.[1]

One can distinguish 4 variants of the Maximum TSP problem according to whether weights are symmetric or not, and whether triangle inequality holds or not. The most

[1]The non-negativity of the weights is what allows approximations even without the triangle inequality since the minimum TSP is hard to approximate in that case.

	Symmetric	Asymmetric
Triangle inequality	7/8 Hassin et al. [45]	3/4 Kostochka et al. [62]
no triangle inequality	3/4 deterministic Serdyukov et al. [78] any fixed $\rho < 25/33$ randomized Hassin et al. [44]	5/8 Lewenstein et al. [67]

Table 2.1: Best known approximation guarantees for maximum TSP variants

important among these variants is also the most general one where weights are asymmetric and triangle inequality may not hold. This variant of the problem is strongly connected to the shortest superstring problem that has important applications in computational molecular biology. Table 2.1 summarizes the approximation factors of the best known polynomial approximation algorithms for these problems. On the negative side, these problems are MAX SNP-hard [75, 35, 36]. A good survey of the maximum TSP is [11].

We will be primarily interested in the general maximum ATSP problem. We describe a new approximation algorithm for this problem whose approximation guarantee is 2/3. For the asymmetric version with triangle inequality we also describe a new approximation algorithm whose approximation guarantee is 10/13. Thereby we improve both results of the second column of Table 2.1. Our algorithm rounds a fractional solution of an LP relaxation of the problem. This rounding procedure is an implementation of a new combinatorial decomposition theorem of directed multigraphs which we describe below. This decomposition result is of independent interest and may have other applications.

Since the maximum ATSP problem is related the several other important problems our results have implications for the following problems.

1. The shortest superstring problem. Given a set of strings $S = \{s_1, \cdots, s_n\}$, we search for the shortest string s such that every string in S is a (contiguous) substring of s. This problem which arises in DNA sequencing and data compression, has many proposed approximation algorithms, see [18, 82, 34, 61, 8, 9, 20, 80]. In [20] it was shown that a ρ approximation factor for maximum ATSP implies a $3.5-(1.5\rho)$ approximation for shortest superstring, see Chapter 3.4 for the description of the reduction. Thus our 2/3-approximation algorithm for maximum ATSP gives 2.5-approximation algorithm for the shortest superstring problem, matching the approximation guarantee of [80] with a much simpler algorithm. For a more detailed description of this problem see Chapter 3.

2. The maximal compression problem [81] has been approximated in [81] and [84]. Given a set of strings $S = \{s_1, \cdots, s_n\}$ and a superstring s of the strings in S we define the overlap of s with respect to S as $(\sum_{i=1}^{n} |s_i|) - |s|$. In the maximal compression problem, we search for a superstring s of the strings in S, such that the overlap of s with respect to S is the largest among all the superstrings of the strings in S. We denote the overlap of s with respect to S as maximum overlap. Notice that the shortest superstring is such a string.

The approximation problem is to find a superstring of the strings in S that approximates the maximum overlap. Notice that a constant approximation do this problem does not necessarily imply a constant approximation to the shortest common superstring problem. The problem can be transformed into a maximum ATSP problem with vertices representing strings and edges representing weights of string overlaps. Hence, a ρ approximation for maximum ATSP implies a ρ approximation for the maximal compression problem. This yields a 2/3-approximation algorithm improving previous results.

3. The minimum asymmetric {1,2}-TSP problem is the minimum ATSP problem where the edge weights are 1 or 2, see [75, 86, 17]. The problem can easily be transformed into the maximum variant, where the weights of 2 are replaced by 0. A ρ factor approximation for maximum ATSP implies a $2-\rho$ for this problem [75]. Thus our approximation algorithm for maximum ATSP yields a new 4/3 approximation for the minimum asymmetric {1,2}-TSP, matching the previous result of [17]. Recently, Bläser [14] obtained a 5/4 approximation for this problem.

4. In the maximum 3-cycle cover problem one is given a directed weighted graph for which a cycle cover containing no cycles of length 2 is sought [15]. Obviously, a solution to maximum ATSP is a solution to Max 3-cycle cover. Since the LP which we round is also a relaxation of the max 3-cycle cover problem we get that our algorithm is in fact also a 2/3-approximation to Max 3-cycle cover improving the 3/5-approximation of [15]. Recently Bläser *et al.* [16] improved the approximation ratio for this problem to 3/4, by rounding the same LP we used.

2.1.1 An overview of our rounding technique

To approximate maximum ATSP the following LP was used in [67], which is a relaxation of the problem of finding a maximum cycle cover which does not contain 2-cycles.

LP for Cycle Cover with Two-Cycle Constraint

Max $\sum_{(u,v)\in K(V)} w_{uv}x_{uv}$ subject to

$\sum_u x_{uv} = 1,$	$\forall v$	(indegree constraints)
$\sum_v x_{uv} = 1,$	$\forall u$	(outdegree constraints)
$x_{uv} + x_{vu} \leq 1,$	$\forall u \neq v$	(2-cycle constraints)
$x_{uv} \geq 0,$	$\forall u \neq v$	(non-negativity constraints)

The 2-cycle constraints are in fact a subset of the well known *subtour elimination* constraints studied by Held and Karp [47, 48]. In the Held-Karp formulation each subtour elimination constraint corresponds to a subset S of the vertices and requires that the tour contains at least one edge outgoing of S. Focusing on sets S consisting only of two distinct vertices say u and v, and using the indegree and outdegree constraints, it is easy to see that

the subtour elimination constraint corresponding to S is equivalent to the 2-cycle constraint defined by u and v. It follows that any solution to the Held-Karp LP is also a solution to our LP.

The algorithm in [67] transformed the solution of this LP into a collection of cycle covers with useful properties. This collection of cycle covers is then merged with a matching in a way similar to the procedure in [61]. (Note that the results of [67] and those we describe here hold also for the minimum version of the LP.)

Here we take a completely different approach and directly round the LP. We scale up the fractional solution to an integral one by multiplying it by the least common denominator D of all variables. This (possibly exponential) integral solution defines a multigraph, where the sum of the weights of its edges is at least D times the value of the optimal solution.[2] From the definition of the LP this multigraph has the property that it does not contain more than $\lfloor D/2 \rfloor$ copies of any 2-cycle.

We prove that any such multigraph can be represented as a positive linear combination of pairs of cycle covers (2-regular multigraphs) where each such pair contains at most one copy of any 2-cycle. This decomposition theorem can be viewed as a generalization of Hall's theorem that takes advantage of the 2-cycle constraints to get a decomposition of the regular multigraph into cycle covers with stronger structural guarantees. This decomposition can be computed in polynomial time using an implicit representation of the multigraph. As it is quite general we hope that it might have applications other than the one presented in this work.

Once we have the decomposition it follows that one pair of cycle covers has weight at least twice (at most twice, in case of minimum) the weight of the optimum solution. (This pair of cycle covers defines a *fractional* solution to the LP whose weight is at least (at most, respectively) 2/3 the weight of OPT.) It turns out that it is slightly simpler to directly extract one such pair of cycle covers with the same weight bounds. Since this is what is necessary for our applications, we present the simpler method. However, for the sake of generality and potential future applications, we also show how to obtain the complete decomposition.

For minimum ATSP we show how to obtain a TSP by recursively constructing cycle covers and choosing at each recursive step from the better of three possibilities, either one of the two cycle covers of the pair or the combination of both.

For maximum ATSP we show how to decompose this pair of cycle covers into three collections of paths. The heaviest among these collections has weight at least 2/3 of the weight of OPT so by patching it into a tour we get our approximate solution.

When a graph satisfies the triangle inequality better results for the maximum ATSP are known, as depicted in Table 1. We improve upon these results obtaining a 10/13 approximation factor. Our solution uses the method for finding the pair of cycle covers as before. The idea is to take the graph containing all 2-cycles of both cycle covers, decompose

[2] We never represent this multigraph explicitly.

it into two collections of paths, and patch the heavier collection into a tour. Then by taking the best among this tour and another tour constructed from the two cycle covers using an older technique, we obtain the improved approximation guarantee.

2.2 Roadmap

In Section 2.3 we present some basic definitions. In Section 2.4 we show how to find the pair of heavy cycle covers with the desired properties. In Section 2.5 we present an algorithm for the minimum ATSP. In Section 2.6 we show how to partition a pair of cycle covers satisfying the 2-cycle property into 3 path collections to obtain a 2/3 approximation algorithm for maximum ATSP and in Section 2.7 we present an improved algorithm for graphs that satisfy triangle inequality. In Section 2.8 we present the full decomposition theorem. Finally, in Section 2.9, we mention subsequent results that improved some of the bounds that we achieved.

2.3 Preliminaries, definitions and algorithms outline

Let $G = (V, E)$ be a TSP instance, maximum or minimum. We assume that G is a complete graph without self loops. Let $n = |V|$, and $V = \{v_1, v_2, \cdots, v_n\}$, and $E = K(V) = (V \times V) \backslash \{(v, v) \mid v \in V\}$. Each edge e in G has a non negative integer weight $w(e)$ associated with it. We further denote by $w(G)$ the weight of a graph (or a multigraph) G i.e. $w(G) = \sum_{e \in G} w(e)$. A *cycle cover* of G is a collection of vertex disjoint cycles covering all vertices of G. An i-cycle is a cycle of length i.

Consider the LP described in the introduction and let $\{x^*_{uv}\}_{uv \in K(V)}$ be an optimal solution of it. Let D be the minimal integer such that for all $(u, v) \in K(V), Dx^*_{uv}$ is integral. We define $k \cdot (u, v)$ to be the multiset containing k copies of the edge (u, v). We define the weighted multigraph $D \cdot G = (V, \hat{E}, w)$ where $\hat{E} = \{(Dx^*_{uv}) \cdot (u, v) \mid (u, v) \in K(V)\}$. All copies of the same edge $e \in E$ have the same weight $w(e)$. The multiplier D may be exponential in the graph size but $\log D$, which is the number of bits needed to represent D, is polynomial in n. We also denote by $m_G(e)$ the number of copies of the edge e in the multigraph G. We represent each multigraph G' that we use throughout the proof in polynomial space by maintaining the multiplicity $m_{G'}(e)$ of each edge $e \in G'$. We denote by $C_{u,v}$ the 2-cycle consisting of the edges $(u, v), (v, u)$.

Notice that the solution to the LP is such that $D \cdot G$ is D-regular, and for any two nodes u, v, $D \cdot G$ contains at most $\lfloor D/2 \rfloor$ copies of the 2-cycle $C_{u,v}$. We say that a regular multigraph with this property is *half-bound*. In other words a D regular multigraph is *half-bound* if for any $u, v \in V$ there are at most $\lfloor \frac{D}{2} \rfloor$ edges from u to v or there are at most $\lfloor \frac{D}{2} \rfloor$ edges from v to u.

In particular a 2-regular multigraph is half-bound if it has at most one copy of each

2-cycle. It is well known that we can decompose a d-regular multigraph into d cycle covers.[3] So if we decompose a 2-regular half-bound multigraph we obtain two cycle covers that do not share a 2-cycle. Therefore a 2-regular half-bound multigraph is equivalent to two cycle covers that do not share a 2-cycle.

We denote by OPT the value of the optimal solution to the TSP problem. It follows from the definition of the LP that $w(D \cdot G) \geq D \cdot OPT$ in case of a maximization LP, and $w(D \cdot G) \leq D \cdot OPT$ in case of a minimization LP.

Using the graph $D \cdot G$, we find 2 cycle covers C_1 and C_2 in G such that,

(a) C_1 and C_2 do not share a 2-cycle, i.e. if C_1 contains the 2-cycle $C_{u,v}$ then C_2 does not contain at least one of the edges $(u, v), (v, u)$, and vice versa.

(b) $w(C_1) + w(C_2) \geq 2OPT$.

Analogously, we can replace the second requirement to be:

(b) $w(C_1) + w(C_2) \leq 2OPT$.

in case the solution to the LP minimized the cost. Our methods for finding two cycle covers will work in the same way for either requirement. The appropriate requirement will be used in accordance with the application at hand.

For maximum ATSP, we partition the edges of C_1 and C_2 into 3 collections of disjoint paths. Since $w(C_1) + w(C_2) \geq 2OPT$ at least one of these collections has weight of $\frac{2}{3}OPT$. Finally we patch the heaviest collection into a Hamiltonian cycle of weight $\geq \frac{2}{3}OPT$.

For minimum ATSP, we improve upon the recursive cycle cover method by choosing at a recursive step one of C_1, C_2 and $C_1 \cup C_2$ according to the minimization of an appropriate function. Here we use the fact that $w(C_1) + w(C_2) \leq 2OPT$.

Finally, we also use this result, in a different way, to achieve better results for maximum ATSP with triangle inequality.

2.4 Finding two cycle covers

In this section we show how to find two cycle covers (CC) C_1 and C_2 that satisfy the conditions specified in Section 2.3. First we show how to find such a pair of CCs when D is a power of two, then we give an algorithm for all values of D.

[3] One can do this by decomposing a related d-regular bipartite multigraph into d perfect matchings. See Section 2.4.1.

2.4.1 Finding two CCs when D is a power of 2

In the following procedure we start with $D \cdot G$ which satisfies the half-bound invariant and also has overall weight $\geq D \cdot OPT$. By careful rounding we recurse to smaller D maintaining the half-bound invariant and the weight bound, i.e. $\geq D \cdot OPT$. We recurse until we reach $D = 2$ which gives us the desired result. We describe the algorithm for the maximization version of the problem, i.e. we will find two cycle covers such that $w(C_1) + w(C_2) \geq 2OPT$. The algorithm for the minimization problem is analogous.

Let $G_0 = D \cdot G$. Recall that G_0 is D-regular, $w(G_0) \geq D \cdot \mathrm{OPT}$, and G_0 is half-bound, i.e. it contains each 2-cycle at most $D/2$ times. We show how to obtain from G_0 a $D/2$-regular multigraph G_1, such that $w(G_1) \geq \frac{D}{2}\mathrm{OPT}$, and G_1 is half-bound, i.e. it contains each 2-cycle at most $D/4$ times. By applying the same procedure $\log(D) - 1$ times we obtain a 2-regular multigraph, $G_{\log(D)-1}$, such that $w(G_{\log(D)-1}) \geq 2\mathrm{OPT}$, and $G_{\log(D)-1}$ contains at most one copy of each 2-cycle. It is then straightforward to partition $G_{\log(D)-1}$ into two CCs that satisfy the required conditions.

We first build from $G_0 = D \cdot G$ a D-regular bipartite undirected multigraph B as follows. For each node $v_i \in V$ we have 2 nodes v_i, and v_i' in B, that is $V_B = \{v_1, \cdots, v_n, v_1', \cdots, v_n'\}$. For each edge (v_i, v_j) in G_0 we have an edge (v_i, v_j') in B.

We use the following technique of Alon [2] to partition B into two $D/2$-regular bipartite multigraphs B_1, B_2. For each edge $e \in B$ with $m_B(e) \geq 2$, we take $\lfloor m_B(e)/2 \rfloor$ copies of e to B_1 and $\lfloor m_B(e)/2 \rfloor$ copies to B_2, and omit $2\lfloor m_B(e)/2 \rfloor$ copies of e from B. Next we find an Eulerian cycle in each connected component of the remaining subgraph of B, and assign its edges alternately to B_1 and B_2.[4]

Let G', be the subgraph of G_0 that corresponds to B_1, and let G'', be the subgraph of G_0 that correspond to B_2. Clearly G' and G'' are $D/2$-regular directed multigraphs. It is also straightforward to see that each of G' and G'' contains at most $D/4$ copies of each 2-cycle. The reason is as follows. Since we had at most $D/2$ copies of each 2-cycle in G_0, then for each pair of vertices at least one of the edges (u, v), and (v, u) appeared at most $D/2$ times in G_0. By the definition of the algorithm above this edge will appear at most $D/4$ times in both graphs G', and G''.

We let G_1 be the heavier among G' and G''. Since G' and G'' partition G it is clear that $w(G') + w(G'') = w(G)$. Therefore, since $w(G) \geq D \cdot \mathrm{OPT}$ we obtain that $w(G_1) \geq \frac{D}{2}\mathrm{OPT}$.

The running time of this algorithm is $O(n^2 \log D)$, since we have $\log D$ iterations, each of them costs $O(n^2)$.

[4]The technique of using an Euler tour to decompose a multigraph has been used since the early 80's mainly for edge coloring (See [55] and the references there). Alon does the partitioning while balancing the number of copies of each individual edge that go into either of the two parts. We use it to keep the multigraphs half-bound.

2.4.2 Finding 2 cycle covers when D is not a power of 2

We now handle the case where D is not a power of 2. In this case we first describe a rounding procedure that derives from the solution to the LP a 2^y-regular multigraph. Then we apply the algorithm from Section 2.4.1 to this multigraph. We assume that G has at least 5 vertices.

2.4.2.1 Rounding the solution of the LP into a multigraph

Let W_{max} be the maximum weight of an edge in G. Let y be an integer such that $2^{y-1} < 12n^2W_{max} \leq 2^y$, and define $\bar{D} = 2^y - 2n$. Let $\{x^*_{uv}\}_{uv \in K(V)}$ be an optimal solution for the LP. We round each value x^*_{uv} down to the nearest integer multiple of $1/\bar{D}$. Let $\bar{x}_{uv}$ be the value obtained after rounding x^*_{uv}. Notice that $\bar{x}_{uv} \geq x^*_{uv} - 1/\bar{D}$.

We define the multigraph $\bar{D} \cdot G = (V, \hat{E}, w)$ where $\hat{E} = \{(\bar{D}\bar{x}_{uv}) \cdot (u, v) \mid (u, v) \in K(V)\}$. The graph $\bar{D} \cdot G$ is well defined, since each $\bar{x}_{uv}$ is a multiple of $1/\bar{D}$. We need the following easy observation.

Lemma 2.4.1 *Let d^+_v be the outdegree of vertex v, and let d^-_v be the indegree of vertex v in $\bar{D} \cdot G$. Then, $\bar{D} - (n-1) \leq d^+_v, d^-_v \leq \bar{D}$.*

Proof: Since we rounded each value x^*_{uv} down to the closest integer multiple of $1/\bar{D}$ we have that

$$\bar{x}_{uv} \geq x^*_{uv} - 1/\bar{D} \ . \tag{2.1}$$

From the definition of $\bar{D} \cdot G$ follows that

$$d^-_v = \bar{D} \sum_u \bar{x}_{uv} \ . \tag{2.2}$$

Substituting inequality (2.1) into inequality (2.2) and using the outdegree constraint $\sum_u x^*_{uv} = 1$ we obtain that

$$d^-_v \geq \bar{D} \sum \left(x^*_{uv} - \frac{1}{\bar{D}} \right) \geq \bar{D} \left(1 - \frac{n-1}{\bar{D}} \right) \ .$$

from which the lower bound on d^-_v follows. The upper bound follows immediately from the fact that $\bar{x}_{vu} \leq x^*_{uv}$. The upper and lower bounds on d^+_v are proved analogously. ∎

As before, because of the 2-cycle constraints, $\bar{D} \cdot G$ contains at most $\bar{D}$ edges (u, v) and (v, u) between every pair of vertices u and v and therefore at most $\lfloor \bar{D}/2 \rfloor$ copies of each 2-cycle.

We want to apply the procedure of Section 2.4.1 to the graph $\bar{D} \cdot G$. In order to do so, we first complete it into a 2^y-regular graph. We have to add edges carefully so that the resulting multigraph is half-bound. Actually, we will maintain a stronger property during the completion, namely we will assure that there are at most 2^y edges (u, v) and (v, u)

between every pair of vertices u and v. We do it in two stages as follows. First we make the graph almost $\bar{D}$-regular in a greedy fashion using the following procedure.

Edge addition stage: As long as there are two distinct vertices i and j such that $d_i^+ < \bar{D}$ and $d_j^- < \bar{D}$, and there are strictly less than $\bar{D}$ edges between i and j we add a directed edge (i, j) to the graph $\bar{D} \cdot G$.

Let G' denote the multigraph when the edge addition phase terminates, then the following holds.

Lemma 2.4.2 *When the edge addition stage terminates, the indegree and outdegree of every vertex in G' is equal to $\bar{D}$ except for at most two vertices i and j. If indeed there are such two vertices i and j, then the total number of edges (i, j) and (j, i) is exactly $\bar{D}$.*

Proof: Assume that the edge addition stage terminates and there are at least three vertices such that for each of them either the indegree or the outdegree is less than $\bar{D}$. Since the sum of the outdegrees of all vertices equal to the sum of the indegrees of all vertices it must be the case that we have at least one vertex, say i, with $d_i^+ < \bar{D}$, and at least one vertex, say $j \neq i$, with $d_j^- < \bar{D}$. Assume that for a third vertex k we have that $d_k^- < \bar{D}$. The case where $d_k^- = \bar{D}$ but $d_k^+ < \bar{D}$ is symmetric. Since the edge addition phase did not add an edge (i, j) although $d_i^+ < \bar{D}$ and $d_j^- < \bar{D}$ we know that the number of edges (i, j) and (j, i) is $\bar{D}$. Similarly, since the edge addition phase did not add an edge (i, k) we know that the number of edges (i, k) and (k, i) is $\bar{D}$. This implies that $d_i^+ + d_i^- = 2\bar{D}$. But since during the edge addition phase we never let any indegree or outdegree to be bigger than $\bar{D}$ we obtain that $d_i^+ = \bar{D}$ in contradiction with our assumption that $d_i^+ < \bar{D}$. ∎

By Lemma 2.4.2 in G' all indegrees and outdegrees equal to $\bar{D}$ except possibly for at most 2 vertices i and j. For i and j we still know by Lemma 2.4.1 that $\bar{D} - (n-1) \leq d_i^+, d_i^-, d_j^+, d_j^-$. In the second stage we pick an arbitrary vertex $k \neq i, j$ and add multiple copies of the edges (k, i), (k, j), (i, k), (j, k) until $d_i^+ = d_i^- = d_j^+ = d_j^- = \bar{D}$. At this point any vertex $v \neq k$ has $d_v^+ = d_v^- = \bar{D}$.

Since we have added at most $2(n-1)$ edges (k, i), (k, j) and at most $2(n-1)$ edges (i, k), (j, k), and $\bar{D} = 2^y - 2n$ we still have $d_k^+, d_k^- < 2^y$. Furthermore, since after the edge addition phase we had at most $\bar{D}$ edges (k, i) and (i, k), and we have added at most $(n-1)$ edges (k, i) and at most $(n-1)$ edges (i, k) we now have at most 2^y edges (i, k) and (k, i). By a similar argument we have at most 2^y edges (j, k) and (k, j). So it is still possible to augment the current graph to a 2^y-regular graph where between any pair of vertices we have at most 2^y edge in both directions.

Notice that since k is now the only vertex whose indegree or outdegree is bigger than $\bar{D}$ we must have that $d_k^+ = d_k^-$. Let $L = d_k^+ = d_k^-$. Clearly $\bar{D} \leq L \leq 2^y$. We finish the construction by adding $L - \bar{D}$ arbitrary cycles that go through all vertices except k, and $2^y - L$ arbitrary Hamiltonian cycles. In all cycles we do not use any of the edges (k, i), (i, k), (j, k), and (k, j). We call the resulting graph G_0. Notice that G must have at least 5 vertices so that such Hamiltonian cycles exist. The following lemma is now obvious.

Lemma 2.4.3 *The graph G_0 is 2^y-regular and satisfies the half-bound invariant.* ■

2.4.2.2 Extracting the two cycle covers

We now apply the procedure of Section 2.4.1 to partition G_0 into two $2^y/2$-regular graphs G' and G''. Repeating the same arguments of Section 2.4.1 both G' and G'' contain each at most $2^y/4$ copies of each 2-cycle. We now let G_1 be the heavier among G' and G'' when solving maximum ATSP and be the lighter graph among G' and G'' when solving minimum ATSP. We apply the same procedure to G_1 to get a $2^y/4$-regular graph G_2. After $y-1$ iterations we get a 2-regular graph G_{y-1}. Let C be the graph G_{y-1}. The indegree and the outdegree of each vertex in C is 2. Using the same arguments of Section 2.4.1, C contains at most one copy of each 2-cycle. The next lemma proves a bound on the weight of C.

Lemma 2.4.4 *Applying the algorithm of Section 2.4.1 to G_0 for maximum ATSP gives a 2-regular graph C such that $w(C) \geq 2OPT$. For minimum ATSP we obtain a 2-regular graph C such that $w(C) \leq 2OPT$.*

Proof: We first prove the statement for maximum ATSP and then indicate the changes required to obtain the statement for minimum TSP.

Let us denote the fractional optimum $\sum_{(u,v)\in E} w(u,v)x^*_{uv}$ by OPT^f. The weight of the graph G_0 is

$$\begin{aligned} w(G_0) &\geq \textstyle\sum_{(u,v)\in E} \bar{D}w(u,v)\bar{x}_{uv} \\ &\geq (2^y - 2n)\left(OPT^f - \textstyle\sum_{(u,v)\in E} w(u,v)/(2^y-2n)\right) \\ &\geq 2^y OPT^f - 2nOPT^f - n^2 W_{max} \\ &\geq 2^y OPT^f - 3n^2 W_{max} \end{aligned}$$

where the second inequality follows since $\bar{x}_{uv} \geq x^*_{uv} - \frac{1}{\bar{D}} = x^*_{uv} - \frac{1}{2^y - 2n}$. At each iteration we retain the heavier graph. Hence, after $y-1$ iterations we get a graph C, such that

$$w(C) \geq \frac{2^y OPT^f - 3n^2 W_{max}}{2^{y-1}} \geq 2OPT^f - \frac{3n^2 W_{max}}{6n^2 W_{max}}$$

The second inequality follows from the fact that $2^y \geq 12n^2 W_{max}$. If we use the fact that $OPT^f \geq OPT$ we get that

$$w(C) \geq 2OPT - \frac{1}{2}\ .$$

Since $w(C)$ and $2OPT$ are integers we obtain that $w(C) \geq 2OPT$.

For minimum ATSP we use the following upper bound on $w(G_0)$.

$$\begin{aligned} w(G_0) &\leq \textstyle\sum_{(u,v)\in E} \bar{D}w(u,v)x^*_{uv} + 3n^2 W_{max} \\ &= \bar{D}OPT^f + 3n^2 W_{max}. \end{aligned}$$

This bound holds since we add at most $3n^2$ edges, n^2 in the edge addition stage and $2n^2$ when adding the Hamiltonian cycles in the last stage. Since we retain the lighter graph in each iteration we obtain that

$$w(C) \leq \frac{\bar{D}OPT^f + 3n^2W_{max}}{2^{y-1}} \leq 2OPT^f + \frac{3n^2W_{max}}{6n^2W_{max}} \leq 2OPT + \frac{1}{2} .$$

Finally, since $w(C)$ and $2OPT$ are integers it follows that $w(C) \leq 2OPT$. ∎

As in Section 2.4.1 it is now straightforward to partition the graph C into 2 cycle covers C_1 and C_2. The running time of the algorithm is $O(n^2y) = O(n^2 \log(nW_{max}))$.

2.5 Minimum Asymmetric TSP

Recall that the *minimum asymmetric TSP (ATSP) problem* is the problem of finding a shortest Hamiltonian cycle in a directed graph with edge weights satisfying the triangle inequality. In this section we design a $\frac{4}{3}\log_3 n$-approximation algorithm for minimum ATSP.[5]

Following the result of the previous section if G has at least 5 vertices we can obtain a graph C which is a union of two cycle covers C_1 and C_2 of G such that (1) $W(C_1)+W(C_2) \leq 2OPT$ and (2) C_1 and C_2 do not share a 2-cycle. Observe that all connected components of C have at least three vertices. Moreover the connected components of C are Eulerian graphs. We may assume that C does not contain two oppositely oriented cycles since if this is the case we may reverse the edges of the heavier cycle without increasing the total weight of C. The graph C remains 2-regular, and therefore we can decompose it into two cycle covers C_1 and C_2 with the same properties as before.

Consider the following deterministic recursive algorithm. Let $G_1 = G$ be the input graph, and let $C^1 = C$, $C_1^1 = C_1$, and $C_2^1 = C_2$. (The superscript indicates the iteration number.) For any graph G, let $N(G)$ be the number of vertices in G, and let $c(G)$ be the number of connected components in graph G. Recall that $w(G)$ is the total weight of the edges of G. Notice that since C_1^1 and C_2^1 are in fact cycle covers, then $c(C_1^1)$ and $c(C_2^2)$ are the number of cycles of C_1^1 and C_1^2, respectively.

Among the graphs C^1, C_1^1 and C_2^1 we choose the one that minimizes the ratio

$$\frac{w(G)}{\log_2(N(G)/c(G))} .$$

In each connected component of the chosen graph, we pick exactly one, arbitrary vertex. Let G_2 be the subgraph of G_1 induced by the chosen vertices. The next iteration proceed in exactly the same way starting from G_2. That is we find an appropriate union of cycle covers $C^2 = C_1^2 \cup C_2^2$ of G_2, pick the one among C^2, C_1^2, and C_2^2 that maximizes the ratio above and so on.[6]

[5] Notice that $4/3 \log_3 n = 4/3 \log_2 n / \log_2 3 \approx 0.841 \log_2 n$.

[6] This algorithm is similar to the algorithm of Frieze, Galbiati and Mafioli [38], who simply discarded a cycle cover at each iteration. We are more careful at the graph which we discard.

This recursion ends when at some iteration p, G_p has less than 27 vertices. In this case we can find an optimal minimum TSP tour T by an exhaustive search and add its edges to our collection of chosen subgraphs. For a technical reason if G_p contains just one vertex we still say that the algorithm has p iterations even if we do not add any edges in the last iteration.

The collection of edges chosen in all steps of this procedure satisfies the following properties. Each edge is chosen in at most one step. Moreover, the edges form a connected Eulerian graph. We obtain our TSP tour by taking an Eulerian tour of this collection of chosen edges, and transforming it into a Hamiltonian cycle by using shortcuts that do not increase the weight, by the triangle inequality.

To calculate the weight of the final Eulerian graph, we note that by the triangle inequality, the weight of the minimum TSP tour in G_i is $\leq OPT$. Hence $w(C_1^i) + w(C_2^i) = w(C^i) \leq 2OPT$. Note also that $N(C^i) = N(C_1^i) = N(C_2^i)$ so we denote this number by n_i. The next property is an upper bound on total number of connected components in the graph considered at step $i = 1, \ldots, p-1$.

Lemma 2.5.1 *$c(C^i) + c(C_1^i) + c(C_2^i) \leq n_i$, for $i = 1, \ldots, p-1$.*

Proof: Consider a connected component A of size k in C^i. We show that this component consists of at most $k-1$ cycles (components) in C_1^i and C_2^i. So together with the component A itself we obtain that the k vertices in A break down to at most k components in C^i, C_1^i, and C_2^i. From this the lemma clearly follows.

Assume first that k is odd. Since k is odd there is at least one cycle in C_1^i between vertices of A that is of length ≥ 3. The same holds for C_2^i. Hence, A breaks into at most $(k-1)/2$ cycles in C_1^i and into at most $(k-1)/2$ cycles in C_2^i, for a total of $k-1$ cycles in both C_1^i and C_2^i.

Consider now the case where k is even. If A breaks down only to 2-cycles in both C_1^i and C_2^i then A forms an oppositely oriented pair of cycles in $C_1^i \cup C_2^i$ which is a contradiction. Hence, without loss of generality, C_1^i contains a cycle of length greater than 2. If it contains a cycle of length exactly 3 then there must be another cycle of odd length 3 or greater since the total size of A is even. However, it is also possible that A in C_1^i contains a single cycle of even length ≥ 4 and all other cycles are of length 2.

Hence, the component A in C_1^i contains at least $\frac{k-4}{2} + 1 = \frac{k-2}{2}$ cycles in case it contains a cycle of length at least 4, or at least $\frac{k-6}{2} + 2 = \frac{k-2}{2}$ cycles in case it contains at least two cycles of length 3. Therefore even in case A breaks into $\frac{k}{2}$ 2-cycles in C_2^i the total number of cycles this component breaks into in both C_1^i and C_2^i is at most $k-1$. ∎

Let p be the number of iterations of the recursive algorithm. Then the total cost of the final Eulerian graph (and therefore the TSP tour) is at most $\sum_{i=1}^{p} w_i$ where w_i is a weight of a graph we chose on iteration i. Recall that $n_i = N(C^i) = N(C_1^i) = N(C_2^i)$, so in particular $n_1 = n$ and $1 \leq n_p \leq 27$. Note also $n_{p-1} \geq 27$, $w_p \leq OPT$ and $w_{p-1} \leq 2OPT$.

If $p \leq 2$ then $\sum_{i=1}^{p} w_i \leq 3OPT$. Otherwise,

$$\frac{\sum_{i=1}^{p-2} w_i}{\log_2(n_1/n_{p-1})} = \frac{\sum_{i=1}^{p-2} w_i}{\sum_{i=1}^{p-2} \log_2(n_i/n_{i+1})} \leq \max_{i=1,\ldots,p-2} \frac{w_i}{\log_2(n_i/n_{i+1})}. \tag{2.3}$$

Consider an iteration j on which maximum of the right hand side of Equation (2.3) is achieved. Since n_{j+1} is equal to the number of connected components in the graph we chose on iteration j we obtain

$$\begin{aligned} &\frac{w_j}{\log_2(n_j/n_{j+1})} && \leq \\ &\min\{\frac{w(C^j)}{\log_2(N(C^j)/c(C^j)}, \frac{w(C_1^j)}{\log_2(N(C_1^j)/c(C_1^j)}, \frac{w(C_2^j)}{\log_2(N(C_2^j)/c(C_2^j)}\} && \leq \\ &\frac{w(C^j)+w(C_1^j)+w(C_2^j)}{\log_2(N^3(C^j)/(c(C^j)\cdot c(C_1^j)\cdot c(C_2^j))} && \leq \frac{4OPT}{3\log_2 3}. \end{aligned}$$

The last inequality follows from the facts that $w(C^j) + w(C_1^j) + w(C_2^j) \leq 4OPT$ and that $c(C^j) \cdot c(C_1^j) \cdot c(C_2^j)$ subject to $0 \leq c(C^j) + c(C_1^j) + c(C_2^j) \leq N(C^j)$ (Lemma 2.5.1) is maximized when $c(C^j) = c(C_1^j) = c(C_2^j) = N(C^j)/3$.

Therefore,

$$\sum_{i=1}^{p} w_i \leq \sum_{i=1}^{p-2} w_i + 3OPT \leq \frac{4\log_3(n_1/n_{p-1})OPT}{3} + 3OPT \leq \left(\frac{4\log_3(n_1)}{3} - 1\right) OPT.$$

To conclude we have the following theorem.

Theorem 2.5.2 *There is a polynomial approximation algorithm for minimum ATSP that produces a tour of weight at most $\frac{4}{3}\log_3 n$ the weight of the optimal tour. The running time of the algorithm is $O(n^2 \log(nW_{max}) \log n)$.*

2.6 Maximum Asymmetric TSP

Consider Lemma 2.8.3. Let C_1 and C_2 be the two cycle covers such that (1) $W(C_1) + W(C_2) \geq 2OPT$ and (2) C_1 and C_2 do not share a 2-cycle. We will show how to partition them into three collections of disjoint paths. One of the path collections will be of weight $\geq \frac{2}{3}OPT$ which we can then patch to obtain a TSP of the desired weight.

Consider the graphs in Figure 2.1, an oppositely oriented pair of 3-cycles and a pair of identical 2-cycles. Obviously, neither graph can be partitioned into 3 sets of disjoint paths. However in C we do not have two copies of the same 2-cycle. Furthermore, as in Section 2.5 if C contains a doubly oriented cycle of length at least 3 we can reverse the direction of the cheaper cycle to get another graph C with the same properties as before and even larger weight. So we may assume that there are no doubly oriented cycles at all. We call

Figure 2.1: A 2 regular directed graph is 3 path colorable iff it does not contain neither of these obstructions: 1) Two 3 cycles oppositely oriented on the same set of vertices. 2) Two copies of the same 2-cycle.

a graph 3-*path-colorable* if we can partition its edges into three sets such that each set induces a collection of node-disjoint paths. We color the edges of the first set *green*, edges of the second set *blue*, and edges of the third set *red*. We prove the following theorem.

Theorem 2.6.1 *Let $G = (V, E)$ be a directed 2-regular multigraph. The graph G is 3-path-colorable if and only if G does not contain two copies of the same 2-cycle or two copies, oppositely oriented, of the same 3-cycle (see Figure 2.1).*

Proof: We consider each connected component of the graph G separately and show that its edges can be partitioned into red, green, and blue paths, such that paths of edges of the same color are node-disjoint.

Consider first a connected component A that consists of two oppositely oriented cycles. Then A must contain at least four vertices v_1, v_2, v_3, v_4 because G does not contain two copies of the same 2-cycle or two oppositely oriented copies of the same 3-cycle (see Figure 2.1). We do the path coloring in the following way. We color the edges (v_2, v_1) and (v_3, v_4) green, the path from v_1 to v_2 blue, and the path from v_4 to v_3 red. So in the rest of the proof we only consider a connected component which is not a union of two oppositely oriented cycles.

As mentioned in Section 2.3 we can decompose a 2-regular digraph G into two cycle covers. Fix any such decomposition and call the first cycle cover *red* and the second cycle cover *blue*. We also call each edge of G either red or blue depending on the cycle cover containing it.

We show how to color some of the red and blue edges *green* such that each color class form a collection of paths. Our algorithm runs in phases where at each phase we find a set of one or more disjoint alternating red-blue paths $P = \{p_1, \ldots, p_k\}$, color the edges on each path $p_j \in P$ green, and remove the edges of any cycle that has a vertex in common with at least one such path from the graph. The algorithm terminates when there are no more cycles and the graph is empty. Let $V(P)$ be the set of vertices of the paths in P, and let $E(P)$ be the set of edges of the paths in P.

The set of paths P has the following key property:

Property 2.6.2 *Each red or blue cycle that intersects $V(P)$ also intersects $E(P)$.*

If indeed we are able to find a set P of alternating paths with this property at each phase then correctness of our algorithm follows: By Property 2.6.2 we color green at least one edge from each red cycle that we remove, and we color green at least one edge from each blue cycle that we remove, so clearly after we remove all cycles, the remaining red edges induce a collection of paths, and the remaining blue edges induce a collection of paths. By the definition of the algorithm after each phase the vertices of $V(P)$ become isolated. Therefore the set of paths colored green in different phases are node-disjoint.

To be more precise, there are cases where the set P picked by our algorithm would violate Property 2.6.2 with respect to one short cycle C. In these cases, in addition to removing P and coloring its edges green we also change the color of an edge on C from red to blue or from blue to red. We carefully pick the edge to recolor so that no red or blue cycles exist in the collection of edges which we remove from the graph.

To construct the set P we use the following algorithm that finds a maximal alternating red-blue path.

Algorithm 2.6.3 (Finding an alternating path) *We first construct a maximal alternating path p in the graph G greedily as follows. We start with any edge (v_1, v_2) such that either $(v_2, v_1) \notin G$ or if $(v_2, v_1) \in G$ then it has the same color as (v_1, v_2). An edge like that must exist on every cycle C. Otherwise there is a cycle oppositely oriented to C, and we assumed that our component is not a pair of oppositely oriented cycles.*

Assume we have already constructed an alternating path $v_1, \ldots, v_k$ with edges of alternating color (blue and red) and the last edge was, say, blue. We continue according to one of the following cases.

1. *If there is no red edge outgoing from v_k we terminate the path at v_k.*

2. *Otherwise, let (v_k, u) be the unique red edge outgoing from v_k, and let C be the red cycle containing it. If C already contains an edge (v_i, v_{i+1}) on p we terminate p at v_k.*

3. *If C does not contain an edge (v_i, v_{i+1}) on p then $u \neq v_i$ for $2 \leq i < k$. Since if $u = v_i$ for some $2 \leq i < k$ it must imply that the edge (v_i, v_{i+1}) is on $C \cap p$ and colored red. If $u \neq v_1$ then we add (v_k, u) to p, define $v_{k+1} = u$, and continue to extend p by performing one of these cases again to extend $v_1, \ldots, v_k, v_{k+1}$.*

4. *If $u = v_1$ we stop extending p.*

If we stopped extending p in Case 1 or Case 2 we continue to extend p backwards in a similar fashion: If we started, say, with a blue edge (v_1, v_2) we look at the unique red edge (u, v_1) (if it exists) and try to add it to p if it does not belong to a cycle that has an edge on p and $u \neq v_k$. We continue using cases symmetric to the four cases described above.

Assume now that $p = v_1, \ldots, v_k$ denotes the path we obtain when the greedy procedure described above stopped extending it in both directions. There are two cases to consider.

1. We stopped extending p both forward and backwards because of Case 1 or Case 2 above. That is, if there is an edge (v_k, v_1) that closes an alternating cycle with p then p already contains some other edge on the same cycle of (v_k, v_1).

2. There is an edge (v_k, v_1) that closes an alternating cycle A with p and belongs to a cycle that does not contain any other edge on p.

The first of these two cases is the easy one. In this case P consists of a single path p, and we can prove the following claim.

Claim 2.6.4 *Let p be an alternating path constructed by Algorithm 2.6.3 and assume that if there is an edge (v_k, v_1) that closes an alternating cycle with p then p already contains some other edge on the same cycle of (v_k, v_1). Then $P = \{p\}$ satisfies Property 2.6.2.*

Proof: Let C be a cycle that intersects p in a vertex v_i. If $1 < i < k$ then C has an edge on p since p contains an edge from both the blue cycle and the red cycle that contain v_i. Assume that C intersects p in v_k. If $(v_{k-1}, v_k) \notin C$ then C contains an edge (v_k, u) of color different from the color of (v_{k-1}, v_k). If p does not contain any edge from C then the algorithm that constructed p should have added (v_k, u) to p which gives a contradiction. We get a similar contradiction in case C intersects p in v_1. ■

To finish the proof of Theorem 2.6.1 we need to consider the harder case is when p together with the edge (v_k, v_1) form an alternating cycle A, and the red or blue cycle containing (v_k, v_1) does not have any other edge in common with p. Note that $|A| \geq 4$ since we started growing p with an edge (z, y) such that there is no edge (y, z) with a different color. We continue according to the algorithm described below that may add several alternating paths to P one at a time. After adding a path it either terminates the phase by coloring P green (and possibly changing the color of one more edge) and deleting all cycles it intersects, or it starts growing another alternating path p. When we grow each of the subsequent paths p we use Algorithm 2.6.3 slightly modified so that it stops in Case 2 if the next edge (v_k, u) is on a cycle C that contains an edge on p, or an edge on a path already in P.

Let p be the most recently constructed alternating path. The algorithm may continue and grow another path only in case when there is an edge on a cycle that has no edges in common with p nor with any previously constructed path in P, that closes p to an alternating cycle A. In case there is no such edge we simply add p to P and terminate the phase. Here is the general step of the algorithm, where A is the most recently found alternating cycle.

1. The alternating cycle A contains an edge (v_i, v_{i+1})[7] on a 2-cycle. Assume that this

[7]Indices of vertices in A are added modulo $|A|$.

2-cycle is blue. The case where the 2-cycle is red is analogous. We let $P := P \cup \{p_1\}$ where p_1 is the part of A from v_{i+1} to v_i. This P does not satisfy Property 2.6.2 since the 2-cycle consisting of the edges (v_i, v_{i+1}), and (v_{i+1}, v_i) has vertices in common with P but no edges. Next we terminate the phase but when we discard P and all cycles intersecting it, we also recolor the edge (v_{i+1}, v_i) red. This keeps the collection of blue and red edges which had been discarded acyclic since it destroys the blue 2-cycle without creating a red cycle: The edges which remain red among the red edges of the red cycle containing (v_{i-1}, v_i), and the edges which remain red among the edges of the red cycle containing (v_{i+1}, v_{i+2}) join to one longer path when we color the edge (v_{i+1}, v_i) red.

2. The alternating cycle A does not contain any edge on a 2-cycle. Let (v, u) and (u, w) be two consecutive edges on A. Assume that (v, u) is red and (u, w) is blue. (The case where (v, u) is blue and (u, w) is red is symmetric.) Let (u, x) be the edge following (v, u) on its red cycle, and let (y, u) be the edge preceding (u, w) on its blue cycle. Notice that since A does not contain any edge on a 2-cycle, and at most one edge from each red or blue cycle, the vertices x, and y are not on A. Now we have two subcases.

   (a) Vertices x and y are different ($x \neq y$). We add the part of A from w to v to P, and we grow another alternating path starting with the edges (y, u) and (u, x). If we end up with another alternating cycle A', and when we process A' we have to perform this case again then we pick on A' a pair of consecutive edges different from (y, u) and (u, x). That would guarantee that at the end P satisfies Property 2.6.2 for all cycles intersecting paths in P, except for at most one cycle intersecting the last path added to P.

   (b) We have $x = y$. Let C be the cycle containing the edges (v, u) and (u, x). If $|C| = 3$ (i.e. C contains the vertices x, v, and u only.) then we add to P the path that starts with the blue edge (x, u) and continue with the alternating path in A from u to v.[8] This set P satisfies Property 2.6.2 with respect to any cycle except C that has vertices in common with the last path we added to A but no edges in P. Next we terminate the phase but when we delete P and color its edges green we also change the color of the edge (x, v) on C from red to blue. This turns C into a path and extends the blue path from w to x by the edge (x, v) and a subpath of the blue cycle containing v. Thus there would be no cycles in the set of red and blue edges which we discard.

   If $|C| > 3$ we add to P the path from u to v on A. Then we grow another alternating path p' starting with the red edge $(x, x') \in C$. The greedy procedure

[8] Note that this path starts with 2 blue edges, so it is not completely alternating. The correctness of our argument however does not depend on the paths being alternating, but on Property 2.6.2.

that constructs an alternating path indeed must end up with a path since the cycle containing the blue edge entering x already has an edge on the path from u to v. We add p' to P and terminate the phase.

■

The proof of Theorem 2.6.1 in fact specifies an algorithm to obtain the partition into three sets of paths. If when we add an edge to an alternating path we mark all the edges on the cycle containing it, then we can implement a phase in time proportional to the number of edges that we discard during the phase. This makes the running time of all the phases $O(n)$.

As mentioned at the beginning of this section, Theorem 2.6.1 together with the algorithm to obtain two cycle covers in Section 2.4 imply the following theorem.

Theorem 2.6.5 *There is a polynomial approximation algorithm for maximum ATSP that produces a tour of length at least $\frac{2}{3}$ of the length of the optimal tour.*

2.7 Maximum Asymmetric TSP with Triangle Inequality

Consider the maximum ATSP problem in a graph G with edge weights that satisfy the triangle inequality, i.e. $w(i,j) + w(j,k) \geq w(i,k)$ for all vertices $i, j, k \in G$.

A weaker version of the following theorem appears in a paper by Serduykov and Kostochka [62]. We provide a simpler proof here for completeness.

Theorem 2.7.1 *Given a cycle cover with cycles $C_1, \ldots, C_k$ in a directed weighted graph G with edge weights satisfying the triangle inequality. We can find in polynomial time an Hamiltonian cycle in G of weight*

$$\sum_{i=1}^{k} \left(1 - \frac{1}{2m_i}\right) w(C_i) \tag{2.4}$$

where m_i is the number of edges in the cycle C_i, and $w(C_i)$ is the weight of cycle C_i.

Proof: Consider the following straightforward randomized algorithm for constructing an Hamiltonian cycle. Discard a random edge from cycle C_i for every i. The remaining edges form k paths. The paths are connected either in order $1, 2, \ldots, k$ or in reverse order $k, k-1, \ldots, 1$ into a cycle. We choose either order at random with probability $1/2$.

We show that the expected weight of this Hamiltonian cycle is $\geq \sum_{i=1}^{k} \left(1 - \frac{1}{2m_i}\right) w(C_i)$. We estimate separately the expected length of edges remaining from the cycle cover and edges added between cycles to connect them to an Hamiltonian cycle. Since we discard

each edge from cycle C_i with probability $1/m_i$ the expected weight of edges remaining from the cycle cover is $\sum_{i=1}^{k}(1-\frac{1}{m_i})w(C_i)$.

Each edge between a vertex of C_i and a vertex of C_{i+1} is used in the Hamiltonian cycle with probability $\frac{1}{2m_im_{i+1}}$. So the expected weight of edges added to connect the paths remaining from the cycle cover is $\sum_{i=1}^{k}\frac{w(V_i,V_{i+1})}{2m_im_{i+1}}$ where $w(V_i,V_{i+1})$ is the total weight of edges between the vertex set of C_i, which we denote by V_i, and the vertex set of C_{i+1}, which we denote by V_{i+1}.

We claim that

$$\frac{w(V_i,V_{i+1})}{2m_im_{i+1}} \geq \frac{w(C_i)}{2m_i}$$

from which our lower bound on the expected weight of the Hamiltonian cycle follows.

To prove this claim we observe that all edges between V_i and V_{i+1} could be partitioned into m_i groups. One group per each edge in C_i. The group corresponding to an edge $(u,v) \in C_i$ contain m_{i+1} pairs of edges from u to some $w \in V_{i+1}$ and from $w \in V_{i+1}$ to v. The total weight of edges in one group is lower bounded by $|V_{i+1}|w_{(u,v)} = m_{i+1}w_{(u,v)}$ by the triangle inequality. ■

The proof of Theorem 2.7.1 suggests a randomized algorithm to compute an Hamiltonian cycle with expected weight at least as large as the sum in Equation (2.4). Since we can compute the expected value of the solution exactly even conditioned on the fact that certain edges must be deleted we can derandomize our algorithm by the standard method of conditional expectations.

We now use Theorem 2.7.1 together with our algorithm to obtain two cycle covers C_1 and C_2 not sharing 2-cycles such that $w(C_1)+w(C_2) \geq 2OPT$, to get the main result of this section.

Theorem 2.7.2 *There is a polynomial approximation algorithm for maximum ATSP in a graph G with edge weight that satisfy the triangle inequality that produces a tour of weight at least $\frac{10}{13}$ the maximum weight of such tour.*

Proof: In Section 2.4 we proved that we can construct in polynomial time two cycle covers C_1 and C_2 not sharing 2-cycles such that $w(C_1)+w(C_2) \geq 2OPT$. Let W_2' be the weight of all the 2-cycles in C_1 and C_2 divided by 2 and let W_3' be the weight of all the other cycles in C_1 and C_2 divided by two. Applying the algorithm to construct an Hamiltonian cycle as in Theorem 2.7.1 to C_1 and C_2 and choosing the Hamiltonian cycle H of largest weight, we obtain from (2.4) that

$$\sum_{e\in H} w(e) \geq \frac{3}{4}W_2' + \frac{5}{6}W_3' \ .$$

We propose another algorithm based on the results of Section 2.4. Let G' be the graph which is the union of all 2-cycles in $C_1 \cup C_2$. Remember that C_1 and C_2 do not share a 2-cycle. Moreover, we can assume, without loss of generality, that C_1 and C_2 do not contain

oppositely oriented cycles. (See Section 2.6.) Hence it follows that G' is a collection of chains consisting of 2-cycles and therefore we can decompose it to two path collections. If we complete these path collections to Hamiltonian cycles and choose the one with larger weight then we obtain an Hamiltonian cycle of weight at least W'_2.

We now take the two algorithms described above and trade off their approximation factor to receive an improved algorithm, i.e., in polynomial time we can construct an Hamiltonian cycle of weight at least

$$\max\{\frac{3}{4}W'_2 + \frac{5}{6}W'_3, W'_2\} \geq \frac{10}{13}OPT \ .$$

The last inequality follows from the fact that $W'_2 + W'_3 \geq OPT$. ■

2.8 Full Decomposition into Cycle Covers

By Hall's theorem one can represent a D-regular multigraph as a combination of at most n^2 cycle covers each taken with an appropriate multiplicity. In this section we prove (constructively) that a D-regular half-bound multigraph, has a similar representation as a linear combination of at most n^2, 2-regular multigraphs (i.e. pairs of cycle covers) that satisfy the half-bound invariant. The sum of the coefficients of the 2-regular multigraphs in this combination is $D/2$.

In particular, we can use this decomposition to extract a pair of cycle covers from a solution of the LP in Section 2.1 with the appropriate weight as we did in Section 2.4. By multiplying the optimal solution x^* of the LP by the least common denominator, D, of all fractions x^*_{uv}, we obtain a half-bound D regular multigraph $D \cdot G$. We then decompose this multigraph into a combination of 2-regular half-bound multigraphs. If the weight of $D \cdot G$ is $\geq D \cdot OPT$ and the sum of the coefficients of the 2-regular multigraphs in this combination is $D/2$, one of the 2-regular graphs must have weight $\geq 2 \cdot OPT$. Similarly if the weight of $D \cdot G$ is $\leq D \cdot OPT$ then one of the 2-regular graphs must have weight $\leq 2 \cdot OPT$.

Let G_0 be an arbitrary half-bound D regular multigraph, where D is even. In Section 2.8.1 we describe an algorithm to extract two cycle covers, C_1 and C_2 from the multigraph that satisfy Property 2.8.1

Property 2.8.1
1) C_1 and C_2 do not share a 2-cycle,
2) every edge that appear in G_0 exactly $D/2$ times appears either in C_1 or in C_2.

This procedure is the core of the decomposition algorithm which we present in Section 2.8.3.

We also show in Section 2.8.2 how to use the algorithm that extracts two cycle covers C_1 and C_2 satisfying the properties above to describe an algorithm that finds two cycle covers

that do not share a 2-cycle and their weight is at least $1/(D/2)$ fraction of the weight of G_0.[9] This without computing the full decomposition of G_0. This method provides a combinatorial alternative to the algorithm of Section 2.4: By applying it to the graph $D \cdot OPT$ where D is the least common denominator of the fractions x^*_{uv} we get the required two cycle covers.

Note that the method of Section 2.4 used a 2^y regular graph where $y = O(\log(nW_{\max}))$. Here the regularity of the graph is proportional to the least common denominator D of x^*_{uv}. The running time of the method we suggest here is $O(n^2 \log(D) \log(nD))$ rather than $O(n^2 \log(nW_{\max}))$ in Section 2.4. I.e. we have double logarithmic dependence in the degrees rather than a single one in Section 2.4. This suggests that the method here is preferable when D is small and the weights are large.

Even if we start out with a half-bound D regular graph G_0 (rather than with a solution to the LP in Section 2.1), we can think of the normalized (by D) multiplicities of the edges as a fractional solution to the LP in Section 2.1 and get a half-bound 2-regular graph using the algorithm in Section 2.4. However notice that this approach can produce a 2-regular graph that contains edges not in G_0. In Section 2.8.2 we show how to get a half-bound 2-regular graph which is a subgraph of G_0.

2.8.1 Finding 2 Cycle Covers

In this section we show how to find in G_0 two cycle covers C_1 and C_2 that satisfy Property 2.8.1.

We build from G_0 a D-regular bipartite graph B as described in Section 2.4.1. In order to find the pair of cycle covers C_1 and C_2 in G_0, we find a pair of perfect matchings M_1 and M_2 in B. We use the technique described by Alon in [2] to find a perfect matching in a D-regular bipartite graph. Let $m = Dn$ be the number of edges of B. Let t be the minimum such that $m \leq 2^t$. We construct a 2^{t+1}-regular bipartite graph B_1 as follows. We replace each edge in B by $\lfloor 2^{t+1}/D \rfloor$ copies of itself. We get a $D\lfloor 2^{t+1}/D \rfloor$- regular graph $B_1 = (V_{B_1}, E_{B_1})$. Recall that D is even so $D\lfloor 2^{t+1}/D \rfloor$ is also even. Let $y = 2^{t+1} - D\lfloor 2^{t+1}/D \rfloor$, clearly y is even. To complete the construction of B_1 we find two perfect matchings M and M', and add $y/2$ copies of M and $y/2$ copies of M' to B_1.

We find M and M' as follows. We define $B' = (V_B, E')$ to be the subgraph of B where $E' = \{(a, b) \in B \mid m_B(a, b) = D/2\}$. Since B is D-regular, the degree of each node of B' is at most two. We complete B' into a 2-regular multigraph A, (we can use edges not contained in B), and we obtain M and M' by partitioning A into two perfect matchings. We call the edges in $M - B'$ and in $M' - B'$ *bad edges*. All other edges in B_1 are *good* edges. Since $y < D$ we have at most Dn bad edges in B_1.

[9] We can similarly find two cycle covers that do not share a 2-cycle and their weight is no larger than $1/D$ fraction of the weight of G_0.

Lemma 2.8.2 *If we have exactly $D/2$ copies of an edge e in B, then we have exactly 2^t good copies of e in B_1. If we have less than $D/2$ copies of an edge e in B, then we have less than 2^t good copies of e in B_1.*

Proof: Suppose we have $D/2$ copies of the edge (a, b) in the graph B, then if we ignore bad edges, by construction (a, b) appears in exactly one of the matchings M, and M'. So the number of instances of (a, b) in B_1 is: $D/2\lfloor 2^{t+1}/D \rfloor + y/2 = \frac{1}{2}(D\lfloor 2^{t+1}/D \rfloor + y) = 2^t$ all of them are good edges. The proof of the second part of this lemma is similar. ∎

Now, using the algorithm of [2] described in Section 2.4.1, we can divide B_1 into two 2^t-regular graphs B' and B''. While doing the partitioning, we balance as much as possible the number of good copies of each edge that go to each of the two parts. Let B_2 be the one among B' and B'' containing at most half of the bad edges of B_1. So B_2 contains at most $\lfloor Dn/2 \rfloor$ bad edges. Applying the same algorithm to B_2 we get a 2^{t-1} regular graph B_3 that contains at most $\lfloor Dn/4 \rfloor$ bad edges. After t iterations we get a $2^{t+1-t} = 2$-regular graph B_{t+1}, containing at most $\lfloor Dn/2^t \rfloor = 0$ bad edges. We partition B_{t+1} into the matchings M_1 and M_2. The matchings M_1 and M_2 contain only edges from B and therefore correspond to cycle covers C_1, C_2 in G_0.

Lemma 2.8.3 *The cycle covers C_1 and C_2 satisfy Property 2.8.1.*

Proof: Since a 2-cycle $C_{u,v}$ appears in G_0 at most $D/2$ times, then at least one of the edges (u, v), or (v, u) appears at most $D/2$ times in G_0. By Lemma 2.8.2 this edge has at most 2^t good copies in B_1. Since we partition the good copies of each edge as evenly as possible the algorithm ensures that an edge that has at most 2^t good copies in B_1 will appear at most once in B_{t+1}. Therefore B_{t+1} will contain at most one instance of one of the edges of $C_{u,v}$ so there is at most one copy of $C_{u,v}$ in B_{t+1}.

Suppose we have $D/2$ copies of e in G_0, then by Lemma 2.8.2, this edge will have exactly 2^t good copies in B_1. The algorithm ensures that an edge that has exactly 2^t good copies in B_1 will appear exactly once in B_{t+1}. ∎

Since the regularity of the graph we start out with is $O(Dn)$, the algorithm performs $O(\log(Dn))$ iterations. Each iteration takes $O(\min\{Dn, n^2\})$ time, since the number of distinct edges in G_0 is $O(\min\{Dn, n^2\})$.
So to conclude the overall running time is $O(\min\{Dn, n^2\} \log(Dn)) = O(n^2 \log(Dn))$.

2.8.2 Finding 2 Cycle Covers with at least the average weight

In this section we show how to find in a D regular multigraph G_0 a half-bound 2-regular subgraph whose weight is at least the weight of all edges in G_0 divided by $D/2$. An analogous algorithm can find a half-bound 2-regular subgraph whose weight is at most the weight of all edges in G_0 divided by $D/2$. This procedure is a combinatorial alternative to the algorithm in Section 2.4.2 when we apply it to the multigraph obtained by multiplying the solution to the LP of section 2.1 by the least common denominator of all fractions x^*_{uv}.

The algorithm is recursive. In each iteration we obtain a multigraph with smaller regularity while keeping the properties of the original multigraph. We also keep the average weight nondecreasing so at the end we obtain a 2-regular half-bound multigraph with average weight as required.

We assume that D is even (otherwise we multiply by two the number of copies of each edge of G_0 to make D even). If $D \bmod 4 = 0$, then we partition G_0 into two $D/2$-regular multigraphs, G' and G'', as described in Section 2.4.1. We let G_1 be the heavier among G' and G''. So G_1 is a $D/2$-regular graph ($D/2$ is even), G_1 is half-bound, i.e. it contains at most $D/4$ copies of each 2-cycle, and $w(G_1) \geq \frac{1}{2}w(G_0)$ (so $\frac{w(G_1)}{D/4} \geq \frac{w(G_0)}{D/2}$).

If $D \bmod 4 = 2$, then we find two cycle covers C_1 and C_2 in G_0 that satisfy Property 2.8.1 using the algorithm in Section 2.8.1.

If $w(C_1)+w(C_2) \geq \frac{w(G_0)}{D/2}$, then C_1 and C_2 together are the half-bound 2-regular graph we wanted to find and the process ends. If $w(C_1)+w(C_2) < \frac{w(G_0)}{D/2}$, then let $G_1 = G_0 - C_1 - C_2$. Notice that G_1 is a $(D-2)$-regular graph, $(D-2) = 0 \bmod 4$, and $w(G_1) = w(G_0)-(w(C_1)+w(C_2)) \geq w(G_0) - \frac{w(G_0)}{D/2} = \frac{D-2}{D}w(G_0)$. So $\frac{w(G_1)}{(D-2)/2} \geq \frac{w(G_0)}{D/2}$. The next lemma shows that G_1 contains at most $\frac{D-2}{2}$ copies of each 2-cycle.

Lemma 2.8.4 *G_1 contains at most $\frac{D-2}{2}$ copies of each 2-cycle.*

Proof: Suppose that G_1 contains more than $\frac{D-2}{2}$ copies of some 2-cycle $(u, v), (v, u)$. Then G_1 and hence also G_0 contain exactly $D/2$ copies of this 2-cycle. (They cannot contain more by our assumption on G_0.) But then one of the edges (u, v), or (v, u) appears exactly $D/2$ times in G_0 and therefore is contained in exactly one of C_1 or C_2 so G_1 must have less than $D/2$ copies of it, which gives a contradiction. ∎

We next repeat the step just described with G_1. Since the average weight of G_1 is no smaller than that of G_0 it follows that if we continue and iterate this process then we will end up with a 2-regular multigraph whose weight is at least $\frac{w(G_0)}{D/2}$. Furthermore since we keep the multigraphs half-bound the resulting 2-regular graph is also half-bound. The number of iterations required is $O(\log D)$ since the regularity of the graph drops by a factor of at least 2 every other iteration. From Section 2.8.1 we know that the time it takes to find the two cycle covers when $D \bmod 4 = 2$ is $O(n^2 \log(Dn))$. Therefore the total running time of the algorithm is $O(n^2 \log(Dn) \log D)$.

2.8.3 Succinct Convex Combination Representation

We now show how to use the algorithm of Section 2.8.1 to decompose a D-regular half-bound multigraph into 2-regular half-bound multigraphs (pairs of cycle covers which do not share a 2-cycle).

For a multigraph G, we define the matrix $A(G)$ as $a_{i,j} = m_G(i, j)$, (entry $a_{i,j}$ of $A(G)$ contains the multiplicity of the edge (i, j) in G). Let G be D regular, containing at most

$\lfloor D/2 \rfloor$ copies of each 2-cycle. We partition G into pairs of cycle covers, $P_i = C_{i_1}, C_{i_2}$, where $1 \leq i \leq k$, and $k \leq n^2$, such that

1. $A(G) = \sum_{i=1}^{k} c_i A(P_i)$, where $2\sum_{i=1}^{k} c_i = D$, and $c_i \geq 0$.
2. For every $1 \leq i \leq k$, C_{i_1} and C_{i_2} do not share a 2-cycle, i.e P_i is a half-bound 2-regular graph.

We find the pairs of cycle covers $P_1, \ldots, P_k$ iteratively in k iterations. After iteration i, $1 \leq i \leq k$, we have already computed $P_1, \ldots, P_i$ and $c_1, \ldots, c_i$, and we also have a D_i regular multigraph G_i, and an integer x_i such that the following invariant holds.

Invariant 2.8.5

1. *D_i is even, and furthermore for each $e \in G_i$, $m_{G_i}(e)$ is even.*
2. *G_i contains at most $D_i/2$ copies of each 2-cycle.*
3. *G is a linear combination of the pairs of cycle covers we found so far and G_i, namely, $A(G) = \sum_{j=1}^{i} c_j A(P_j) + \frac{1}{2^{x_i}} A(G_i)$*

We define G_0 to be G, and $x_0 = 0$ if all the multiplicities of the edges of G are even. Otherwise we define $G_0 = 2 \cdot G$ and $x_0 = 1$. This guarantees that Invariant 2.8.5 holds before iteration 1 with $i = 0$. In iteration $i + 1$, $i \geq 0$ we perform the following.

1. We use the procedure described in Section 2.8.1 to find two cycle covers C_1 and C_2 in G_i such that
   (a) C_1 and C_2 do not share a 2-cycle.
   (b) Every edge that appears in G_i exactly $D_i/2$ times, will appear in exactly one of the cycle covers C_1 or C_2.
2. We define $G_{i+1} = G_i - c(C_1 + C_2)$ for an appropriately chosen c. We will show that G_{i+1} satisfies the properties of Invariant 2.8.5(2) and Invariant 2.8.5(3), where $P_{i+1} = \{C_1, C_2\}$, $c_{i+1} = c/2^{x_i}$, and $x_{i+1} = x_i$.
3. If there is an edge $e \in G_{i+1}$ such that $m_{G_{i+1}}(e)$ is odd, we increment x_{i+1} and multiply by 2 the number of copies of each edge in G_{i+1}. Now G_{i+1} also satisfies Invariant 2.8.5(1).

We now assume that we have a graph G_i that meets the conditions of Invariant 2.8.5. We use the procedure of Section 2.8.1 to find two cycle covers C_1, C_2 that meet the conditions of item 1. Next we show how to calculate the integer c such that if we remove from G_i, c copies of C_1 and c copies of C_2 then we obtain a graph G_{i+1} that also satisfies Invariant 2.8.5(2) and Invariant 2.8.5(3), where $P_{i+1} = \{C_1, C_2\}$ and $c_{i+1} = c/2^{x_i}$.

By Invariant 2.8.5 all the entries of $A(G_i)$ consists of even integers. Let c' be the minimum such that if we remove c' copies of C_1 and c' copies of C_2 from G_i we zero an entry in the matrix $A(G_i)$. Since all entries of $A(G_i)$ are even integers, c' is also an integer (c' may be odd).

We define the multiplicity, $\ell(a,b)$, of a 2-cycle $C_{a,b}$ in G_i to be the minimum between multiplicity of (a,b) and multiplicity of (b,a). By deleting one copy of C_1 and C_2 we decrease the multiplicity of each 2-cycle $C_{a,b}$ whose edge with smaller multiplicity (or just any edge if they both have the same multiplicity), is contained either in C_1 or in C_2. Since C_1 or C_2 contain a copy of each edge with multiplicity $D_i/2$ it follows that we decrease the multiplicity of each cycle whose multiplicity is exactly $D_i/2$. Deleting a copy of C_1 and C_2 may not decrease the multiplicity of cycles whose multiplicity is smaller than $D/2$. We denote the largest multiplicity of a 2-cycle whose multiplicity does not decrease by deleting a copy of C_1 and C_2 by max_c.

If $D_i - 2c' > 2max_c$, we set $c = c'$. If $D_i - 2c' \leq 2max_c$ we set $c = (D_i - 2max_c)/2$. In both cases c is a positive integer. Let G_{i+1} be the graph that corresponds to the matrix $A' = A(G_i) - c(A(C_1) + A(C_2))$. The graph G_{i+1} is well defined since the definition of c guarantees that all entries of A' are nonnegative. The following lemma implies the correctness of the algorithm.

Lemma 2.8.6 *G_{i+1} satisfies Invariant 2.8.5(2) and Invariant 2.8.5(3).*

Proof: Let $D_{i+1} = D_i - 2c$. Clearly G_{i+1} is a D_{i+1}-regular graph. Since c is an integer, and D_i is even, D_{i+1} is also even. Since for each $e \in G_{i+1}$, $m_{G_{i+1}}(e) \in \{m_{G_i}(e), m_{G_i}(e) - c, m_{G_i}(e) - 2c\}$, $m_{G_{i+1}}(e)$ is a non-negative integer. At the end of iteration $i+1$, G_i and x_{i+1} satisfy that

$$G'_0 = \sum_{j=1}^{i} c_j P_j + \frac{1}{2^{x_{i+1}}} G_i =$$

$$\sum_{j=1}^{i} c_j P_j + \frac{1}{2^{x_{i+1}}}(c(C_1 + C_2) + G_{i+1}) = \sum_{j=1}^{i+1} c_j P_j + \frac{1}{2^{x_{i+1}}} G_{i+1}$$

Next we show that for any two nodes $u, v \in G_{i+1}$ the number of 2-cycles $C_{u,v}$ in G_{i+1} is at most $D_{i+1}/2$. By construction, $D_{i+1}/2 = D_i/2 - c \geq max_c$. By the definition of max_c and of C_1 and C_2, each 2-cycle $C_{a,b}$ with $\ell(a,b) > max_c$ has at least one edge $e \in C_{a,b}$ with $m_{G_i}(e) = x \leq D_i/2$ in one of the two cycle covers C_1, C_2. So after creating G_{i+1} we have at most $x - c \leq D_i/2 - c = D_{i+1}/2$ instances of e in G_{i+1}. ∎

It is straightforward to check that after the last step of the algorithm G_{i+1} also satisfies Invariant 2.8.5 (1). We continue doing these iterations as long as G_i is not empty. The next lemma shows that G_i gets empty after at most n^2 iterations and at that point by Invariant 2.8.5 we have a partition of G' into pairs of cycle covers.

Lemma 2.8.7 *After at most $n^2 - n + 1$ iterations G_i becomes empty.*

Proof: Since G_0 does not contain self loops, $A(G_0)$ contains at most n^2-n non-zero entries. We will ignore the n zero entries of the self loops, and we will refer only to the n^2-n entries of the edges of G_0. Assume for a contradiction that G_{n^2-n+1} is not empty.

Let U_i be the set of edges (a,b) of G_i such that $m_{G_i}(a,b) = D_i/2$. Notice that if $(a,b) \in U_i$ then by lemma 2.8.3, it will appear in exactly one of the two cycle covers C_1, and C_2 that we find in iteration $i+1$. Then when we remove c copies of C_1 and c copies of C_2 from G_i we get a $D_i - 2c = D_{i+1}$-regular graph, G_{i+1}, in which the edge (a,b) appears $D_i/2 - c = D_{i+1}/2$ times. So once an edge belongs to U_i it stays in U_k for every $k \geq i$ until it disappears from some G_k. Moreover, if an edge $(a,b) \in U_i$ is zeroed in iteration $i+1$, then $c = D_i/2$ and $D_{i+1} = D_i - 2c = 0$. So once an edge $(a,b) \in U_i$ disappears from the graph the whole graph is zeroed. Thus if $i+1$ is not the last iteration, all edges that are zeroed at iteration $i+1$ do not belong to U_i.

It is also easy to verify that in each iteration in which we do not zero an entry of the matrix $A(G_i)$, U_i is increased by at least one. Let j be the cardinality of U_{n^2-n}, then $j \leq n^2 - n$. In each iteration i in which the cardinality of U_i didn't increase we zeroed an entry that didn't belong to U_i. Thus at the end of iteration $n^2 - n$, we zeroed at least $n^2 - n - j$ entries, and were left with j entries that belong to U_{n^2-n}. Since all non-zero entries of $A(G_{n^2-n})$ belong to U_{n^2-n}, their value is $D_{(n^2-n)/2}$, and in the next iteration all of these entries will be zeroed, and G_{n^2-n+1} must be empty. ∎

The algorithm consists of at most n^2 iterations, and each iteration takes $O(n^2(\log(Dn)))$ time. Therefore we find the complete decomposition in $O(n^4(\log(Dn)))$ time.

2.9 Concluding Remarks

Chen and Nagoya [21] used our framework to improve the approximation ratio for maximum asymmetric TSP with triangle inequality from 10/13 to 27/35. Bläser, Shankar and Sviridenko [16] further improved the bound to 31/40, using a different technique to round the LP for cycle cover with two cycle constraints. Finally, Kowalik and Mucha [63], used our framework and improved the approximation ratio to this problem to 35/44.

The approximation ratio for minimum asymmetric TSP with triangle inequality was improved form $4/3 \log_3 n$ to $2/3 \log_2 n$, by Feige and Singh [37] by modifying our algorithm.

Our approximation algorithms are based on rounding a linear program for cycle cover without cycles of length two. It may be interesting to see if using a linear program for cycles cover without cycles of length two and without cycles of length three (and so on), may help obtain better approximation algorithms. Another interesting direction is getting these bounds without using linear programming.

Chapter 3

The Shortest Superstring

3.1 Introduction

In the shortest superstring problem the input is a set $S = s_1, \cdots, s_n$ of strings and we seek the shortest possible string s such that every string in S is a (contiguous) substring of s. This problem is known to be NP-hard and even MAX-SNP hard [18]. The best known approximation algorithm finds a string whose length is at most 2.5 times the length of the optimal string [50, 80]. The superstring problem has applications in data compression and in DNA sequencing. For example, in a shotgun DNA sequencing, a DNA molecule can be represented as a string over the set of nucleotides $\{A, C, G, T\}$. Only Small overlapping fragments of the DNA molecule can be sequenced at a time. So the DNA sequence has to be reconstructed from these fragments. We can model this as a shortest superstring problem in which each string in S represents a sequenced DNA fragment, and a shortest superstring of S is the DNA sequence representation for the whole DNA molecule.

There is a natural greedy algorithm for the shortest superstring problem, which we refer to as GREEDY. The GREEDY algorithm maintains a set of strings, initialized to be equal to S. At each iteration GREEDY picks two strings with maximum overlap from its set, combines them into one string, which it then puts back into the set. Blum *et al.* [18] proved that the length of the string produced by GREEDY is within a factor of 4 from optimal. Blum *et al.* [18] also gave the input $S = \{c(ab)^k, (ba)^k, (ab)^k c\}$ on which GREEDY produces a sequence twice as long as the optimal. It's easy to see that GREEDY will first join $c(ab)^k$ and $(ab)^k c$ to create the string $c(ab)^k c$, and then it'll concatenate $c(ab)^k c$ with $(ba)^k$ to get the string $c(ab)^k c(ba)^k$ of length $4k + 2$. The optimal superstring however, is $c(ab)^{k+1}c$ and its length is $2k + 4$. Blum *et al.* [18] conjectured that indeed the approximation guarantee of GREEDY is 2.

Zaritsky and Sipper [88] presented results of experiments that support the conjecture that the string produced by GREEDY is of length at most 2 times the length of the shortest superstring. Zaritsky and Sipper compared the performance of several algorithms for the

String length	Number of blocks	Average length of the superstring produced by GREEDY.
250	50	381
400	80	596
450	90	677
500	100	768

Table 3.1: The first column is the random string length, the second column is the number of blocks (number of input strings for GREEDY), the last column is the average length of the superstring produced by GREEDY. In the first two rows the average is computed over 50 random strings, in the last two rows the average is over 20 random strings.

shortest superstring problem to that of the GREEDY algorithm. In their experiments they generated a random binary string, took 5 disjoint copies of it, and divided each copy into blocks (substrings) of random length between 20 and 30 bits. The blocks were the input to the superstring algorithms. Since the string was randomly generated it is likely that the length of the shortest superstring of these blocks is close to the length of the original random string. The results of these experiments with respect to the GREEDY algorithm are presented in Table 3.1. In all of their experiments the average length of the string produced by GREEDY is less than 2 times the length of the random string.

Recently, Bim Ma [70], showed that for a given instance I of the shortest superstring problem, the average approximation ratio of the greedy algorithm on a random perturbation of I, which is relatively close to I is $1 + o(1)$. This also implies that in practice the approximation ratio of the Greedy algorithm is usually very good.

Despite a considerable progress achieved since the work of Blum *et al.* on designing approximation algorithms other than GREEDY with better approximation guarantees, (see [82, 34, 61, 8, 9, 20, 80, 50]), there has been no progress since on narrowing the gap regarding the approximation guarantee of GREEDY. Many of the approximation algorithms for the superstring problem are more complicated and less efficient than GREEDY. This makes it important to know whether the approximation ratio of GREEDY is in fact better than that of the other approximation algorithms.

In this thesis we prove that the string produced by GREEDY is in fact within a factor of 3.5 from the optimal string. To get this result, we use the "overlap-rotation lemma" of Breslauer, Jiang and Jiang (see [20], Section 3), to tighten the upper bound on the total length of the so called "culprit bad back edges" in the proof of Blum *et al.* [18].

Finally, in Section 3.4 we show how to get a 2.5-approximation algorithm for the shortest superstring, using our approximation algorithm for maximum asymmetric TSP (see Chapter 2), together with the reduction of Breslauer *et al.* [20].

3.2 Preliminaries

We assume that no string in S contains another and denote the length of the shortest superstring by $OPT(S)$. We first give some basic definitions. For two strings s and t, we call $ov(s,t)$ the amount of overlap of s with respect to t. I.e. $ov(s,t)$ is the length of the longest string y such that $s = xy$ and $t = yz$ for non-empty strings x and z. We define $pref(s,t)$ to be the string x, and $d(s,t)$ to be $|pref(s,t)|$. These definitions imply that $|s| = d(s,t) + ov(s,t)$.

We denote by $s =< s_{i_1}, \cdots, s_{i_k} >$ the string $pref(s_{i_1}, s_{i_2})pref(s_{i_2}, s_{i_3}) \cdots pref(s_{i_{k-1}}, s_{i_k})s_{i_k}$. String s is a superstring of $s_{i_1}, \cdots, s_{i_k}$. In fact s is the shortest superstring string in which $s_{i_1}, \cdots, s_{i_k}$ appear such that s_{i_j} starts before $s_{i_{j+1}}$ for every $1 \leq j \leq k-1$. So clearly, the shortest superstring of S must be $< s_{i_1}, \cdots, s_{i_n} >$, for some permutation of the strings $s_1, \cdots, s_n$. Notice that, $OPT(S) = \sum_{j=1}^{n} |s_{i_j}| - \sum_{j=1}^{n-1} ov(s_{i_j}, s_{i_{j+1}})$, and we get the following equality.

$$\sum_{i=1}^{n} |s_i| = OPT(S) + \sum_{j=1}^{n-1} ov(s_{i_j}, s_{i_{j+1}}). \tag{3.1}$$

With this terminology the definition of GREEDY is as follows. We initialize a set of strings to contain all the input strings. In each iteration we remove s and t ($s \neq t$) from S where s and t are such that $ov(s,t)$ is as large as possible and add back to S the string $< s,t >= pref(s,t)t$. Greedy terminates when there is only one string in S.

The distance graph $G_S = (V, E, w)$ is a complete directed graph ($E = \{(u,v) | u, v \in V\}$). The set of vertices V is the set of strings, $s_1, \cdots, s_n$. The weight of an edge (s_i, s_j), is $d(s_i, s_j)$. The *overlap graph* is similar to the distance graph where the weight of an edge (s_i, s_j) is $ov(s_i, s_j)$ rather than $d(s_i, s_j)$. Let $c = s_{i_1}, s_{i_2}, \ldots, s_{i_k}$ be a cycle in G_S. Cycle c corresponds to the string $pref(s_{i_1}, s_{i_2})pref(s_{i_2}, s_{i_3}) \cdots pref(s_{i_{k-1}}, s_{i_k})pref(s_{i_k}, s_{i_1})$. The weight of c in G_S is $w(c) = d(s_{i_1}, s_{i_2}) + d(s_{i_2}, s_{i_3}) + \cdots + d(s_{i_k}, s_{i_1})$. Notice that,

$$| < s_{i_1}, \cdots, s_{i_k} > | = d(s_{i_1}, s_{i_2}) + d(s_{i_2}, s_{i_3}) + \cdots + d(s_{i_{k-1}}, s_{i_k}) + |s_{i_k}|$$

$$= d(s_{i_1}, s_{i_2}) + d(s_{i_2}, s_{i_3}) + \cdots + d(s_{i_{k-1}}, s_{i_k}) + d(s_{i_k}, s_{i_1}) + ov(s_{i_k}, s_{i_1}) = w(c) + ov(s_{i_k}, s_{i_1}).$$

Let $s =< s_{i_1}, \cdots, s_{i_n} >$ be the shortest superstring of S, and let $c' = s_{i_1}, s_{i_2}, \ldots, s_{i_n}$ be the corresponding cycle in G_S, then $w(c') = OPT(S) - ov(s_{i_n}, s_{i_1})$. It follows that if $TSP(G_S)$ is the cost of a minimum weight Hamiltonian cycle in G_S, then $TSP(G_S) \leq w(c') \leq OPT(S)$. We denote the minimum weight cycle cover (CC) in a graph G by $CYC(G)$. Then we have that $w(CYC(G_S)) \leq TSP(G_S) \leq OPT(S)$.

We now briefly describe the connection between the minimum asymmetric TSP problem and the shortest superstring problem. Let $c' = s_{i_1}, s_{i_2}, \ldots, s_{i_{n-1}}, s_{i_n}$ be the minimum weight cycle in G_S, then we can obtain a superstring s' out of c' by "opening" c' at some edge, say $(s_{i_{n-1}}, s_{i_n})$. We have that $|s'| = TSP(G_S) + ov(s_{i_{n-1}}, s_{i_n}) \leq 2OPT(S)$. This shows that a c-approximation to the minimum asymmetric TSP problem yields a $2c$-approximation to the

shortest superstring problem. However since there is no known constant approximation to the minimum asymmetric TSP problem, (see Chapter 2), we do not use it to approximate the shortest superstring problem.

We now define the *root*[1] of a string and other related terms that are used in [20]. For a string s, $root(s)$ is the shortest string x such that $s = x^i y$, and y is a prefix of x, (y may be empty). We denote $period(s)$ to be $|root(s)|$. A semi-infinite string $s = a_1 a_2 \cdots$ is *periodic* if $s = xs$ for some nonempty string x. The shortest string x such that $s = xs$ is the *root* of s, denoted by $root(s)$. As for finite strings we also define $period(s) = |root(s)|$. The semi-infinite strings we use in the followings lemmas are obtained by breaking a cycle c at a certain position j. Let $c = s_{i_1}, s_{i_2}, \ldots, s_{i_k}$ be a cycle in G_S, then we if we break the cycle at some index j we get a string $u = pref(s_{i_j}, s_{i_{j+1}})pref(s_{i_{j+1}}, s_{i_{j+2}}) \cdots pref(s_{i_{k-1}}, s_{i_k})pref(s_{i_k}, s_{i_1}) \cdots pref(s_{i_{j-1}}, s_{i_j})$. The string $uuuu \cdots$ is the semi-infinite string obtained from the cycle c by breaking it at position j.

Let s and t be two strings either finite or periodic semi-infinite, we call s and t *equivalent* if $root(t)$ is a cyclic shift $root(s)$. I.e. there are strings x, y such that $root(s) = xy$ and $root(t) = yx$. Otherwise they are *inequivalent*. The following lemmas are proved in [18], and were restated in [20].

Lemma 3.2.1 *Let $c = s_{i_1}, s_{i_2}, \ldots, s_{i_k}$ be a cycle in $CYC(G_S)$, then $root(< s_{i_1}, \cdots, s_{i_k} >) = root(< s_{i_1}, \cdots, s_{i_k}, s_{i_1} >) = pref(s_{i_1}, s_{i_2}) \cdots pref(s_{i_{k-1}}, s_{i_k})pref(s_{i_k}, s_{i_1})$, and $period(< s_{i_1}, \cdots, s_{i_k}, s_{i_1} >) = w(c)$.*

Notice that the strings s_{i_j} on the cycles c are not necessarily equivalent, to each other and to $< s_{i_1}, \cdots, s_{i_k} >$.

Lemma 3.2.2 *Let $c = s_{i_1}, s_{i_2}, \ldots, s_{i_k}$ and $c' = s_{j_1}, s_{j_2}, \ldots, s_{j_l}$ be two different cycles in $CYC(G_S)$, then $< s_{i_1}, \cdots, s_{i_k} >$ is inequivalent to $< s_{j_1}, \cdots, s_{j_l} >$.*

The following lemma was used to get the bounds in [18].

Lemma 3.2.3 *If s, t are inequivalent then $ov(s, t) \leq period(s) + period(t)$.*

Lemma 3.2.1, Lemma 3.2.2 and Lemma 3.2.3 imply that if $c = s_{i_1}, s_{i_2}, \ldots, s_{i_k}$ and $c' = s_{j_1}, s_{j_2}, \ldots, s_{j_l}$ are two different cycles in $CYC(G_S)$, then

$$ov(< s_{i_1}, \cdots, s_{i_k} >, < s_{j_1}, \cdots, s_{j_l} >) \leq w(c) + w(c'). \tag{3.2}$$

Given a periodic semi-infinite string $\alpha = a_1 a_2, \ldots$, let $\alpha[k]$ be the string $a_k a_{k+1} \cdots$. We now state the overlap rotation lemma proved in [20].

[1] In [20], they used the term factor instead of root

Lemma 3.2.4 *[overlap rotation lemma] Let α be a periodic semi-infinite string. There exists an integer $k \leq period(\alpha)$ such that for any finite string s that is inequivalent to α*

$$ov(s, \alpha[k]) < period(s) + \frac{1}{2}period(\alpha).$$

The following Lemma is also from [20]. It shows based on the overlap rotation lemma how we can derive a string from each cycle in the cycle cover such that these strings have pairwise small overlaps (smaller than the overlap in Inequality (3.2)).

Lemma 3.2.5 *Let $c = s_{i_1}, \cdots, s_{i_r}$ be a cycle in $CYC(G_S)$, then there is a string t_c and an index j such that*

1. *The string $< s_{i_{j+1}}, \cdots, s_{i_r}, s_{i_1}, \cdots, s_{i_j} >$ is a suffix of the string t_c.*
2. *String t_c is contained in $s_c =< s_{i_j}, \cdots, s_{i_r}, s_{i_1}, \cdots, s_{i_j} >$*
3. *$root(t_c)$ is equivalent to $root(< s_{i_{j+1}}, \cdots, s_{i_r}, s_{i_1}, \cdots, s_{i_j} >)$.*
4. *The semi-infinite string $u = root(t_c)root(t_c)\cdots$ is the desired string $\alpha[k]$ in Lemma 3.2.4, where α is any semi-infinite string defined by the cycle c. Specifically, Let x be the string produced by this lemma that corresponds to a different cycle $c' \in CYC(G_S)$, then $ov(x, t_c) \leq w(c') + \frac{1}{2}w(c)$.*

The next lemma gives a bound on the length of the superstring for the set that consists of strings chosen according to Lemma 3.2.5 for each cycle in $CYC(G_S)$. It easily follows from Property 2 in Lemma 3.2.5.

Lemma 3.2.6 *Let $A = \{t_c \mid c \in CYC(G_s)$ t_c is the string guaranteed to exist by to Lemma 3.2.5$\}$. Then $OPT(A) \leq OPT(S) + w(CYC(G_S))$.*

Proof: Let $B = \{s_c \mid c \in CYC(G_S)$ s_c is chosen according to Property 2 of lemma 3.2.5$\}$. Since each string $t_c \in A$ is contained in the string $s_c \in B$, a superstring for the set B is also a superstring of the set A. In [18], Blum *et al.* proved that $OPT(B) \leq OPT(S) + w(CYC(G_S))$, so it follows that $OPT(A) \leq OPT(B) \leq OPT(S) + w(CYC(G_S))$. ∎

We now mention some definitions and details from the 4-approximation proof of GREEDY in [18]. For this proof, Blum *et al.* [18] introduced another algorithm called MGREEDY and proved that it finds a minimum cycle cover on G_S. MGREEDY is similar to GREEDY except that when there is a string $t \in S$ such that $ov(t, t) > ov(s', t')$ for any two strings $s', t' \in S$, MGREEEDY removes t from S, and continues with the rest of the strings. Each of the strings extracted from S in this process defines a cycle in a minimum cycle cover.

Blum *et al.* looked at the GREEDY algorithm as taking a list of all edges in the overlap graph sorted in decreasing order by their overlap, and going down the list deciding for each edge whether to include it or not. For the rest of the paper assume that the strings are

renumbered such that if s' is the string produced by GREEDY, then $s' =< s_1, s_2, \cdots, s_n >$. We say that an edge e *dominates* an edge f if e comes before f in the sorted list (I.e. $ov(e) \geq ov(f)$), and e share its head or tail with f. It is easy to verify that GREEDY doesn't include an edge f if either

1. f is dominated by an already chosen edge e.
2. f is not dominated but would form a cycle with the strings already chosen.

If f wasn't chosen because of the latter reason, then f is a *bad back edge*. Let $f = (s_j, s_i)$ be a bad back edge, then since f closes a cycle with the strings already chosen by GREEDY, f corresponds to a string $< s_i, s_{i+1}, \cdots, s_j >$. When GREEDY considers f it had already chosen all edges on the path from i to j. Thus the overlap of all edges on the path from i to j is greater or equal to $ov(f)$. Also, when GREEDY considers f it had not yet chosen the edges (s_{i-1}, s_i) and (s_j, s_{j+1}). (Otherwise f wouldn't be chosen because of the first reason). We say that the edge f *spans* the closed interval $I_f[i, j]$. Blum *et al.* proved the following lemma.

Lemma 3.2.7 *Let e and f be two bad back edges. Then the closed intervals I_e, and I_f, spanned by e and f respectively are either disjoint or one contains the other.*

Thus intervals of bad back edges do not cross each other. *Culprits* are minimal (with respect to containment) such intervals. By definition, the intervals of all culprits are disjoint. Each culprit $[i, j]$ corresponds to a string $< s_i, \cdots, s_j >$. As mentioned above the edge (s_j, s_i) has the lowest overlap among the edges of the culprit $(s_i, s_{i+1}), \cdots, (s_{j-1}, s_j)$. Let $S_m \subseteq S$ be the set of all strings that belong to culprits. I.e. for each culprit $[i, j]$, $s_i, s_{i+1}, \cdots, s_j \in S_m$. Let C_m be the cycle cover over S_m defined by the culprits. I.e. each culprit $[i, j]$, defines the cycle $c_{ij} = s_i, s_{i+1}, \cdots, s_j$ and $w(c_{ij}) = d(s_i, s_{i+1}) + d(s_{i+1}, s_{i+2}) + \cdots + d(s_{j-1}, s_j) + d(s_j, s_i)$. It is straightforward to see that if we apply MGREEDY to the subgraph of the distance graph induced by S_m the algorithm will construct the cycle cover C_m. This implies that C_m is a minimum cycle cover in the distance graph defined by the set S_m. We denote by w_c the weight of the cycle cover C_m. Since $S_m \subseteq S$, and C_m is a minimum cycle cover over S_m, we have that $w_c \leq OPT(S_m) \leq OPT(S)$. We denote by o_c the sum of the overlaps of all culprits back edges. I.e. $o_c = \sum_{[i,j]=culprit} ov(s_j, s_i)$.

3.3 A bound of 3.5 on the approximation ratio

Blum *et al.* [18] showed that the length of the string t produced by GREEDY is

$$|t| \leq 2OPT(S) + o_c - w_c. \tag{3.3}$$

To get the 4-approximation they used Inequality (3.2) to bound o_c. Using Inequality (3.2), they showed that $o_c \leq OPT(S) + 2w_c$, which together with Inequality (3.3) implies

an upper bound of 4 on the approximation ratio. We get our result by proving that $o_c \leq OPT(S) + 1.5w_c$, which together with Inequality (3.3) implies a bound of 3.5 on the approximation ratio.

Let $c_{ir} = s_1, \cdots s_r$ be the cycle in C_m that corresponds to the culprit $[i, r]$. Let $a_{ir} = t_{c_{ir}}$ be the string guaranteed to exist for that cycle by Lemma 3.2.5. Let $A = \{a_{ir} \mid [i, r] \text{ is a culprit } \}$. The following is a key lemma in establishing our result.

Lemma 3.3.1 *For each culprit* $[i, r]$, $ov(s_r, s_i) \leq |a_{ir}| - w(c_{ir})$

Proof: Let $j \in [i, r]$ be such that by Lemma 3.2.5, a_{ir} contains $< s_{j+1}, \cdots, s_r, s_1, \cdots, s_j >$ as a suffix. Since the length of the overlap between s_r and s_i is the smallest among all the overlaps of strings in c_{ir}, we have that $ov(s_r, s_i) \leq ov(s_j, s_{j+1})$. String a_{ir} contains $< s_{j+1}, \cdots, s_j >$, so,

$$\begin{aligned} |a_{ir}| &\geq |< s_{j+1}, \cdots, s_j >| \\ &= |pref(s_{j+1}, s_{j+2}) \cdots pref(s_{j-1}, s_j) s_j| \\ &= |pref(s_{j+1}, s_{j+2}) \cdots pref(s_{j-1}, s_j) pref(s_j, s_{j+1})| + ov(s_j, s_{j+1}) \\ &\geq w(c_{ir}) + ov(s_r, s_i). \end{aligned}$$

■

Finally we establish the bound we claimed on o_c.

Lemma 3.3.2 *We have that* o_c *the total overlap of the bad back edges is at most* $OPT(S) + 1.5w_c$.

Proof: We denote by $|A|$ the sum of the lengths of all the strings in A, then $|A| = \sum_{[i,r]=culprit} |a_{ir}|$. We denote by $OV(A)$ the total overlap between adjacent strings in the shortest superstring of A. I.e. if $< b_1, \cdots, b_k >$ is the shortest superstring of A, then $OV(A) = \sum_{j=1}^{k-1} ov(b_j, b_{j+1})$. Let $c_i \in C_m$ be the cycle that corresponds to the string $b_i \in A$. Using Lemma 3.2.5,we get that

$$OV(A) = \sum_{j=1}^{k-1} ov(b_j, b_{j+1}) \leq \sum_{j=1}^{k-1} w(c_j) + \frac{1}{2} w(c_{j+1}) \leq 1.5w_c \tag{3.4}$$

Recall that by Equality (3.1), $|A| = OPT(A) + OV(A)$. By Lemma 3.2.6, $OPT(A) \leq OPT(S_m) + w_c \leq OPT(S) + w_c$. Putting it all together we get that,

$$|A| = OPT(A) + OV(A) \leq OPT(S) + w_c + 1.5w_c = OPT(S) + 2.5w_c.$$

By Lemma 3.3.1 for each culprit $[i, r]$, $ov(s_r, s_i) \leq |a_{ir}| - w(c_{ir})$. Summing over all culprits we get that $o_c \leq |A| - w_c$, and we get that $o_c \leq |A| - w_c \leq OPT(S) + 1.5w_c$. ■

3.4 Using the approximation algorithm for maximum ATSP to get the 2.5-approximation ratio for the shortest superstring

For completeness, we describe here the reduction of Breslauer *et al.* [20] from the maximum asymmetric TSP to the shortest superstring problem. The algorithm is as follows. We build the distance graph G_S. Recall that in G_S each vertex v_i represents a string s_i and the weight of each directed edge (v_i, v_j) is $d(s_i, s_j)$. We find the minimum cycle cover in $CYC(G_S)$ in G_S. Let $c_i = s_{i_1}, \cdots s_{i_r}$ be the cycle in $CYC(G_S)$. Recall that $w(c) = d(s_{i_1}, s_{i_2}) + d(s_{i_2}, s_{i_3}) + \cdots + d(s_{i_{r-1}}, s_{i_r}) + d(s_{i_r}, s_{i_1})$. Let t_c be the string guaranteed to exist for that cycle by Lemma 3.2.5. That is t_c is a superstring of the strings $s_{i_1}, \cdots s_{i_r}$. Let c' be a different cycle in the cycle cover, and let $t_{c'}$ be the string produced by Lemma 3.2.5 for this cycle cover. Then $ov(t_c, t_{c'}) \leq w(c') + \frac{1}{2}w(c)$. Let $c_1, \cdots, c_m$ be the cycles in the cycle cover, and let $A = \{t_{c_i} | i = 1 \cdots m\}$ be the set of their corresponding strings. Then a superstring for the strings in A is also a superstring for $s_1, \cdots, s_n$. So we now concentrate on finding a superstring for the strings in the set A

We denote by $OPT(A)$ the length of the shortest superstring of the strings in A. We also denote by $OV(A)$ the total overlap between adjacent strings in the shortest superstring of A, see Section 3.3. By equality (3.1), we have that $OPT(A) = \Sigma_{i=1}^{m}|t_{c_i}| - OV(A)$. Let G' be the overlap graph over the strings in A. Each vertex v_i, $i = 1 \cdots m$ in G' represents the string t_{c_i}, and $w(v_i, v_j) = ov(t_{c_i}, t_{c_j})$. Let $v_{i_1}, \cdots v_{i_m}$ be a maximal hamiltonian path in G', then it follows from equation 3.1 that $< t_{c_{i_1}}, \cdots, t_{c_{i_m}} >$ is the shortest superstring for A and that $\Sigma_{j=1}^{m-1} w(v_{i_j}, v_{i_{j+1}}) = OV(A)$

Recall that by Equality (3.1), we have that $\Sigma_{i=1}^{m}|t_{c_i}| = OPT(A) + OV(A)$. By Lemma 3.2.6, $OPT(A) \leq OPT(S) + w(CYC(G_S))$, where $w(CYC(G_S)) \leq OPT(S)$ is the weight of the minimum cycle cover in G_S. So we have that

$$\Sigma_{i=1}^{m}|t_{c_i}| = OPT(A) + OV(A) \leq OPT(S) + w(CYC(G_S)) + OV(A) \tag{3.5}$$

Let $< b_1, \cdots, b_m >$ be the shortest super string for A, (where each $b_i = t_{c_j}$ for some j). Since t_i and t_j for $i \neq j$ are inequivalent, see Lemma 3.2.2 and Lemma 3.2.5, then by Lemma 3.2.5, the total amount of overlap of the superstring is bounded by

$$OV(A) = \Sigma_{i=1}^{m-1} ov(b_i, b_{i+1}) \leq \frac{3}{2} w(CYC(G_S)) \tag{3.6}$$

We build the following complete graph G. The set of vertices of G is $v_1, \cdots v_m$ and we have an additional vertex u. Each vertex v_i represents the string $t_{c_i}, i = 1, \cdots, m$, and the weight of the directed edge (v_i, v_j) is $ov(t_{c_i}, t_{c_j})$. We also define $w((v_i, u)) = 0, i = 1 \cdots m$ and $w((u, v_i)) = 0, i = 1 \cdots m$.

Notice the weight of the maximum hamiltonian cycle in G is the weight of the maximal hamiltonian path in the overlap graph G', which is equal to $OV(A)$. We use the

approximation algorithm for maximum asymmetric TSP described in Chapter 2 to find an hamiltonian cycle in G. The algorithm finds an Hamiltonian cycle of weight at least $\frac{2}{3}OV(A)$. Let $v_{i_1} \cdots, v_{i_m}, u$ be the cycle found by our approximation algorithm. Then $\Sigma_{j=1}^{m-1} w(v_{i_j}, v_{i_{j+1}}) \geq \frac{2}{3}OV(A)$. The superstring we report is $t =< t_{c_{i_1}}, \cdots t_{c_{i_m}} >$. We have that

$$|t| = \Sigma_{j=1}^{m}|t_{c_j}| - \Sigma_{j=1}^{m-1} ov(t_{c_{i_j}}, t_{c_{i_{j+1}}}) = \Sigma_{j=1}^{m}|t_{c_j}| - \Sigma_{j=1}^{m-1} w(v_{i_j}, v_{i_{j+1}}) \leq \Sigma_{i=1}^{m}|t_{c_i}| - \frac{2}{3}OV(A)$$

Using inequality 3.5 we get that

$$|t| \leq OPT(S) + w(CYC(G_S)) + OV(A) - \frac{2}{3}OV(A) = OPT(S) + w(CYC(G_S)) + \frac{1}{3}OV(A)$$

Using inequality 3.6 we get that

$$|t| \leq OPT(S) + w(CYC(G_S)) + \frac{1}{3}(\frac{3}{2}w(CYC(G_S))) = OPT(S) + \frac{3}{2}w(CYC(G_S)) \leq 2.5OPT(S)$$

3.5 Concluding Remarks

There is still a big gap between the lower bound of 2 to our upper bound of 3.5 on the approximation ratio of the greedy algorithm for the shortest superstring problem. It would be interesting to close or a least narrow this gap.

It is also interesting to find other algorithms (not necessarily greedy), for the shortest superstring problem with better approximation ratio than the best known approximation ratio of 2.5.

Part II

The Greedy Algorithm for Edit Distance with Moves

Chapter 4

The Greedy Algorithm for Edit Distance with Moves

4.1 Introduction

The problem of finding the edit distance when move operations are allowed is defined as follows. Given two strings S and T, find the minimum number of insert character, delete character, and move substring operations required to transform the string S into the string T. Formally a move substring operation has three parameters p_1, ℓ, and p_2, which specify the position p_1 in which the substring that we move starts, the length ℓ of the substring which we move, and the position p_2 before which the substring should be reinserted. We assume that p_2 is either smaller than p_1 or larger than $p_1 + \ell$. If $S = a_1, \ldots, a_n$ is a string then a move operation with parameter p_1, ℓ, p_2, transform s to $s' = s_1, \cdots, s_{p_1-1}, s_{p_1+\ell}, \cdots s_{p_2-1}, s_{p_1}, \cdots, s_{p_1+\ell-1}, s_{p_2}, \cdots, s_n$. Here we assume that $p_2 > p_1 + \ell$.

This problem is known to be NP-Hard [79]. An approximation algorithm for the problem can be derived from the work of Cormode and Muthukrishnan [29]. This algorithm produces a sequence of operations (insert, delete, and move) of length at most $OPT_A \log n \log^* n$ that transform the string S to the string T, where OPT_A is the length of the minimum such sequence and $n = |S| + |T|$.

Shapira and Storer [79] showed that we can reduce the edit distance with moves problem into the following problem which we call the block permutation problem. Let S and T be two strings such that $|S| = |T|$, and T contains exactly the same character multiset as S (each character in S appears the same number of times in T and vice versa). We want to find a partition of S and T into disjoint blocks (substrings), such that each block that appears in S also appears in T (the same number of times), and vice versa. I.e. we can obtain T from S by permuting the blocks of S. Since S and T consist of exactly the same multiset of characters such a partition always exists since we can take each character

in S and in T to be a block. However it could be that there are other partitions with smaller number of blocks. The block permutation problem looks for such a partition into the smallest possible number of blocks.

Shapira and Storer [79] showed that given an instance of the edit distance with moves problem, we can create an instance of the block permutation problem such that if OPT_A is the value of the optimal solution to the edit distance with moves problem, and OPT is the optimal solution to the block permutation problem, then $OPT_A \leq cOPT$ for some constant c. Furthermore, they in fact show how to obtain from a partition into k blocks, an edit sequence with substring moves, insertions and deletions of characters of length $\Theta(k)$. Conversely, Shapira and Storer [79] also showed that given an instance S and T of the block permutation problem, we can solve it by finding an edit sequence with moves[1] for S and T, and then extracting a partition into blocks from it. If the length of the edit sequence is k then the number of blocks is at most $c'k$ for a small constant c'. In particular it follows that $OPT \leq cOPT_A$ and we can derive an approximation algorithm for the block permutation problem from an approximation algorithm for the edit distance with moves problem suffering only a constant increase in the approximation guarantee. Thus, up to a constant factor, the edit distance with moves problem and the block permutation problem are equivalent with respect to the best approximation factor one can achieve for them.

Shapira and Storer [79] also gave a simple GREEDY algorithm to the block permutation problem. They claimed that GREEDY is a $\log n$-approximation to the problem (where n is the length of S). However Chrobak *et al.* [23] proved a lower bound of $\Omega(n^{0.43})$ on the approximation ratio of GREEDY, thereby showing that the claim of Shapira and Storer regarding the performance of GREEDY is false. The example of Chrobak *et al.* that realizes the lower bound uses an alphabet of size $\Omega(n^{0.43})$. Chrobak *et al.* also proved an upper bound of $O(n^{0.69})$ on the performance guarantee of GREEDY.

In this thesis we give a tighter lower bound on performance of GREEDY, that uses a smaller alphabet size. We show an infinite family of strings for which GREEDY finds a block partition of size $\Omega(n^{0.46}OPT)$. The alphabet size of the strings in our family is $O(\log n)$.

More distantly related work: Chrobak *et al.* [23] also studied restricted versions of the problem. The $k-$MCSP problem is to find minimum partition into blocks where each letter occurs at most k times in the input strings. They showed that the approximation ratio of GREEDY for $2-$MCSP is 3. They also proved a lower bound of $\Omega(\log n)$ on the approximation ratio of GREEDY for $4-$MCSP. Goldstein *et al.* [40] showed that $2-$MCSP is APX-hard, and gave approximation algorithms for $2-$MCSP and for $3-$MCSP.

The edit distance problem without moves is one of the fundamental string matching problems. A classical dynamic programming algorithm solves it in $O(mn)$ time where $m = |S|$ and $n = |T|$. See e.g. the book by Gusfield [41]. Several variations of the

[1]Since S and T contain the same multiset of characters then it is easy to see that we can convert any edit sequence to an edit sequence with move operations only without increasing its length.

edit distance with moves problem have been studies. Tichy [83] studied the problem of finding a minimal cover of a string T by substrings of a string S. In his version of the problem you are allowed to use parts of S multiple times in order to cover T. Tichy described a polynomial time algorithm to find a minimal cover. Lopresti and Tomkins [69] studied variations of the block permutation problem. In their problem the input includes in addition to the strings S and T a distance function to measure similarity between blocks. We look at a partition of S into blocks and a partition of T into blocks, and a matching between the blocks, such that the sum of the distances between matched blocks is minimized. Lopresti and Tomkins also consider variations where the blocks need not be disjoint and where the blocks need not cover either S or T or both. They prove that some of these versions are NP-hard, and describe polynomial time algorithms for others. Muthukrishnan and Sahinalp [74] considered a more general version of the edit distance with moves problem where we also allow reversing blocks, and linear transformations on blocks. They describe an approximation algorithm for this problem with approximation guarantee of $O(\log n(\log^* n)^2)$.

4.2 Roadmap

The structure of this chapter is as follows. In Section 4.3 we review the definition of the GREEDY algorithm. In Section 4.4 we describe an infinite family of sequences for which $OPT = 2$ and GREEDY produces a partition into about $n^{0.46}$ blocks. This example uses alphabet of size $O(\log n)$.

4.3 The GREEDY algorithm to the block partition problem

We denote by $LCS(S,T)$ the longest common substring of S and T, and by Σ the alphabet. GREEDY is defined as follows.

while($|LCS(S,T)| > 1$) {

- Let $P = LCS(S,T)$
- Let x be a new character ($x \notin \Sigma$)
- replace the same number of occurrences of P in S and in T by x.
- $\Sigma = \Sigma \bigcup \{x\}$

}

Remarks:

1. We can find $LCS(S,T)$ using a generalized suffix tree for S and for T, See Section 1.2.3. (We need to build this suffix tree in each iteration of the loop).

2. Each new symbol introduced by GREEDY cannot be included in any LCS since this would contradict the fact that the symbol replaced an LCS. Thus each new symbol corresponds to a block in the final solution. An original symbol is a block by itself if it remains in the final string.

3. If there are several substrings that are $LCS(S,T)$, GREEDY may choose any one of them and we will use this in the example which we describe. (The question of whether one can construct an example with no ties is also open.)

4. We did not insist that GREEDY replaces the maximum possible number of occurrences of P in S and in T. Our example, however, works (with minor changes) for both variants of GREEDY: the one described above and an alternative one where we insist on replacing the maximum possible number of occurrences of P in S and in T.

4.4 Bad Example for GREEDY

We mark the k^{th}-character in a string S by $s[k]$ for $k = 0, \cdots, |S| - 1$. For every $i \geq 0$ we define strings S_i and T_i on which we run GREEDY. Let Σ_i be the set of characters that appear in the string S_i. To define S_i and T_i we also define strings A_i, B_i and C_i.

For $i = 0$ we define

$$A_0 = a, \;\; B_0 = b, C_0 = c \;\; S_0 = A_0B_0 = ab, \;\; T_0 = B_0A_0 = ba \; .$$

In general we define A_i, B_i and C_i recursively based on A_{i-1} and B_{i-1} and once we have defined A_i and B_i (we'll do so below) we set $S_i = A_iB_i$ and $T_i = B_iA_i$. So clearly the value of OPT for S_i and T_i is 2. For $i \geq 1$, let d_i and $e_i \neq d_i$ be two new characters that don't appear in the strings S_{i-1}, T_{i-1}, $(d_i, e_i \notin \Sigma_{i-1})$. For $i \geq 1$, we define A_i, B_i and C_i as follows.

$$A_i = B_{i-1}A_{i-1}C_{i-1}A_{i-1}B_{i-1} \; ,$$

$$B_i = A_{i-1}C_{i-1}d_iC_{i-1}A_{i-1} \; ,$$

and

$$C_i = A_{i-1}C_{i-1}e_iC_{i-1}A_{i-1}.$$

For example $A_1 = bacab$ and $B_1 = acdca$, $C_1 = aceca$. Recall that $S_i = A_iB_i$ and $T_i = B_iA_i$, so we have $S_1 = bacabacdca$ and $T_1 = acdcabacab$. Notice that with this definition $|\Sigma_i| = |\Sigma_{i-1}| + 2$, and we get that $|\Sigma_i| = O(\log |S_i|)$.

Since $|C_i| = |B_i|$, it is straightforward to verify that the size of the strings satisfies the recurrence $|S_i| = |T_i| = 5|B_{i-1}| + 4|A_{i-1}| + 1$, $|A_i| = 2|A_{i-1}| + 3|B_{i-1}|$ and $|B_i| = 2|A_{i-1}| + 2|B_{i-1}| + 1$ for $i \geq 1$. Also notice that for $i \geq 0$, $|B_i| \leq |A_i|, |B_i| \leq \frac{1}{2}|S_i|$.

We now apply the GREEDY algorithm to

$$S_i = A_iB_i = B_{i-1}A_{i-1}C_{i-1}A_{i-1}B_{i-1}A_{i-1}C_{i-1}d_iC_{i-1}A_{i-1} \; ,$$

and

$$T_i = B_iA_i = A_{i-1}C_{i-1}d_iC_{i-1}A_{i-1}B_{i-1}A_{i-1}C_{i-1}A_{i-1}B_{i-1} \ .$$

It is easy to verify that the longest common substring of S_i, T_i ($LCS(S_i, T_i)$), is of length $|A_i|$: Let z be a common substring of S_i and T_i of length at least $|A_i|$. Clearly z must completely contain either the first C_{i-1} block in S_i or the second C_{i-1} block in S_i. Assume first that z contains the first C_{i-1} block of S_i. In order to be of length at least $|A_i|$, z must also span the following A_{i-1} block of S_i. Therefore the corresponding occurrence of z in T_i must span at least one of the two occurrences of $C_{i-1}A_{i-1}$ in T_i. If it spans the first occurrence of $C_{i-1}A_{i-1}$ in T_i then z must start at C_{i-1} and cannot go beyond the last C_{i-1} block of T_i. On the other hand if z spans the second occurrence of $C_{i-1}A_{i-1}$ in T_i then it must terminate before the second A_{i-1} block of S_i. In both cases we obtain that $|z| \leq |A_i|$. Assume now that z contains the second C_{i-1} block in S_i. Then to be of length at least $|A_i|$, z must contain also the preceding A_{i-1} block of S_i. So the occurrence of z in T_i must span one of the occurrences of $A_{i-1}C_{i-1}$ in T_i. If z spans the first occurrence of $A_{i-1}C_{i-1}$ in T_i then it cannot span any character of the first $|A_i|$ characters of S_i, this in contradiction to the assumption that $|z| \geq |A_i|$. If z contain the second occurrence of $A_{i-1}C_{i-1}$ in T_i then it cannot span any character beyond the first and the second occurrence of C_{i-1} in S_i. So it also follows that $|z| \leq |A_i|$.

It is easy to see that S_i and T_i have two common substrings of length $|A_i|$. These are A_i and the substring $X_i = C_{i-1}A_{i-1}B_{i-1}A_{i-1}C_{i-1}$. Assume that GREEDY chooses the string X_i, in this case we are left with the following strings.

$$S'_i = B_{i-1}A_{i-1}xd_iC_{i-1}A_{i-1}$$

and

$$T'_i = A_{i-1}C_{i-1}d_ixA_{i-1}B_{i-1} \ ,$$

where x is the new character GREEDY uses to replace the occurrence of X_i in both strings. Now, it is easy to verify that $|LCS(S'_i, T'_i)| = |A_{i-1}|$. In fact, the set of the longest common substrings consists of the first and the second occurrence of A_{i-1}, and the two occurrences of the string X_{i-1}. [2]

We will now show that GREEDY may repeat these choices of longest common substrings recursively and alternately, on $B_{i-1}A_{i-1} \in S_i$, $A_{i-1}B_{i-1} \in T_i$, and also on $C_{i-1}A_{i-1} \in S_i$ and $A_{i-1}C_{i-1} \in T_i$. I.e. GREEDY will choose next the string $X_{i-1} = C_{i-2}A_{i-2}B_{i-2}A_{i-2}C_{i-2}$ as $LCS(B_{i-1}A_{i-1}, A_{i-1}B_{i-1})$, and in the following step GREEDY will again choose the second occurrence of X_{i-1}, as the corresponding LCS of $C_{i-1}A_{i-1}, A_{i-1}C_{i-1}$ and so on.

Let

$$U = B_{i-1}A_{i-1}[j..k] = A_{i-1}B_{i-1}[\ell..\ell + k - j]$$

[2]Notice that X_{i-1} is also the longest common substring of $C_{i-1}A_{i-1}$ and $A_{i-1}C_{i-1}$, of $B_{i-1}A_{i-1}$ and $A_{i-1}C_{i-1}$ and of $C_{i-1}A_{i-1}$ and $A_{i-1}B_{i-1}$.

be a common substring of $B_{i-1}A_{i-1}$, and $A_{i-1}B_{i-1}$. We say that a common substring Q of $C_{i-1}A_{i-1}$, and $A_{i-1}C_{i-1}$ *corresponds to* U, if

$$Q = C_{i-1}A_{i-1}[j..k] = A_{i-1}C_{i-1}[\ell..\ell + k - j] \ .$$

We claim that GREEDY may choose recursively and alternately corresponding common substrings of $B_{i-1}A_{i-1}$, and $A_{i-1}B_{i-1}$ and of $C_{i-1}A_{i-1}$, and $A_{i-1}C_{i-1}$. Starting with S'_i, and T'_i, GREEDY iterates as follows. In iteration $p = 2k + 1$, $k \geq 0$, GREEDY chooses recursively a common substring X of $B_{i-1}A_{i-1}$, and $A_{i-1}B_{i-1}$ and in iteration $p + 1$ GREEDY chooses the corresponding common substring W of $C_{i-1}A_{i-1}$, and $A_{i-1}C_{i-1}$.

We now show by induction that in each of the following iterations GREEDY may indeed choose a longest common substring as described above. For a basis it is clear that in the first iteration GREEDY may choose the substring $X_{i-1} = LCS(B_{i-1}A_{i-1}, A_{i-1}B_{i-1})$ (as defined above). Then in the second iteration GREEDY may choose the second occurrence of the string $X_{i-1} = LCS(C_{i-1}A_{i-1}, A_{i-1}C_{i-1})$.

Suppose that until iteration $2k$ GREEDY indeed chose pairs of corresponding common substrings of $B_{i-1}A_{i-1}$ and $A_{i-1}B_{i-1}$ and of $C_{i-1}A_{i-1}$ and $A_{i-1}C_{i-1}$. Suppose that at iteration $2k+1$ we have that $B_{i-1}A_{i-1}[j..k] = A_{i-1}C_{i-1}[\ell..\ell+k-j]$. Then by the definition of C_{i-1} , $B_{i-1}A_{i-1}[j..k]$ doesn't contain the character d_{i-1}, and $A_{i-1}C_{i-1}[\ell..\ell + k - j]$ does not contain the character e_{i-1}. Since 1) $A_{i-1}C_{i-1}[q] = A_{i-1}B_{i-1}[q]$, if $A_{i-1}C_{i-1}[q] \neq e_{i-1}$ and 2) by the induction hypothesis the substring $A_{i-1}B_{i-1}[\ell..\ell+k-j]$ has not been matched and replaced if $A_{i-1}C_{i-1}[\ell..\ell + k - j]$ still have not been replaced, we get that we also have $B_{i-1}A_{i-1}[j..k] = A_{i-1}B_{i-1}[\ell..\ell + k - j]$. Similarly, if $C_{i-1}A_{i-1}[j..k] = A_{i-1}B_{i-1}[\ell..\ell + k - j]$, then we also have $C_{i-1}A_{i-1}[j..k] = A_{i-1}C_{i-1}[\ell..\ell + k - j]$. It follows that GREEDY may indeed continue and choose a longest common substring of $B_{i-1}A_{i-1}$, and $A_{i-1}B_{i-1}$ in iteration $2k + 1$ and a corresponding longest common substring of $C_{i-1}A_{i-1}$, and $A_{i-1}C_{i-1}$ in iteration $2k + 2$.

Let $R(i)$ be the number of blocks that GREEDY finds when it works on the strings S_i, T_i. Then $R(i) = 2 + 2R(i - 1), i > 0$. This follows from the fact that the number of blocks greedy finds is 2 (X_i and d_i)+ the number of blocks GREEDY finds when applying the recursion to $B_{i-1}A_{i-1}, A_{i-1}B_{i-1}$ and to $C_{i-1}A_{i-1}, A_{i-1}C_{i-1}$. Thus we get that

$$R(i) = 2 + 2R(i - 1) \cong 2^i$$

We now show that $|S_i| = O((4.5)^i)$ for $i \geq 1$.

$$|S_i| = 4|A_{i-1}| + 5|B_{i-1}| + 1 = 4(|A_{i-1}| + |B_{i-1}|) + |B_{i-1}| + 1 =$$

$$4|S_{i-1}| + |B_{i-1}| + 1 \leq 4.5|S_{i-1}| + 1 \cong 2(4.5)^i.$$

Putting it all together we get that,

$$R(i) \cong 2^i = 4.5^{(\log_{4.5} 2)i} \cong |S_i|^{(\log_{4.5} 2)} \cong |S_i|^{0.46} \ .$$

We recall that $OPT = 2$ so this show that the approximation guarantee of GREEDY is $\Omega(n^{0.46})$.

4.5 Concluding remarks

Our example and the work of Chrobak *et al.* [23] emphasize many intriguing questions:

1. Find a correct $O(\log n)$ approximation algorithm to the block partition problem and to the edit distance with moves problem.
2. Is there a constant approximation algorithm for these problems ?
3. Can one improve the upper bound $O(n^{0.69})$ on the performance of GREEDY ?
4. Can one improve the lower bound of $\Omega(n^{0.46})$ on the approximation guarantee of GREEDY ?
5. Can one construct an example realizing the lower bound that uses an alphabet of constant size ?

Part III

Finding the position of the k-mismatch and approximate repetitions

Chapter 5

Introduction

5.1 Introduction

Let P be a pattern of length m and let T be a text of length n. Let $T(i, \ell)$ denote the substring of T of length ℓ starting at position i.[1] In the *k-mismatch problem* we determine for every $1 \leq j \leq n - m + 1$, if $T(j, m)$ matches P with less than k mismatches. In case $T(j, m)$ does not match P with less than k mismatches we compute the position $s(j)$ in P of the k-mismatch. If P matches T starting at position j with at least one but less than k mismatches, we compute the position in P of the last mismatch. Otherwise, we report that there is an exact match at position j. See Figure 5.1.

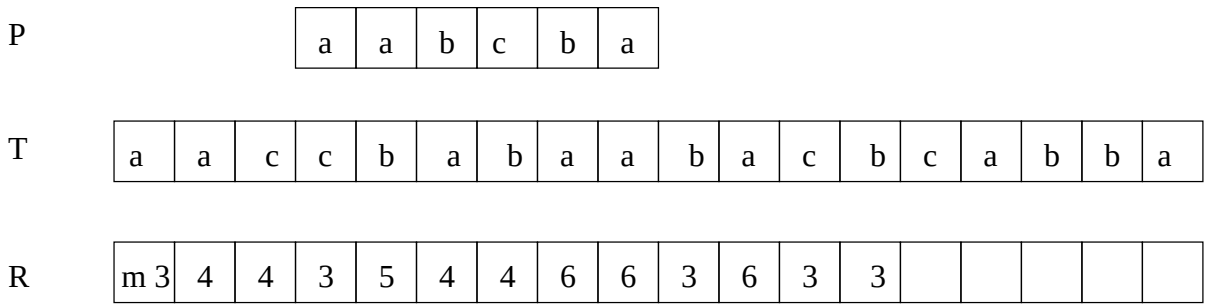

Figure 5.1: An example of the result vector (R) returned by the exact algorithm for finding the position of the k-mismatch. Here $k = 3$. Notice that $T(1, 6)$ matches P with one mismatch. Therefore in $R[1]$ we mark by "m" that there is a match with less than 3 mismatches of P with $T(1, 6)$, and we also write the position of the last mismatch which is 3 in this case. In all other positions of T there are at least 3 mismatches.

Several classical results are related to the k-mismatch problem. Abrahamson [1], gave

[1] We always assume that $i \leq n - m + 1$ when we use this notation.

an algorithm that finds for each $1 \leq j \leq n - m + 1$, the number of mismatches between $T(j, m)$ and P. The running time of Abrahamson's algorithm is $O(n\sqrt{m \log m})$. Amir et al. [6], gave an algorithm that for each $1 \leq j \leq n - m + 1$, determines if the number of mismatches between $T(j, m)$ and P is at most k in $O(n\sqrt{k \log k})$ time. Both of these algorithms do not give any information regarding the position of the last mismatch or the position of the k-mismatch. This information is useful for applications that want to know not only if the pattern matches with at most k-mismatches, but also how long is the prefix of the pattern that matches with at most k-mismatches.

The major technique used by the algorithms of Abrahamson and of Amir et al. is convolution. Let us fix a particular character $x \in \Sigma$. Suppose we want to compute for every $1 \leq j \leq n-m+1$, the number of places in which an x in P does not coincide with an x in T when we align P with $T(j, m)$. We can perform this task by computing a convolution of a binary vector $P(x)$ of length m, and a binary vector $T(x)$ of length n as follows. The vector $P(x)$ contains 1 in every position where P contains the character x and 0 in all other positions. The vector $T(x)$ contains 1 in every position where T does not contains x and 0 in every position where T contains x. We can perform the convolution between $P(x)$ and $T(x)$ in $O(n \log m)$ time using the Fast Fourier Transform. So if P contains only $|\Sigma|$ different characters we can count for each $1 \leq j \leq n - m + 1$, the number of mismatches between $T(j, m)$ and P in $O(|\Sigma| n \log m)$. We do that by performing $|\Sigma|$ convolutions as described above, one for each character in P, and add up the mismatch counts.

There is a simple deterministic algorithm for the k-mismatch problem that runs in $O(nk)$ time and $O(n)$ space. This algorithm is often called the *kangaroo method* and was discovered by Landau and Vishkin [65], and Galil and Giancarlo [39]. To apply this method we construct a generalized suffix tree for the text and the pattern, with a data structure for lowest common ancestor (LCA) queries, to allow to skip equal substrings in the text and pattern in constant time. For each position j we find the first mismatch between $T(j, m)$ and P by an appropriate LCA query. We skip this mismatch and find the next one by another LCA query and so on. This way we find the k-mismatch in $O(k)$ time. We repeat this for every j so that total running time is $O(nk)$. We give an alternative algorithm that runs in $O(nk^{\frac{2}{3}} \log^{\frac{1}{3}} m \log k)$ time and linear space.

To see why the bound of $O(nk^{\frac{2}{3}} \log^{\frac{1}{3}} m)$ may be natural, consider a pattern of length $m = O(k)$. In this case, we can solve the problem using the method of Abrahamson [1]. We divide the pattern into $k^{\frac{1}{3}} / \log^{\frac{1}{3}} k$ blocks, each block of size $z = O(k^{\frac{2}{3}} \log^{\frac{1}{3}} k)$. By applying the algorithm of Abrahamson with the first block as the pattern, we determine in $O(n\sqrt{z \log z}) = O(nk^{\frac{1}{3}} \log^{\frac{2}{3}} k)$ time, the number of mismatches of each text position with the first block. Similarly, by applying the method of Abrahamson to each of the subsequent $k^{\frac{1}{3}} / \log^{\frac{1}{3}} k$ blocks of the pattern, and accumulating the number of mismatches for each text position, we know in $O(nk^{\frac{2}{3}} \log^{\frac{1}{3}} k)$ time for each text position, which block contains the k-mismatch. Moreover we also know for each text position the number of mismatches in the blocks preceding the one that contains the k-mismatch. With this information, we can

find for each text position the k-mismatch in the relevant block in $O(k^{\frac{2}{3}} \log^{\frac{1}{3}} k)$ time by scanning the block character by character looking for the appropriate mismatch. We do not know how to get a better bound even for this simple example.

If the alphabet size is small we can get a better running time. Consider a pattern of length $m = O(k)$, over an alphabet Σ of constant size. In this case, we can find the k-mismatch in $O(n\sqrt{k \log k})$ time by performing convolutions. We divide the pattern into $\sqrt{k/\log k}$ blocks, each block of size $O(\sqrt{k \log k})$. We perform $|\Sigma|$ convolutions to find the number of mismatches in the first block in $O(n|\Sigma| \log k)$ time. After repeating it sequentially for all $\sqrt{k/\log k}$ blocks, we know in $O(n|\Sigma|\sqrt{k \log k}) = O(n\sqrt{k \log k})$ time for each text position, which block contains the k-mismatch. We also know the number of mismatches in the blocks preceding the one that contains the k-mismatch. With this information, we can find for each text position the k-mismatch in the relevant block in $O(\sqrt{k \log k})$ time by scanning the block character by character looking for the appropriate mismatch. The total running time is $O(n\sqrt{k \log k})$.

We also define the *approximate k-mismatch problem*. This problem has an additional accuracy parameter ϵ. The task is to determine for every $1 \leq j \leq n - m + 1$ a position $s(j)$ in P such that the number of mismatches between $T(j, s(j))$ and $P(1, s(j))$ is at least $(1 - \epsilon)k$ and at most $(1 + \epsilon)k$, or report that there is no such position.

We give a deterministic and randomized algorithms for the *approximate k-mismatch problem*. We describe the deterministic algorithm in Section 6.4. The running time of this algorithm is $O(\frac{1}{\epsilon^2} n\sqrt{k} \log^3 m)$. In Section 6.5, we give a randomized algorithm with running time of $O(\frac{1}{\epsilon^2} n \log n \log^2 m \log k)$. The randomized algorithm guarantees that for each j the number of mismatches between $T(j, s(j))$ and $P(1, s(j))$ is at least $(1 - \epsilon)k$ and at most $(1 + \epsilon)k$ with high probability.[2]

A position $s(j)$ computed by our algorithms for the *approximate k-mismatch problem* may not contain an actual mismatch. That is, the character $s(j)$ of P may in fact be the same as character $j + s(j) - 1$ of T. We can change both algorithms such that $s(j)$ would always be a position of a mismatch in $O(n)$ time as follows. For a string S we denote by $\overleftarrow{S}$ the string obtained by reversing S. We build a suffix tree for $\overleftarrow{T}$ and $\overleftarrow{P}$, with a data structure for LCA queries in constant time. For each position j in T we perform an LCA query for the suffixes $\overleftarrow{P(1, s(j))}$ of $\overleftarrow{P}$ and $\overleftarrow{T(1, j + s(j) - 1)}$ of $\overleftarrow{T}$. Let h be the string depth of the resulting node. Clearly h is the length of the longest common prefix of $\overleftarrow{P(1, s(j))}$ and $\overleftarrow{T(1, j + s(j) - 1)}$, and $s(j) - h$ is the position of the last mismatch between P and $T(j, m)$ preceding position $s(j)$. We change $s(j)$ to $s(j) - h$.

In Section 7.1, we use our algorithms for the k-mismatch problem to solve an approximate version of the k-mismatch tandem repeats problem. The *exact tandem repeats problem* is defined as follows. Given a string S of length n, find all substrings of S of the form uu. Main and Lorentz [71] gave an algorithm that solves this problem in $O(n \log n + z)$ time,

[2]By 'high probability', we mean probability that is polynomially small in n.

where z is the number of tandem repeats in S. Kolpakov and Kucherov [58] gave a linear time algorithm for finding all maximal consecutive runs of tandem repeats.

Repeats occur frequently in biological sequences, but they are usually not exact. Therefore algorithms for finding approximate tandem repeats were developed. The *k-mismatch tandem repeats problem* is defined as follows. Given a string S and a parameter k find all substrings uv of S such that $|u| = |v| > k$ and the number of mismatches between u and v is at most k. The best known algorithms for this problem is the algorithm of Landau, Schmidt and Sokol [64] that runs in $O(nk \log(n/k)+z)$ time, and the algorithm of Kolpakov and Kucherov [57] that runs in $O(nk \log k + z)$ time, where z is the number of k-mismatch tandem repeats.

We define the *approximate k-mismatch tandem repeats problem* which is a relaxation of the k-mismatch tandem repeats problem. In this relaxation we require that the algorithm will find all substrings uv of S such that $|u| = |v| > k$ and the number of mismatches between u and v is at most k, but we also allow the algorithm to report substrings uv such that the number of mismatches between u and v is at most $(1 + \epsilon)k$.

By combining the algorithm of [64] with our exact algorithm for the k-mismatch problem we get an algorithm for approximate k-mismatch tandem repeats that runs in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z)$ time where z is the number of approximate k-mismatch tandem repeats that we report. Similarly, using our deterministic algorithm for the approximate k-mismatch problem we get an algorithm for approximate k-mismatch tandem repeats that runs in $O(\frac{1}{\epsilon^3}n\sqrt{k} \log^3 n \log(n/k) + z)$ time. We can also use the randomized algorithm of Section 6.5 and get an algorithm that reports all k-mismatch tandem repeats with high probability, and possibly tandem repeats with up to $(1 + \epsilon)k$ mismatches in $O(\frac{1}{\epsilon^3}n \log^3 n \log k \log(n/k) + z)$ time.

In Chapter 7 we use the exact algorithm for the k-mismatch problem to approximate general repetitions. Kolpakov and Kucherov [57] defined two kinds of repeats with at most k mismatches that, in a sense, generalize k-mismatch tandem repeats.

1. A string R of length n is called a *k-mismatch globally-defined repeat (gd-repeat) of period length p*, if $p \leq n/2$ and the number of times that $R[i] \neq R[i + p]$ is at most k.

2. A string R of length n is a *run of k-mismatch tandem repeats with period length p* if $p \leq n/2$ and all substrings of R of length $2p$ are k-mismatch tandem repeats.

Kolpakov and Kucherov [57] describe an algorithm for finding all maximal k-mismatch gd-repeats and an algorithm for finding all maximal runs of k-mismatch tandem repeats. (A repeat is maximal if it cannot be extended neither to the right nor to the left and remain a repeat of the same kind and the same period.) The running time of both algorithms is $O(nk \log k + z)$ where z is the number of repeats of the appropriate kind. (Note that if the same string is a repeat for more than one period length then we count it with multiplicity equal to the number of lengths of which it is a period.) In particular, their algorithm for

finding all runs of k-mismatch tandem repeats, implies an algorithm for the k-mismatch tandem repeats problem with running time of $O(nk \log k + z)$.

We define the *approximate k-mismatch gd-repeat problem* as follows. Given a string S and $\epsilon > 0$, we want to find a set of substrings of S which are maximal $(1+\epsilon)k$-mismatch gd-repeats, and that each maximal k-mismatch gd-repeat is a substring of a string from the set.

Using our exact algorithm for the k-mismatch problem we construct two algorithms for the approximate k-mismatch gd-repeat problem that run in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log^2 k + z)$ and in $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z\right)$ time respectively, where z is the number of maximal $(1+\epsilon)k$-mismatch gd-repeats that we report. We also use our first algorithm for the approximate k-mismatch gd-repeat problem to get another algorithm for approximate tandem repeats that runs in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log^2 k + z)$ time, where z is the number of approximate k-mismatch tandem repeats that we report.

Similarly, we define the *approximate run of k-mismatch tandem repeats problem* as follows. Given a string S and $\epsilon > 0$, we want to find a set of substrings which are runs of $(1+2\epsilon)k$-mismatch tandem repeats, (not necessarily maximal), such that each maximal run of k-mismatch tandem repeats is a substring of a string that belongs to this set. We also require that the number of strings that we report is at most the number of maximal runs of $(1+\epsilon)k$-mismatch tandem repeats in S.

Using our exact algorithm for the k-mismatch problem, we get an algorithm for the approximate runs of k-mismatch tandem repeats problem that runs in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z)$ time, where z is the number of maximal runs of $(1+\epsilon)k$-mismatch tandem repeats in S.

Our algorithms for approximate tandem repeats and for approximate general repetitions, use the following generalization of the algorithm for the k-mismatch problem that works also for text positions $j > n - m + 1$. Let $j > n - m + 1$, if $T(j, n-j+1)$ matches $P(1, n-j+1)$ with at least k-mismatches then our algorithms will find the exact (or approximate) position of the k-mismatch between $T(j, n-j+1)$ and $P(1, n-j+1)$. If $T(j, n-j+1)$ matches $P(1, n-j+1)$ with less than k mismatches, then our algorithms will find the exact (or approximate) position of the last mismatch between $T(j, n-j+1)$ and $P(1, n-j+1)$, or report that $T(j, n-j+1)$ matches $P(1, n-j+1)$ with no mismatches. We can do that by changing the text to be the string $T' = T\$^m$ which is the concatenation of T and the string $\$^m$, where \$ is a new character that doesn't appear in T or in P. Then we run our algorithm on P and T'. For each text position $j > n-m+1$ for which $j + s(j) > n$ we decrease $s(j)$ to be the last mismatch of $P(1, n-j+1)$ with $T(j, n-j+1)$ if there is such a mismatch. We can find this mismatch using the suffix tree for $\overleftarrow{S}$ and $\overleftarrow{T}$.

Chapter 6

Finding the position of the k-mismatch

6.1 Preliminaries

A string s is *periodic* with period u, if $s = u^j w$, where $j \geq 2$ and w is a prefix of u. The *period* of s is the shortest substring u such that $s = u^j w$ and w is a prefix of u.

A *break* of s is an aperiodic substring of s. An *ℓ-break* is a break of length ℓ. We use a parameter $\ell < k$ that will be fixed later. We use the method of [27] to find a partition of the pattern into ℓ-breaks separated by substrings which are shorter than ℓ, or periodic substrings with period of length $\leq \ell/2$. We call the substrings that separate the breaks *periodic stretches* (although the small ones are not really periodic).

For completeness we describe the algorithm of [27] that finds such a partition. We use the following properties of periodic strings, (see e.g. [33]). If a string S is periodic with period u, and the string Sa, where a is a single character, is not periodic with a period u, then any suffix of Sa of length $\geq 2|u|$ is aperiodic. We prove this property in Lemma 6.1.1. It is also known that we can find the period of a string in time linear in the length of the string.

Lemma 6.1.1 *If a string S is periodic with period u, and the string Sa, where a is a single character, is not periodic with a period u, then any suffix of Sa of length $\geq 2|u|$ is aperiodic.*

Proof: First notice that since Sa, is not periodic with a period u, this implies that $S[|S| + 1 - |u|] \neq a$.

Let A be a suffix of Sa of length greater or equal of $2|u|$. Then, the claim above implies that A is not periodic with period length $|u|$.

Assume that A is periodic with period v such that $|v| > |u|$, and let A' be the suffix of A of length $2|v|$. Then, since S is periodic with period u, we have that $A'[|v|] = A'[|v| - |u|]$,

and since A' is periodic with period v we get that $A'[2|v|] = A'[2|v| - |u|]$. But this contradicts the fact that $A'[2|v|] = a$ and $A'[2|v| - |u|] = S[|S| + 1 - |u|] \neq a$.

Assume that A is periodic with period v such that $|v| < |u|$, and let A' be the suffix of A of length $2|u|$. then, we must have $A'[|u|] = A'[|u| - |v|] = x \neq a$ and $A'[2|u|] = A'[2|u| - |v|] = a$. But this implies that $A'[|u| - |v|] \neq A'[2|u| - |v|]$, which contradicts the fact that S is periodic with period u.

■

The algorithm for finding the partition is defined as follows. Let W be the prefix of length ℓ of P. If W is aperiodic then W is the first break of P and we continue to find a partition of the suffix of P that starts at position $\ell + 1$. If W is periodic with period u of length $\leq \ell/2$, then let a be the character at position $|W| + 1$ of P. If a equals to the character at position $|W| + 1 - |u|$ then Wa is periodic and we continue to check the character at position $|W| + 2$. If the character at position $|W| + 1 - |u|$ is not equal to a, then by the observation above the suffix of Wa of length ℓ is aperiodic, and we add it to the list of breaks. We continue to find a partition of the suffix of the pattern following the break that we found. Clearly the running time of the algorithm is $O(m)$.

In Sections 6.2 and 6.3, we show how to solve the k-mismatch problem when the pattern P contains at most $2k$ ℓ-breaks in the time and space bounds mentioned above. In case the pattern P contains more than $2k$ ℓ-breaks, we reduce it to the case where P contains $2k$ ℓ-breaks as follows.

Assume P contains more than $2k$ ℓ-breaks and let P' be the prefix of P with exactly $2k$ ℓ-breaks. We run our algorithm using P' rather than P. Our algorithm also finds all positions in T that match P' with at most k mismatches. Amir et al. [6] proved that at most n/ℓ positions of the text T match P' with at most k mismatches. After running our algorithm and finding these positions we use the kangaroo method [65, 39] to check whether each of these positions matches the original pattern P with at most k mismatches, and to find the location of the k-mismatch in case it does not. The total time it takes to check all of these positions is $O(nk/\ell)$. Therefore we assume from now on that the pattern P contains at most $2k$ ℓ-breaks, and add $O(nk/\ell)$ to the running time of the algorithm.

6.2 Finding the position of the k-mismatch

We describe an algorithm that solves the problem in $O(nk^{\frac{3}{4}} \log^{\frac{1}{4}} m)$ time and $O(n)$ space. In Section 6.3, we show that by adding another level of recursion we can reduce the running time to $O(nk^{\frac{2}{3}} \log^{\frac{1}{3}} m \log k)$ and still use $O(n)$ space.

Recall that we assume that the pattern contains at most $2k$ breaks, which are substrings of length ℓ, and at most $2k$ periodic stretches. Let A be a periodic stretch of length greater or equal to ℓ, and let x be the period of A.[1] Recall that $|x| \leq \ell/2$. Let x' be the

[1] A is periodic since $|A| \geq \ell$

lexicographically first cyclic rotation of x. We call x' the *canonical period* of A. We can write $A = yx'^i z, i \geq 1$, where y is a prefix of x, which may be empty, and z is a prefix of x' which may also be empty. Let $A' = x'^i$. We add y and z to the set of breaks, and if $i = 1$ we also add A' to the set of breaks. In addition we add to the set of breaks all the periodic stretches of length smaller than ℓ. We added to the set of breaks $O(k)$ substrings each of length at most ℓ, and we redefine the term *break* to include also these substrings. The string A' remains a periodic stretch if its length is at least ℓ. After this preprocessing, the period of every periodic stretch equals to its canonical period. More specifically, all periodic stretches with period u are of the form u^i, for some $i \geq 2$, where u is the canonical period of the string uu.

Choosing a prefix of the pattern. We choose a prefix S of the pattern such that for every j we can either find the position of the k-mismatch of S with $T(j, |S|)$, or determine that S matches $T(j, |S|)$ with less than k-mismatches. We also prove that S matches $T(j, |S|)$ with at most k-mismatches in at most n/ℓ positions j. Then, in the positions where S matches with less than k-mismatches we apply the kangaroo method to find the position of the k-mismatch as we did when we switched to work with a pattern with at most $2k$ ℓ-break. This adds $O(nk/\ell)$ to the running time and allows us to compare S, that has some useful properties, rather than P, to T.

To define S we partition each periodic stretch of P into *segments* of length ℓ except for a piece at the end of length smaller than ℓ which we call the *tail* of the periodic stretch.

We define S to be the shortest prefix of P that satisfies at least one of the following criteria, or P itself if no prefix of P satisfies at least one of these criteria.

1. S contains a multiset A of $2k$ segments of periodic stretches, such that at most k/ℓ of the segments of A are of the same canonical period.

2. S contains a multiset of $2k$ characters in which each character appears at most k/ℓ times.

The following lemmas show that if $|S| < |P|$, then S cannot match with less than k-mismatches in too many places. These lemmas are similar to Lemma 2 in [6].

Lemma 6.2.1 *Suppose that S contains a multiset A of $2k$ segments such that at most k/ℓ of them have the same canonical period. Then, there are at most n/ℓ positions in the text T that match S with at most k mismatches.*

Proof: For each distinct $u \in A$ let B_u be the set of start indices in the pattern of each occurrence of segment u in A. For each text position j in which an occurrence of some segment $u \in A$ starts, we mark positions $j - i$ in the text for every $i \in B_u$. The marked positions are those that if we align the pattern there, segment u at position j of the text is matched against one of the occurrences of u in A. Since each segment appears at most k/ℓ times in A, we have a total of at most nk/ℓ marks. Each position of the text which S

matches with at most k mismatches, must have at least k marks. So we have at most n/ℓ such positions. ■

The proof of the following lemma is similar to that of Lemma 6.2.1.

Lemma 6.2.2 *Let S be a string that contains a multiset of $2k$ characters such that the number of copies of each character in the multiset is at most k/ℓ. Then, there are at most n/ℓ positions in the text T that match S with at most k mismatches.*

We need the following definitions to specify the useful properties that S has. Let C be the set of canonical periods of the periodic stretches in S. A period $u \in C$ is *frequent* in S, if there are at least k/ℓ segments in periodic stretches with period u, otherwise u is *rare*. Similarly, we define a character to be *frequent* in S, if it appears at least k/ℓ times in S, and *rare* otherwise. The prefix S has the following properties.

1. S contains at most 2ℓ frequent periods. If S contains more than 2ℓ frequent periods, then we can obtain a shorter S satisfying Criterion (1) by taking the shortest prefix that contains k/ℓ segments of each of exactly 2ℓ frequent periods. By a similar argument, the total number of segments of periodic stretches that belong to rare periods in S is at most $2k$.

2. S contains at most 2ℓ frequent characters. Furthermore, the total number of occurrences of rare characters in S is at most $2k$.

We add all segments and tails of rare periodic stretches to the set of breaks. By Property (1) we added $O(k)$ breaks of length at most ℓ. Following these changes, S still contains $O(k)$ breaks but the set C of periods of periodic stretches is of size $O(\ell)$.

We now give two examples of patterns P and their corresponding prefixes S.
Example 1: Let

$$P = (ab)^{10}axa^{15}xb^{15}xc^{15}x(ac)^{10}ax(cb)^{10}cxd^{10}x(ad)^{10}ax(bd)^{10}bx(cd)^{10}cx(ea)^{2}x \ ,$$

and assume that $k = 8$ and $\ell = 4$. The partition of P into breaks and periodic as described in section 6.1 is as follows.

$$(ab)^9|\overline{abax}|a^{12}|\overline{aaax}|b^{12}|\overline{bbbx}|c^{12}|\overline{cccx}|(ac)^9|\overline{acax}|(cb)^9|\overline{cbcx}|d^7|\overline{dddx}|(ad)^9|\overline{adax}|(bd)^9|\overline{bdbx}|(cd)^9|\overline{cdcx}|e|\overline{aeax} \ ,$$

where the breaks are the substrings marked with an overline. Other substrings are periodic stretches. Substrings are separated by the symbol $|$.

Notice that the period of the periodic stretch $(cb)^9$ is cb and cb is not a canonical period. Therefore we write $(cb)^9$ as $c(bc)^8b$ and add the prefix c and the suffix b to the breaks. Also since the periodic stretch e is of length smaller than ℓ we add it to the set of breaks. So following these modifications the partition into breaks and periodic stretches is as follows.

$$(ab)^9|\overline{abax}|a^{12}|\overline{aaax}|b^{12}|\overline{bbbx}|c^{12}|\overline{cccx}|(ac)^9|\overline{acax}|\overline{c}|(bc)^8|\overline{b}|\overline{cbcx}|d^7|\overline{dddx}|(ad)^9|\overline{adax}|(bd)^9|\overline{bdbx}|(cd)^9|\overline{cdcx}|\overline{e}|\overline{aeax}$$

Since $|\Sigma| = 6$, we cannot find a prefix S of P that contains a multiset of $2k = 16$ characters each occurring at most $k/\ell = 2$ times. But we can find a prefix S of P for which the first criteria holds. That it a prefix S that contains a multiset of $2k = 16$ segments (of length $\ell = 4$) each occurring at most $k/\ell = 2$ times. This string S is the following prefix of P.

$$(ab)^9|\overline{abax}|a^{12}|\overline{aaax}|b^{12}|\overline{bbbx}|c^{12}|\overline{cccx}|(ac)^9|\overline{acax}|\overline{c}|(bc)^8|\overline{b}|\overline{cbcx}|d^7|\overline{dddx}|(ad)^9|\overline{adax}|(bd)^2 \ .$$

The set of canonical periods in this example is $\{ab, a, b, c, ac, bc, d, ad, bd\}$. In this prefix S we have two segments of each period in the set $\{ab, a, b, c, ac, bc, ad\}$, and one segment of each of the periods d and bd. Notice that all periods of the periodic stretches are frequent except for the period d of the periodic stretch d^7, which is a rare period. Thus we added the segment d^4 and the tail d^3 to the set of breaks. The final partition of S is:

$$(ab)^9|\overline{abax}|a^{12}|\overline{aaax}|b^{12}|\overline{bbbx}|c^{12}|\overline{cccx}|(ac)^9|\overline{acax}|\overline{c}|(bc)^8|\overline{b}|\overline{cbcx}|\overline{d^4}|\overline{d^3}|\overline{dddx}|(ad)^9|\overline{adax}|(bd)^2 \ .$$

Example 2:
Let

$$P = (ab)^{10}xe^{15}xf^{15}yc^{15}y(ac)^{10}ay(bc)^{10}bxg^{15}x(ad)^{10}ax(bd)^{10}bx(cd)^{10}cx \ ,$$

and assume still that $k = 8$ and $\ell = 4$.

The final partition into breaks and periodic stretches is as follows 3.

$$(ab)^8|\overline{a}|\overline{babx}|e^{12}|\overline{eeex}|f^{12}|\overline{fffy}|c^{12}|\overline{cccy}|(ac)^9|\overline{acay}|(bc)^9|\overline{bcbx}|g^{12}|\overline{gggx}|(ad)^9|\overline{adax}|(bd)^9|\overline{bdbx}|(cd)^9|\overline{cdcx} \ .$$

Here we choose the prefix

$$(ab)^8|\overline{a}|\overline{babx}|e^{12}|\overline{eeex}|f^{12}|\overline{fffy}|c^{12}|\overline{cccy}|(ac)^9|\overline{acay}|(bc)^9|\overline{bcbx}|g^2 \ .$$

This prefix contains a multiset of 16 characters such that each character appears at most twice. (The different characters are a, b, c, x, y, e, f, g).

Finding the position of the k-mismatch in S. We partition S into at most $O(k/y)$ *substrings* each containing at most y breaks, at most y occurrences of rare characters, and at most y periodic stretches. First, in what we call the *preprocessing stage*, we compute for each text position j the substring $W(j)$ of S that contains the k-mismatch of S with $T(j, |S|)$, or decide that S matches $T(j, |S|)$ with less than k-mismatches.

To do that we process the substrings sequentially from left to right, maintaining for each text position j the cumulative number of mismatches of the text starting at position j with the substrings processed so far. We denote this cumulative mismatch count of position j by $r(j)$. Let the next substring W of P that we process start at position i of the pattern. For each text position j, we compute the number of mismatches of $T(j, |W|)$ with W and denote it by $c(j)$. (We show below how to do that.) Then, for each text position j for which we have not yet found the substring that contains the k-mismatch, we update the

information as follows. If $r(j) + c(j+i) < k$, we set $r(j) = r(j) + c(j+i)$. Otherwise, $r(j) + c(j+i) \geq k$, and we set $W(j)$ to be the substring W.

We now show how to find the number of mismatches between a substring W of S and $T(j, |W|)$ for every $1 \leq j \leq n - |W| + 1$. We do that by separately counting the number of mismatches between occurrences of frequent characters in W and the corresponding characters of $T(j, |W|)$, and the number of mismatches between occurrences of rare characters in W and the corresponding characters of $T(j, |W|)$. Then we add these two counts.

By Property 2, W contains at most 2ℓ frequent characters. For each frequent character x we find the number of mismatches of the occurrences of x in W with the corresponding characters in $T(j, |W|)$ for all j, by performing a convolution as described in the introduction. We perform $O(\ell)$ convolutions for each of the $O(k/y)$ substrings, so the total time to perform all convolutions is $O(\frac{k}{y}\ell n \log m)$.

It remains to find the number of mismatches of rare characters in W with the corresponding characters in $T(j, |W|)$. We do that using the algorithm of Amir et al. [6]. This algorithm counts the number of mismatches of a pattern which may contain don't care symbols with each text position. The running time of this algorithm is $O(n\sqrt{g \log m})$, where g is the number of characters in the pattern that are not don't cares. We run this algorithm with a pattern which we obtain from W by replacing each occurrence of a frequent character by a don't care symbol, and the text T. We obtain for each j the number of mismatches between rare characters in W and the corresponding characters in $T(j, |W|)$. Since W contains at most y occurrences of rare characters, the running time of this application of the algorithm of Abrahamson is $O(n\sqrt{y \log m})$. So for all $O(k/y)$ substrings this takes $O(\frac{k}{y} n\sqrt{y \log m}) = O(\frac{k}{y^{1/2}} n\sqrt{\log m})$ time. This completes the description of the preprocessing phase.

We now show how to find the position of the k-mismatch within the substring $W(j)$. Recall that $W(j)$ contains at most y breaks and at most y periodic stretches. Each periodic stretch is of the form u^i, where $u \in C$, and $|C| \leq 2\ell$.

We begin by finding for each text position which periodic stretch or break contains the k-mismatch. We find it by performing a binary search on the periodic stretches and breaks in $W(j)$, simultaneously for all text positions j. After iteration h of the binary search, for each text position we focus on an interval of at most $y/2^h$ consecutive breaks and periodic stretches in $W(j)$ that contains the k-mismatch. In particular, after $\log y$ iterations, we know for each text position which periodic stretch or break contains the k-mismatch.

At the first iteration of the binary search we compute the number of mismatches in the first $y/2$ of the periodic stretches and breaks of $W(j)$. From this number we know if the k-mismatch is in the first $y/2$ breaks and periodic stretches or in the last $y/2$ breaks and periodic stretches of $W(j)$. In iteration h, let $I(j)$ be the interval of $y/2^h$ consecutive breaks and periodic stretches in $W(j)$ that contains the k-mismatch between $W(j)$ and the corresponding substring of $T(j, |S|)$. We compute the number of mismatches between the first $y/2^{h+1}$ breaks and periodic stretches in $I(j)$ and the corresponding part of $T(j, |S|)$.

Using this count we know if to proceed with the first half of $I(j)$ or the second half of $I(j)$.

We describe the first iteration of the binary search. Subsequent iterations are similar. We count the number of mismatches in each of the first $y/2$ breaks in $W(j)$ and the appropriate substring of $T(j, |S|)$ by comparing them character by character in $O(y\ell)$ time for a specific j, and $O(ny\ell)$ total time.

To count the number of mismatches in each of the first $y/2$ periodic stretches we process the different periods in C one by one. For each period $u \in C$ and each text position j we count the number of mismatches in periodic stretches of period u among the first $y/2$ periodic stretches of $W(j)$. The sum of these mismatch-counts over all periods $u \in C$ gives us the total number of mismatches in the first $y/2$ periodic stretches of $W(j)$ and $T(j, |S|)$ for every text position j.

Let $u \in C$. We compute the number of mismatches of u with each text position using the algorithm of Abrahamson [1] in $O(n\sqrt{\ell \log \ell})$ time. We construct $|u|$ prefix sums arrays $A_i, i = 1, \cdots, |u|$, each of size $n/|u|$. We use these arrays to find the number of mismatches of periodic stretches with period u among the first $y/2$ periodic stretches of $W(j)$ for all text positions j. The total size of the arrays is $O(n)$.

The entries of array the A_i correspond to the text characters at positions β such that β modulo $|u| = i$ modulo $|u|$. The first entry of array A_i contains the number of mismatches between $T(i, |u|)$ and u that was computed by the algorithm of Abrahamson. Entry j in A_i contains the number of mismatches between $T(i, j|u|)$ and u^j. It is easy to see that based on entry $j-1$, entry j in A_i can be computed in $O(1)$ time by setting $A_i[j]$ to $A_i[j-1]$ plus the number of mismatches between $T(i + (j-1)|u|, |u|)$ and u. Suppose we need to find the number of mismatches of $T(i + j|u|, r|u|)$ with a periodic stretch u^r. The number of mismatches can be computed in $O(1)$ time given A_i. If $j = 0$, then the number of mismatches is $A_i[r]$. and if $j > 0$, then the number of mismatches is $A_i[j + r] - A_i[j]$.

In each iteration of the binary search we repeat the procedure above for every $u \in C$. Since $|C| = O(\ell)$ we compute the number of mismatches of all periodic stretches in the first $y/2$ periodic stretches of $W(j)$ for all j, in $O(n\ell^{\frac{3}{2}}\sqrt{\log \ell})$ time. Summing up over all iterations the time of counting the number of mismatches within breaks and the time of counting the number of mismatches within periodic stretches, we obtain that the binary search takes $O(n\ell^{\frac{3}{2}}\sqrt{\log \ell} \log y) + O(ny\ell)$ time.

We now know for each text position which periodic stretch or break contains the k-mismatch. If the k-mismatch is contained within a break we find it in $O(\ell)$ time by scanning the break character by character. If the k-mismatch is contained in a periodic stretch, then we find it as follows. For each $u \in C$ we build the $n/|u|$ prefix sum arrays, A_i, as described above. We then compute the position of the k-mismatch, for all text positions for which the k-mismatch occurs in a periodic stretch of period u. Given such text position, we perform a binary search on the appropriate prefix sum array to locate a segment of length $|u|$ within the periodic stretch that contains the k-mismatch. The binary search is performed on a sub-array of length at most $|S|/|u|$ in $O(\log |S|)$ time. At the end

of the binary search, we found the segment of length $|u| < \ell$ that contains the k-mismatch, we search in this segment sequentially in $O(\ell)$ time to find the k-mismatch. We repeat this process for all the periods in C. The running time of this final search within a break or a periodic stretch for each text position is dominated by the running time of the previous steps.

Summing over all stages we obtain that the total running time of the algorithm is $O(\frac{k}{y}\ell n \log m) + O(\frac{k}{y^{1/2}} n\sqrt{\log m}) + O(n\ell^{\frac{3}{2}}\sqrt{\log \ell}\log y) + O(ny\ell)$. The space used by the algorithm is $O(n)$. We also recall that we have to add $O(nk/\ell)$ overhead to the overall running time that follows from the time we spend applying the kangaroo method in positions where S matches with less than k-mismatches. Choosing $\ell = k^{\frac{1}{4}}/\log^{\frac{1}{4}} m$ and $y = \sqrt{k \log m}$ we balance the expressions above and thereby minimize the running time which comes out to be $O(nk^{\frac{3}{4}} \log^{\frac{1}{4}} m)$.

6.3 Bootstrapping to improve the running time

In this section we show that by repeating the algorithm of Section 6.2 with different values of ℓ we can improve the time bound to $O(nk^{\frac{2}{3}} \log^{\frac{1}{3}} m \log k)$. The motivation behind this, is that we want to reduce the running time by performing the kangaroo method on fewer positions. After performing the algorithm once with $\ell = \ell_1$ we are left with n/ℓ_1 places for which we need to find the position of the k-mismatch, now instead of performing the kangaroo method on these text positions, we repeat the algorithm with $\ell = \ell_2 > \ell_1$. In the second execution of the algorithm, we search for the k-mismatch only in at most n/ℓ_1 positions in which we did not find the k-mismatch in the first execution. At the end of the second execution, we are left with $n/\ell_2 < n/\ell_1$ positions for which we need to find the position of the k-mismatch.

We execute the algorithm of Section 6.2, $t = \left\lceil \log \frac{k^{1/3}}{\log^{1/3} m} \right\rceil$ times with increasing values of the parameter ℓ and with certain modifications. In the j-th iteration $j = 1, \cdots, t$, we use the parameter $\ell_j = 2^{j+1}$. In iteration j we truncate the pattern if necessary to contain at most $2k$ ℓ-breaks and then choose a prefix S_j of the pattern as in Section 6.2. The algorithm performs the following steps.

1. For each iteration j, we run the preprocessing stage. Let $y = k^{\frac{2}{3}} \log^{\frac{1}{3}} m$. In this stage, we partition the prefix S_j of the pattern into k/y substrings, each substring contains at most y breaks, at most y periodic stretches, and at most y rare characters. We find the number of mismatches between each text position and each substring by performing convolutions as described in Section 6.2. The running time of this stage for iteration j is $O(\frac{k}{y^{1/2}} n\sqrt{\log m}) + O(\frac{k}{y}\ell_j n \log m)$.

2. After we run the preprocessing stage of the j-th iteration, we search for the k-th mismatch within the appropriate substring (see Section 6.2). We only search at the

positions for which we did not find the k-th mismatch in the previous iteration. That is, for $j = 1$ we search for the position of the k-mismatch in at most n places. For $j > 1$, at most n/ℓ_{j-1} positions were matched with S_{j-1} with at most k mismatches. Thus, in the jth iteration we search for the k-mismatch in at most n/ℓ_{j-1} positions. Nevertheless we still need to build the prefix sum arrays described in Section 6.2. It follows that the cost of the search within a block in iteration 1 is

$$n\ell_1^{\frac{3}{2}}\sqrt{\log \ell_1}\log y + n\ell_1 y \ ,$$

and the cost for the search within a block in iteration $1 < j \leq i$ is

$$n\ell_j^{\frac{3}{2}}\sqrt{\log \ell_j}\log y + \frac{n}{\ell_{j-1}}\ell_j y \ .$$

3. After performing the previous step for every iteration $1 \leq j \leq t$ we are left with at most n/ℓ_t positions in text that may match S_t with at most k mismatches. We verify these positions using the kangaroo method in $O(nk/\ell_t)$ time.

We show that the total preprocessing time of the algorithm for all iterations $1 \leq j \leq t$ is $O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m \log k)$ and the time to search within a block for all iterations $1 \leq j \leq t$ is $O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m \log k)$. This gives a running time of $O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m \log k)$.

The preprocessing time for a particular iteration $1 \leq j \leq t$ is

$$O\left(\frac{k}{y^{1/2}} n\sqrt{\log m}\right) + O\left(\frac{k}{y}\ell_j n \log m\right) \ .$$

Since $\ell_j = 2^{j+1}$, $y = k^{\frac{2}{3}}\log^{\frac{1}{3}} m$, and $\ell_t = O\left(\frac{k^{\frac{1}{3}}}{\log^{\frac{1}{3}} m}\right)$, when we sum up we obtain that the total time of the preprocessing stage of all iterations is:

$$O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m \log k) + O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m).$$

The time to search within a block for $j = 1$ is

$$O(n\ell_1^{\frac{3}{2}}\sqrt{\log \ell_1}\log y + ny\ell_1) = O(ny) = O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m).$$

The time to search within a block for $j > 1$ is $O(n\ell_j^{\frac{3}{2}}\sqrt{\log \ell_j}\log y + ny\ell_j/\ell_{j-1})$. Since

$$n\ell_j^{\frac{3}{2}}\sqrt{\log \ell_j}\log y \leq n2^{\frac{3(j+1)}{2}}\sqrt{j}\log y = o(nk^{\frac{2}{3}}) \ ,$$

and

$$ny\ell_j/\ell_{j-1} = ny2^{j+1}/2^j = O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m),$$

the total running time for searching within a block for all iterations is $O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m \log k)$.

Finally, the cost of running the kangaroo method on at most $n/\ell_t = O\left(n\frac{\log^{\frac{1}{3}} m}{k^{\frac{1}{3}}}\right)$ positions where the pattern may match with at most k mismatches, is $O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m)$, so the total running time is $O(nk^{\frac{2}{3}}\log^{\frac{1}{3}} m \log k)$.

6.4 A deterministic algorithm for the approximate k-mismatch problem

We now give an algorithm that given a text T, and a pattern P, finds for every text position that does not match the pattern with at most k mismatches, a position in the pattern which is between the position of the $(1-\epsilon)k$-mismatch and the position of the $(1+\epsilon)k$-mismatch. The running time of the algorithm is $O(\frac{1}{\epsilon^2}n\sqrt{k}\log^3 m)$. We call such a position an *approximate k-mismatch* (although there need not be a mismatch at that position).

As in Section 6.2 we assume that the pattern contains $O(k)$ breaks, which are substrings of length at most ℓ, and at most $2k$ periodic stretches. All periodic stretches are of the form u^i, where u is a canonical period.

The algorithm is similar to the algorithm of Section 6.2. The main difference is that instead of using convolutions or the algorithm of Abrahamson [1] (that uses convolutions), to count the number of mismatches between parts of the pattern and the text, we use the algorithm of Karloff [54]. Given a pattern P and a text T, the algorithm of Karloff [54], finds in $O(\frac{n}{\epsilon^2}\log^3 m)$ time for every text position $1 \leq j \leq n-m+1$, an *approximate number of mismatches* $g(j)$ such that $f(j) \leq g(j) \leq (1+\epsilon)f(j)$, where $f(j)$ is the exact number of mismatches between P and $T(j,m)$.

We find for each text position j, a position $s(j)$ within the pattern, such that the approximate number of mismatches, g, between $P(1,s(j))$ and $T(j,s(j))$, excluding mismatches in a set of substrings E of total length ϵk, is between $(1-\epsilon)k$ and $(1+\epsilon)k$. More precisely, if f is the exact number of mismatches between $P(1,s(j))$ and $T(j,s(j))$ excluding E, then $f \leq g \leq (1+\epsilon)f$ and $(1-\epsilon)k \leq g \leq (1+\epsilon)k$. ¿From these two inequalities we obtain that $\frac{(1-\epsilon)k}{1+\epsilon} \leq f \leq (1+\epsilon)k$ which implies that $(1-2\epsilon)k \leq f \leq (1+\epsilon)k$. Since the total length of E is ϵk, then if f' is the number of mismatches between $P(1,s(j))$ and $T(j,s(j))$ (including mismatches in E) then $(1-2\epsilon)k \leq f' \leq (1+2\epsilon)k$. So position $s(j)$ is an approximate k-mismatch for $\epsilon' = 2\epsilon$.

Our algorithm uses the fact that the approximate counts of mismatches are additive in the following sense. Let $S = S_1S_2$ and let $T = T_1T_2$, where $|S_1| = |T_1|$ and $|S_2| = |T_2|$. We can get an approximation to the number of mismatches between S and T from an approximation to the number of mismatches between S_1 and T_1 and an approximation to the number of mismatches between S_2 and T_2. Indeed if f_1 is the exact number of mismatches between S_1 and T_1, and g_1 is an approximation that satisfies $f_1 \leq g_1 \leq (1+\epsilon)f_1$, and similarly f_2 is the exact number of mismatches between S_2 and T_2, and g_2 is an approximation that satisfies $f_2 \leq g_2 \leq (1+\epsilon)f_2$, then g_1+g_2 is an approximation to the number of mismatches between S and T that satisfies, $f_1+f_2 \leq g_1+g_2 \leq (1+\epsilon)(f_1+f_2)$, where f_1+f_2 is the exact number of mismatches between S and T. Thus we can approximate the number of mismatches between substrings using the algorithm of Karloff and then add the approximations to get an approximate number of mismatches between

the concatenation of the substrings.

We choose the shortest prefix S of P that satisfies the first of the two criteria of Section 6.2. Let C be the set of canonical periods of the periodic stretches. We change the partition and add to the breaks $O(k)$ segments and tails each of length $\leq \ell$ that belong to rare periodic stretches (See Section 6.2). Following these changes, S contains $O(k)$ breaks. The set C of periods of the periodic stretches is of size $O(\ell)$. Using the analysis of Section 6.2, at most n/ℓ positions in the text match S with at most k mismatches. For every position j of T that S matches with more than k mismatches we can find $s(j)$ within S. We use the kangaroo method to find the k-mismatch in P for each text position that matches S with at most k mismatches in $O(nk/\ell)$ time.

We partition S into at most $O(k/y)$ substrings each containing at most y breaks and at most y periodic stretches. Consider a text position j such that there are more than k mismatches between S and $T(j, |S|)$. Let $W(j)$ be the first substring such that the approximate number of mismatches between the prefix of S up to and including $W(j)$ and the text starting at position j is at least k. We process the substrings from left to right, and for each text position j such that there are more than k mismatches between S and $T(j, |S|)$ we find $W(j)$. We do that as explained in Section 6.2, but with the algorithm of Karloff [54] to approximate the number of mismatches of each text position and a substring in $O(\frac{n}{\epsilon^2} \log^3 m)$ time. It takes $O(\frac{k}{y} \frac{n}{\epsilon^2} \log^3 m)$ time to find $W(j)$ for all text positions j.

Now for each text position such that there are more than k mismatches between S and $T(j, |S|)$ we find $s(j)$ within $W(j)$ as follows. Recall that $W(j)$ has at most y breaks and y periodic stretches. So the total length of the breaks in $W(j)$ is at most $y\ell$. We ignore the mismatches within the breaks of $W(j)$. This set of breaks is the set E of substrings which we exclude. By choosing y and ℓ such that $y\ell < \epsilon k$, we obtain that the total length of the substrings in E is ϵk.

We look for a periodic stretch within $W(j)$ that contains $s(j)$. Let g' be the approximate number of mismatches between the text at position j and the prefix of the pattern up to and excluding $W(j)$, and let α be the approximate number of mismatches of the periodic stretches in $W(j)$ with the corresponding substring of the text. By the definition of $W(j)$, $g' < k$, and $g' + \alpha \geq (1 - \epsilon)k$. We find the first periodic stretch $p(j)$ in $W(j)$ such that the approximate number of mismatches of S up to and including $p(j)$ with the text at position j excluding the substrings of E is at least $(1 - \epsilon)k$.

We find $p(j)$ by performing a binary search on the periodic stretches similar to that of Section 6.2. At the first iteration of the binary search we approximate the number of mismatches in the first $y/2$ periodic stretches of $W(j)$. From the result we know if $p(j)$ is in the first $y/2$ periodic stretches or in the last $y/2$ periodic stretches of $W(j)$. After $O(\log y)$ iterations of this binary search we discover $p(j)$. We describe the first iteration of the binary search, the following iterations are similar. For each period u in C we approximate the number of mismatches of u with each text position using the algorithm of Karloff [54] in $O(\frac{1}{\epsilon^2} n \log^3 m)$ time. As in Section 6.2 we construct $|u|$ prefix sums arrays

each of size $O(n/|u|)$. We use the arrays to approximate the number of mismatches in each periodic stretch with period u in $O(1)$ time. We approximate the number of mismatches in all periodic stretches with period u, among the first $y/2$ periodic stretches of $W(j)$ (for all positions j simultaneously). We then repeat this for all periods in C, and find the approximate number of mismatches of each text position with the first $y/2$ periodic stretches. The entire binary search for all text positions takes $O(\ell \frac{1}{\epsilon^2} n \log^3 m \log y) + O(ny)$ time.

Last we have to find for each text position j a position $s(j)$ within $p(j)$. Let $p(j) = u^\gamma$ for some canonical period $u \in C$, and $\gamma \geq 1$. We find the smallest $\delta \leq \gamma$ such that the number of mismatches of the prefix of S that ends with the prefix u^δ of $p(j)$, with text position j is at least $(1-\epsilon)k$. We set $s(j)$ to be the position of the last character in this prefix.

For each period u in C, we construct again the prefix sum arrays mentioned before to compute the position of the k-mismatch, for all text positions for which the approximate k-mismatch is contained within a periodic stretch with period u. Given such a text position, we find $s(j)$ by performing a binary search on a sub-array of length at most $|S|/|u|$ of the appropriate prefix sum array in $O(\log |S|)$ time. We repeat this process for all the periods in C.

The total running time of this algorithm is $O(\frac{k}{y}\frac{n}{\epsilon^2} \log^3 m) + O(\ell \frac{1}{\epsilon^2} n \log^3 m \log y) + O(ny) + O(nk/\ell)$. If we set $\ell = \epsilon\sqrt{k}/\log k, y = \sqrt{k}$, we get that $y\ell = \epsilon k/\log k$, and the total length of the breaks within each substring is at most $y\ell = \epsilon k/\log k$. This establishes the correctness of the algorithm. The running time is $O(\frac{1}{\epsilon^2} n\sqrt{k} \log^3 m)$.

6.5 A Randomized Algorithm for the approximate k–mismatch problem

In this section we present a randomized algorithm that finds for each text position i a position $s(i)$ in the pattern such that the number of mismatches between $T(i, s(i))$ to the prefix of the pattern of length $s(i)$ is between $(1-\epsilon)k$ to $(1+\epsilon)k$ with high probability. The running time of the algorithm is $O(\frac{n}{\epsilon^2} \log n \log^2 m \log k)$.

We assume w.l.o.g. that the alphabet Σ consists of the integers $\{1, \cdots, |\Sigma|\}$. The algorithm computes signatures for substrings of the pattern and the text. These signatures are designed such that from the signatures of two strings we can approximate the number of mismatches between the two strings with high probability. We construct a random string R *of sparsity* k by setting $R[i]$ to 0 with probability $(1-\frac{1}{k})$, and setting $R[i]$ to be a random integer with probability $\frac{1}{k}$, for every $i = 1, \cdots, |R|$. We choose a random integer from a space Π of size polynomial in n. For a string W and a random string R with sparsity k, we define the signature of W with respect to R as $Sig_k(W, R) = \sum_{i=1}^{|W|} W[i]R[i]$.

Let W_1 and W_2 be two strings of the same length. If W_1 and W_2 agree in all positions

where $R[i] \neq 0$, then $Sig_k(W_1, R) = Sig_k(W_2, R)$. On the other hand, if W_1 and W_2 disagree in at least one position i where $R[i] \neq 0$, then $Sig_k(W_1, R) = Sig_k(W_2, R)$ with probability at most $\frac{1}{|\Pi|}$. Let us call the latter event a *bad event*. Our algorithm compares sub-quadratic number of signatures so by choosing Π large enough, we can make the probability that a bad event ever happens polynomially small. Therefore, we assume in the rest of the section that such event does not happen. So $Sig_k(W_1, R) = Sig_k(W_2, R)$ if and only if W_1 and W_2 agree in all positions where $R[i] \neq 0$.

For $k \geq 2$ we define an algorithm A_k as follows. The input to A_k consists of a substring S of the text and a substring W of the pattern, such that S and W are of the same length. Let y be the true number of mismatches between S and W. The algorithm A_k either detects that $y > 2k$, or detects that $y < k$, or returns an estimate y' of y. The algorithm A_k works as follows. Let $q = \frac{c}{\epsilon^2} \log n$ for some large enough constant c that we determine later, and let $b = |W| = |S|$. Algorithm A_k takes q random strings $R_1, \cdots, R_q$ of length b and sparsity k and compares $Sig_k(W, R_i)$ and $Sig_k(S, R_i)$ for $i = 1, \ldots, q$. Let z be the number of equal pairs of signatures. If $z \geq (1-\epsilon)q(1-\frac{1}{k})^{k/2}$ then A_k reports that the number of mismatches between S and W is smaller than k. If $z \leq (1+\epsilon)q(1-\frac{1}{k})^{3k}$ then A_k reports that the number of mismatches between S and W is greater than $2k$. Otherwise let y' be the largest integer such that $z \leq q(1-\frac{1}{k})^{y'}$. Algorithm A_k returns y' as our estimate of y. One can easily check that if $\epsilon < 1/5$ then $(1+\epsilon)q(1-\frac{1}{k})^{k} \leq (1-\epsilon)q(1-\frac{1}{k})^{k/2}$ and $(1+\epsilon)q(1-\frac{1}{k})^{3k} < (1-\epsilon)q(1-\frac{1}{k})^{2k}$ so our algorithm is well defined.

In the following lemmas we establish that A_k satisfies the following properties with high probability.

1. If $y \leq k/2$ then A_k reports that the number of mismatches is smaller than k.
2. If $y \geq 3k$ then A_k reports that the number of mismatches is larger than $2k$.
3. If $k \leq y \leq 2k$ then A_k gives an estimate y' of y.
4. Whenever A_k gives an estimate y' of y then $(1-\epsilon)y \leq y' \leq (1+\epsilon)y$. (This can happen if $k/2 < y < 3k$ and happens with high probability if $k \leq y \leq 2k$.)

For $k < 2$ we build a generalized suffix tree for P and T. We use this suffix tree to check whether the number of mismatches between a substring of P and a substring of T is at most 2, and if so to find it exactly, by the kangaroo method. We shall refer to this procedure as A_0.

Lemma 6.5.1 *Let S be a substring of the text and let W be a substring of the pattern such that the number of mismatches between S and W is y. Let A_k, for some $k \geq 2$, compare $q = \frac{c}{\epsilon^2} \log n$ signatures of S and W and let z be number of times the signatures were equal. If $y < k/2$ then with high probability $z \geq (1-\epsilon)q(1-\frac{1}{k})^{k/2}$. If $y > 3k$ then with high probability $z \leq (1+\epsilon)q(1-\frac{1}{k})^{3k}$. If $k \leq y \leq 2k$ then with high probability $(1-\epsilon)q(1-\frac{1}{k})^{2k} \leq z \leq (1+\epsilon)q(1-\frac{1}{k})^{k}$.*

Proof: Let p_i be the probability that the i-th signatures of W and S are equal. The probability that $Sig_k(S, R_i) = Sig_k(W, R_i)$ is in fact the probability that for all j such $W[j] \neq S[j]$, we have that $R_i[j] = 0$. Since for each j, $R_i[j] = 0$ with probability $1 - \frac{1}{k}$, it follows that $p_i = (1 - \frac{1}{k})^y$. Let X_i be the random variable that is equal to 1 if the i-th signatures of W and S are identical and is equal to 0 otherwise. Clearly, $E(X_i) = (1 - \frac{1}{k})^y$. Let $X = \sum_{i=1}^{q} X_i$, then by the linearity of expectation $E(X) = q(1 - \frac{1}{k})^y$. If $y < k/2$, then $E(X) > q(1 - \frac{1}{k})^{k/2} \geq q/e$. Using Chernoff bound

$$Pr(X < (1 - \epsilon)E(X)) < e^{\frac{-\epsilon^2 E(X)}{2}},$$

we get that with probability at most $e^{\frac{-\epsilon^2\left(\frac{c}{\epsilon^2 e}\log n\right)}{2}} \cong \frac{1}{n^{c/2e}}$, $z < (1 - \epsilon)q(1 - \frac{1}{k})^y$. So $z \geq (1 - \epsilon)q(1 - \frac{1}{k})^y \geq (1 - \epsilon)q(1 - \frac{1}{k})^{k/2}$ with probability at least $1 - \frac{1}{n^{c/2e}}$.

If $y > 3k$, we show that with probability $\left(1 - \frac{1}{n^{O(c)}}\right)$, $z \leq (1 + \epsilon)q(1 - \frac{1}{k})^{3k}$. Let $i_1, \cdots, i_{3k}$, be the indices of the first $3k$ mismatches of W and S. Let p'_i be the probability that none of these $3k$ mismatches was chosen by the i-th signature of W and S, that is p'_i is the probability that $R_i[i_j] = 0$ for $i_j = i_1, \cdots i_{3k}$. It is easy to see that $p'_i = (1 - \frac{1}{k})^{3k}$. Let Y_i be the random variable that is equal to 1 if $R_i[i_j] = 0$ for $i_j = i_1, \cdots i_{3k}$, and $Y_i = 0$ otherwise. Clearly, $E(Y_i) = (1 - \frac{1}{k})^{3k}$. Let $Y = \sum_{i=1}^{q} Y_i$, then $E(Y) = q(1 - \frac{1}{k})^{3k}$. Notice that $X \leq Y$ where we recall that X is the number of signatures of W and S that match. Using Chernoff bound

$$Pr(Y > (1 + \epsilon)E(Y)) < e^{\frac{-\epsilon^2 E(Y)}{3}} < e^{\frac{-\epsilon^2\left(\frac{c}{\epsilon^2}\log n\right)(1-\frac{1}{k})^{3k}}{3}} \leq e^{\frac{-\frac{c}{64}\log n}{3}} = \frac{1}{n^{c/192}}.$$

So we have that with probability $\left(1 - \frac{1}{n^{O(c)}}\right)$, $X \leq Y \leq (1 + \epsilon)q(1 - \frac{1}{k})^{3k}$.

Similarly, if $k \leq y \leq 2k$, we get as a direct result of Chernoff bounds that with probability at least $1 - \frac{1}{n^{O(c)}}$, $(1 - \epsilon)q(1 - \frac{1}{k})^{2k} \leq z \leq (1 + \epsilon)q(1 - \frac{1}{k})^k$. ∎

Lemma 6.5.1 implies that A_k satisfies Properties 1-3. The next lemma proves that A_k also satisfies Property 4.

Lemma 6.5.2 *Let S be a substring of the text and W be a substring of the pattern such that the number of mismatches between S and W is y. Let A_k, for some $k \geq 2$, compare $q = \frac{c}{\epsilon^2}\log n$ signatures of S and W and let z be number of times the signatures were equal. If $k/2 \leq y \leq 3k$, and the algorithm returns an estimate y' of the number of mismatches between S and W, then $(1 - \epsilon)y \leq y' \leq (1 + \epsilon)y$ with high probability.*

Proof: Let p_i be the probability that the i-th signature of W and S are identical. Then, as in the proof of Lemma 6.5.1, we have that $p_i = (1 - \frac{1}{k})^y$. Let X_i be a random variable that is equal to 1 if the i-th signatures of W and S are identical and to 0 otherwise. Then, $E(X_i) = (1 - \frac{1}{k})^y$. Let $X = \sum_{i=1}^{q} X_i$, then by linearity of expectation $E(X) = q(1 - \frac{1}{k})^y$. Using Chernoff bounds, we know that

$$Pr(X > (1 + \epsilon)E(X)) < e^{\frac{-\epsilon^2 E(X)}{3}} \tag{6.1}$$

$$Pr(X < (1-\epsilon)E(X)) < e^{\frac{-\epsilon^2 E(X)}{2}} \tag{6.2}$$

Substituting $E(X)$, we get that

$$Pr(X > (1+\epsilon)E(X)) < e^{\frac{-\epsilon^2\left(\frac{c}{\epsilon^2}\log n\right)(1-\frac{1}{k})^y}{2}}$$

If $y \leq 3k$, then $(1-\frac{1}{k})^y \geq 1/64$. So we get that $Pr(X > (1+\epsilon)E(X)) \leq \frac{1}{n^{O(c)}}$. A similar argument shows that $Pr(X < (1-\epsilon)E(X)) \leq \frac{1}{n^{O(c)}}$.

We got that with probability at least $1 - \frac{1}{n^{O(c)}}$,

$$(1-\epsilon)q\left(1-\frac{1}{k}\right)^y \leq z \leq (1+\epsilon)q\left(1-\frac{1}{k}\right)^y \tag{6.3}$$

To simplify the presentation we assume that $\frac{\log z - \log q}{\log(1-\frac{1}{k})}$ is an integer. So by the definition of A_k, $y' = \frac{\log z - \log q}{\log(1-\frac{1}{k})}$ is the approximation that A_k returns.[2]
Dividing (6.3) by q we get that

$$(1-\epsilon)\left(1-\frac{1}{k}\right)^y \leq \frac{z}{q} \leq (1+\epsilon)\left(1-\frac{1}{k}\right)^y$$

Taking the logarithms, we obtain that

$$\log\left(1-\frac{1}{k}\right)^y + \log(1-\epsilon) \leq \log z - \log q \leq \log\left(1-\frac{1}{k}\right)^y + \log(1+\epsilon),$$

and dividing by $\log\left(1-\frac{1}{k}\right)^y$, we obtain that

$$1 + \frac{\log(1-\epsilon)}{\log\left(1-\frac{1}{k}\right)^y} \geq \frac{\log z - \log q}{y\log\left(1-\frac{1}{k}\right)} \geq 1 + \frac{\log(1+\epsilon)}{\log\left(1-\frac{1}{k}\right)^y},$$

and since the middle term is y'/y, we have that

$$1 + \frac{\log(1-\epsilon)}{\log\left(1-\frac{1}{k}\right)^y} \geq y'/y \geq 1 + \frac{\log(1+\epsilon)}{\log\left(1-\frac{1}{k}\right)^y}$$

Now, since $\frac{k}{2} \leq y \leq 3k$, we get that $\frac{1}{64} \leq \left(1-\frac{1}{k}\right)^y \leq e^{-1/2}$, and $-6 \leq y\log\left(1-\frac{1}{k}\right) \leq \frac{-\log(e)}{2}$. Therefore

$$1 + \frac{\log(1-\epsilon)}{-6} \geq y'/y \geq 1 + \frac{-2\log(1+\epsilon)}{\log e}.$$

[2] If $\frac{\log z - \log q}{\log(1-\frac{1}{k})}$ is not an integer then $y' = \lfloor \frac{\log z - \log q}{\log(1-\frac{1}{k})} \rfloor$.

Since $\log(1+\epsilon) \cong \epsilon$ for ϵ close to zero, we obtain that

$$1 + \epsilon \geq y'/y \geq 1 - 2\epsilon \ .$$

The theorem then follows if we apply Equations (6.1) and (6.2) with $\epsilon' = \epsilon/2$. ∎

We are now ready to describe the algorithm. To simplify the presentation, we assume that k is a power of 2. Our algorithm compares the pattern with the text, by comparing signatures of substrings of the pattern and substrings of the text. We perform these comparisons by using an algorithm A_j, for some $j \leq k$ which is a power of two, on substrings of length which is a power of two. We prepare all signatures required by this application of A_j in a preprocessing phase using convolutions as follows.

For any 2^j, $0 \leq j \leq \lfloor \log m \rfloor$, and for any 2^i, $0 \leq i \leq \log k$, we generate independently at random $q = \frac{c}{\epsilon^2} \log n$ strings $R_1, \cdots, R_q$, of sparsity 2^i and length 2^j. For each random string R_l of length 2^j, we compute the signature of every substring of T of length 2^j with R_l by a convolution of T and R_l. We also compute the signature of every substring of P of length 2^j with R_l by a convolution of P and R_l. We compute a total of $\frac{c}{\epsilon^2} \log n \log m \log k$ signatures in $O(\frac{n}{\epsilon^2} \log n \log^2 m \log k)$ time.

As in Section 6.4 for every text position j we find a position $s(j)$ in the pattern such that the approximate number of mismatches, g, between $P(1, s(j))$ and $T(j, s(j))$ satisfies $(1-\epsilon)k \leq g \leq (1+\epsilon)k$. We call $s(j)$ an *approximate k-mismatch*. If f is the true number of mismatches between $s(1, s(j))$ and $T(j, s(j))$ then $(1-\epsilon)f \leq g \leq (1+\epsilon)f$ with high probability. So $\frac{(1-\epsilon)k}{1+\epsilon} \leq f \leq \frac{(1+\epsilon)k}{1-\epsilon}$. It follows that $s(j)$ is an approximate k-mismatch for $\epsilon' = \frac{2\epsilon}{1-\epsilon}$.

To find the approximate k-mismatch we use the procedure $approx{-}mis(P', T')$ that given substring P' of the pattern and a substring T' of the text, both of the same length which is a power of two, either reports the approximate number of mismatches y' between P' and T', or $2k$ if the number of mismatches between P' and T' is larger than $2k$. The procedure $approx{-}mis(P', T')$ does a binary search on $A_i(P', T')$, for $i = 0, 2, 4, \cdots, k$. If $A_{\sqrt{k}}(P', T')$ reports that the number of mismatches is smaller than $\sqrt{k}/2$ we recurse for $i = 0, 2, 4, \cdots \sqrt{k}/2$. If $A_{\sqrt{k}}(P', T')$ reports that the number of mismatches is larger than $2\sqrt{k}$, we recurse for $i = 2\sqrt{k}, \cdots, k$. Otherwise $A_{\sqrt{k}}(P', T')$ reports an estimator y' of the number of mismatches between P' and T' which we return. If at the last step of the binary search we apply $A_k(P', T')$ and it reports that the number of mismatches is larger than $2k$ then we return $2k$. If the last step of this binary search applies $A_i(P', T')$ for some $i \neq k$ and A_i does not return an approximation to the number of mismatches between P' and T', or it applies $A_k(P', T')$ which returns that the number of mismatches is smaller than $k/2$ then at least one application of the A_i's returned an incorrect answer. By Lemma 6.5.1 this happens with polynomially small error and we ignore it.

We find the approximate k-mismatch, $s(j)$, by a binary search on the length of the pattern. We perform this binary search by the recursive procedure $mis{-}search(P', T', z)$ which finds an approximate z-mismatch for $z \leq k$, between P' and T' that are substrings

of P and $T(j,m)$, respectively, of length m' which is a power of two. To obtain $s(j)$ when m is a power of two, we perform $mis{-}search(P, T(j,m), k)$. (We discuss the case when m is not a power of two below.)

The procedure $mis{-}search(P', T', z)$ invokes $approx{-}mis(P'(1, m'/2), T'(j, m'/2))$, and let y' be the approximate number of mismatches that it obtains. If $y' > (1+\epsilon)z$ we continue with $mis{-}search(P'(1, m'/2), T'(j, m'/2), z)$. If $y' < (1-\epsilon)z$ we continue with $mis{-}search(P'(m'/2+1, m'/2), T'(j+m'/2, m'/2), z-y')$. If $(1-\epsilon)z \leq y' \leq (1+\epsilon)z$ we return position $m'/2$ of P'. This recursion may also end when applying $mis{-}search$ to a single character of P' and T'. If this happens and we have accumulated at least $(1-\epsilon)z$ mismatches up and including this character of P' then we return the position of this character in P'. Otherwise, if we have not accumulated $(1-\epsilon)z$ mismatches and the character which we compare is not the last character of P' some mistake had occurred in the approximation returned by a previous call to $approx{-}mis$. This can happen with polynomially small probability. Last if we have not accumulated $(1-\epsilon)z$ mismatches and the character which we compare is the last of P' then we conclude that P' matches T' with less than z mismatches.

When m is not a power of 2 then using the binary representation of m, we can write m as a sum of at most $\lceil \log m \rceil$ numbers $a_1, \cdots, a_d, d \leq \lceil \log m \rceil$, such that $a_i > a_j$, for $i < j$ and each a_j is a power of 2. Consider position j in the text, we first find the smallest $1 \leq r \leq d$, such that $P(1, \sum_{i=1}^{r} a_i)$ contains the approximate k-mismatch with $T(j, \sum_{i=1}^{r} a_i)$, or the approximate k-mismatch itself if $\sum_{i=1}^{r} a_i$ is an approximate k-mismatch for some r.

We first find the approximate number of mismatches y_1 between $P(1, a_1)$ and $T(j, a_1)$ by calling $approx{-}mis(P(1,a_1), T(j,a_1))$. If $y_1 \geq (1-\epsilon)k$ we stop: Either $y_1 \leq (1+\epsilon)k$ in which case $s(j) = a_1$ is the approximate k-mismatch or $y_1 \geq (1+\epsilon)k$ and $P(1, a_1)$ contains the k-mismatch. If $y_1 \leq (1-\epsilon)k$ we find the approximate number of mismatches, y_2, between $P(1+a_1, a_2)$ and $T(j+a_1, a_2)$. If $y_1+y_2 \geq (1-\epsilon)k$ we stop: Either $y_1+y_2 \leq (1+\epsilon)k$ in which case $s(j) = a_1 + a_2$ is the approximate k-mismatch or $y_1 + y_2 \geq (1+\epsilon)k$ and $P(1+a_1, a_2)$ contains the approximate k-mismatch. Otherwise, we continue to look for the approximate number of mismatches y_3 between $P(1 + \sum_{i=1}^{2} a_i, a_3)$ and $T(r + \sum_{i=1}^{2} a_i, a_3)$, and so on. This algorithm ends in one of the following ways.

1. We find r such that $\sum_{i=1}^{r} a_i$ is an approximate k-mismatch.

2. We get that $\sum_{i=1}^{d} y_i \leq (1-\epsilon)k$ and therefore P matches $T(j,m)$ with less than k-mismatches.

3. We find an index r such that $P(1 + \sum_{i=1}^{r-1} a_i, a_r)$ contain an approximate k-mismatch of P and $T(j,m)$.

In the latter case let $z = \sum_{i=1}^{r-1} y_i$ be the approximate number of mismatches between $P(1, \sum_{i=1}^{r-1} a_i)$ and $T(r, \sum_{i=1}^{r-1} a_i)$. (Note that $z = 0$ if $r = 1$.) We find the k-mismatch within

$P(1 + \sum_{i=1}^{r-1} a_i, a_r)$ by calling $mis{-}search(P(1 + \sum_{i=1}^{r-1} a_i, a_r), T(j + \sum_{i=1}^{r-1} a_i, a_r), k - z)$. (Remember that a_r is a power of 2).

It is easy to see that the running time of the search for all text positions is $O(\frac{1}{\epsilon^2} n \log n \log^2 m \log\log k)$. The total running time of the algorithm is $O(\frac{1}{\epsilon^2} n \log n \log^2 m \log k)$.

6.6 Concluding Remarks

We gave algorithms for the k-mismatch problem and for the approximate k-mismatch problem that run faster for large values of k.

It would be interesting to find algorithms with better running times both for the k-mismatch problem and for the approximate k-mismatch problem. Another interesting problem is to find other applications for these algorithms, besides the ones given in Chapter 7. It would also be interesting to use these techniques to get algorithms that find the position of the k-mismatch under the edit distance. The definition of the problem is as follows. For each text position j we would like to find the longest prefix P' of the pattern P, such that there exists a prefix of $T[j \cdots m]$ that can be converted into P' using at most k edit operations (character insertion, character deletion, and a substitution of a character by another character).

Chapter 7

Approximate tandem repeats and general repeats.

7.1 Approximate Tandem Repeats

In this section we show how to use the algorithm for finding the position of the k-mismatch, to solve a variant of the approximate tandem repeats problem.

The *tandem repeats* problem is defined as follows. Given a string S of length n, find all occurrences of substrings of the form uu. We call uu a tandem repeat of S. Main and Lorentz [71] gave an algorithm that solves the problem in $O(n \log n + z)$ time, where z is the number of tandem repeats in S.

The *k-mismatch tandem repeats* problem is defined as follows. Find all tandem repeats u_1u_2 such that hamming distance between u_1 and u_2 is at most k. The best known algorithms for this problem is the algorithm of Landau, Schmidt and Sokol [64] that runs in $O(nk \log(n/k) + z)$ time and the algorithm of Kolpakov and Kucherov [57] that runs in $O(nk \log k + z)$ time.

We suggest a relaxation of the k-mismatch tandem repeat problem that has better running time for large values of k. Given parameters k and ϵ, the approximate tandem repeats algorithm will find all tandem repeats u_1u_2 such that the hamming distance between u_1 and u_2 is at most k. The algorithm may also find tandem repeats u_1u_2 such that the hamming distance between u_1 and u_2 is at most $(1+\epsilon)k$. Thus we know that all tandem repeats found by the algorithm have at most $(1+\epsilon)k$ mismatches.

Our algorithm can use either the algorithm of Section 6.2 and Section 6.3 or the algorithm of Section 6.4. In the first case its running time is $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z)$, in the second case its running time is $O(\frac{1}{\epsilon^3}n\sqrt{k} \log^3 n \log(n/k) + z)$. We can also use the algorithm of Section 6.5, and get an algorithm that finds all approximate tandem repeats with high probability, whose running time is $O(\frac{1}{\epsilon^3}n \log^3 n \log k \log(n/k) + z)$. Here z is the number of approximate tandem repeats which the algorithm reports.

Notice that all substrings u_1u_2 such that $|u_1| = |u_2| \leq k$, are approximate tandem repeats. So we are interested in finding only tandem repeats of length greater than $2k$.

We first describe the algorithm for exact tandem repeats. Then we describe the algorithm of [64] for the k-mismatch tandem repeats that runs in $O(nk \log(n/k) + z)$. Finally we show how to change this algorithm to get our algorithm. Let $S[i \cdots j]$ be the substring of S that starts at position i and ends at position j, and recall that $\overleftarrow{S[i \cdots j]}$ is the string obtained by reversing $S[i \cdots j]$. Let $S[i]$ be the character at position i.

We now describe the exact algorithm of Main and Lorentz [71]. Let $h = \lfloor n/2 \rfloor$. Let $u = S[1 \cdots h]$ be the first half of S, and let $v = S[h+1 \cdots n]$ be the second half of S. The algorithm finds all tandem repeats that contain $S[h]$ and $S[h+1]$. That is repeats that are not fully contained in u and are not fully contained in v. Then we apply it recursively to u to find all tandem repeats contained in u and we apply it recursively to v to find all tandem repeats contained in v.

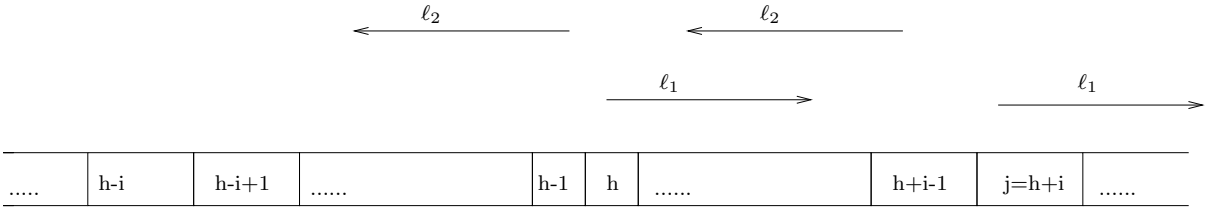

Figure 7.1: Finding left repeats. In this example since both ℓ_1 and ℓ_2 are smaller than i, the tandem repeats found start at positions $h - \ell_2, \cdots, h + \ell_1 - i$.

The repeats that contain $S[h]$ and $S[h+1]$ are classified into *left repeats* and *right repeats*. *Left repeats* are all tandem repeats zz where the first copy of z contains $S[h]$. *Right repeats* are all tandem repeats zz where the second copy of z contains $S[h]$. We describe how to find all left repeats. We find right repeats similarly. We build suffix trees that support LCA queries in $O(1)$ time for S and for $\overleftarrow{S}$. The algorithm for finding left repeats in S has $n/2$ iterations. In the i-th iteration, we find all left repeats of length $2i$ as follows (See Figure 7.1).

1. Let $j = h + i$.
2. Find the longest common prefix of $S[h \cdots n]$ and of $S[j \cdots n]$. Let ℓ_1 be the length of this prefix.
3. Find the longest common prefix of $\overleftarrow{S[1 \cdots h-1]}$ and of $\overleftarrow{S[1 \cdots j-1]}$. Let ℓ_2 be the length of this prefix.
4. If $\ell_1 + \ell_2 \geq i$ there is at least one tandem repeat of length $2i$. All left repeats of length $2i$, begin at positions $\max(h - \ell_2, h - i + 1), \cdots, \min(h + \ell_1 - i, h)$.

Using the suffix trees we can find each longest common prefix in $O(1)$ time. Therefore, we can find an implicit representation of all left repeats of length $2i$ in $O(1)$ time. Similarly, we can find all right repeats of length $2i$ in $O(1)$ time. We have to repeat this for every i so total time it takes to find all repeats that span $S[h]$ or $S[h+1]$ for $h = \lfloor n/2 \rfloor$ is $O(n)$, and the total running time of the algorithm is $O(n \log n + z)$.

The algorithm of [64] for finding k-mismatch tandem repeats is an extension of the algorithm of Main and Lorentz [71]. Here we stop the recursion when the length of the string is at most $2k$, and in each iteration we compute only repeats of length greater than $2k$. Given $h = \lfloor n/2 \rfloor$ and $i > k$ the algorithm for finding all k-mismatch left repeats of size $2i$ is as follows.

1. Let $j = h + i$.

2. We find the positions of the first $k+1$ mismatches of $S[h \cdots n]$ and $S[j \cdots n]$ by performing $k+1$ successive LCA queries on the suffix tree of S. Let ℓ_1 be the length of the longest prefix of $S[h \cdots n]$ that has at most k mismatches with $S[j \cdots n]$.

3. Similarly, we find the positions of the first $k+1$ mismatches of $\overleftarrow{S[1 \cdots h-1]}$ and $\overleftarrow{S[1 \cdots j-1]}$ by performing $k+1$ successive LCA queries on a suffix tree of $\overleftarrow{S}$. Let ℓ_2 be the length of the longest prefix of $\overleftarrow{S[1 \cdots h-1]}$ that has at most k mismatches with $\overleftarrow{S[1 \cdots j-1]}$.

4. If $\ell_1 + \ell_2 \geq i$, the k-mismatch tandem repeats will be those at positions $\max(h-\ell_2, h-i+1) \cdots \min(h+\ell_1-i, h)$ that have at most k mismatches. Notice that the center points of all of these tandem repeats of length $2i$ are in the segment between position h and position $h+i$. Suppose we have a k-mismatch tandem repeat whose center is right after position f. Let f_1 be the number of mismatches between $S[h \cdots f]$ and $S[h+i \cdots f+i]$. Let f_2 be the number of mismatches between $\overleftarrow{S[f+1 \cdots h+i-1]}$ and $\overleftarrow{S[f-i+1 \cdots h-1]}$. Then $f_1 + f_2 \leq k$. Therefore for each position $f \in [h \cdots h+i]$ we need to find the number of mismatches between $S[h \cdots f]$ and $S[h+i \cdots f+i]$ computed in Item (2) and the number of mismatches between $\overleftarrow{S[f+1 \cdots h+i-1]}$ and $\overleftarrow{S[f-i+1 \cdots h-1]}$ computed in Item (3). This leads to the following algorithm. Merge the sorted list of Item (2) containing the positions of the mismatches that are in $[h \cdots h+i]$ with the sorted list of Item (3) containing the positions of the mismatches that are in $[h \cdots h+i]$. Consider a segment between two adjacent positions in the merged list. For every position f in this segment the number of mismatches of $S[h \cdots f]$ with $S[h+i \cdots f+i]$ is the same. Similarly, for every position f in that segment the number of mismatches of $\overleftarrow{S[f+1 \cdots h+i-1]}$ with $\overleftarrow{S[f-i+1 \cdots h-1]}$ is the same. So either all these positions correspond to an approximate tandem repeat of length $2i$ or none does. Thus we can determine all tandem repeats by looking at these $O(k)$ positions. (See [64, 41] for more details).

Steps (2) and (3) take $O(k)$ time, given suffix trees that support constant time LCA queries. The total time it takes to find all left and right k-mismatch tandem repeats spanning $S[h]$ and $S[h+1]$ for $h = \lfloor n/2 \rfloor$ is $O(nk)$, and the total running time of the algorithm is $O(nk \log(n/k) + z)$.

We are now ready to describe our approximate tandem repeats algorithm with parameters ϵ and k. We use the algorithm of Section 6.3 but with minor modifications and with a different scaling of ϵ we can use the algorithms of Sections 6.4, and 6.5 instead. The algorithm has the same steps as the algorithm of [64]. The only difference is in the way left (and right) tandem repeats are computed. Let $h = \lfloor n/2 \rfloor$. Let $P_h = S[h \cdots n]$ and $T_h = S[h \cdots n]$.[1] Let $\overleftarrow{P}_{h-1} = \overleftarrow{S[1 \cdots h-1]}$ and $\overleftarrow{T} = \overleftarrow{S[1 \cdots n]}$. For convenience we assume that ϵk and $\frac{1}{\epsilon}$ are integers. We compute the left repeats as follows.

1. Compute the position of the $r+1$-mismatch between the text T_h and the pattern P_h, for $r = \epsilon k, 2\epsilon k, \cdots, k - \epsilon k, k$. We do that by running the algorithm of Section 6.3 once for every $r = \epsilon k, 2\epsilon k, \cdots, k - \epsilon k, k$. Let $B_r[i]$, $i \geq h$, contain the position of the $r+1$-mismatch between $S[i \cdots n]$ and $S[h \cdots n]$.

2. Compute the position of the $r+1$-mismatch between the text $\overleftarrow{T}$ and the pattern $\overleftarrow{P}_{h-1}$, for $r = \epsilon k, 2\epsilon k, \cdots, k - \epsilon k, k$ with the algorithm of Section 6.3. Let $\overleftarrow{B}_r[i]$, $i \geq h$, contain the position of the $r+1$-mismatch between $\overleftarrow{S[1 \cdots i]}$ and $\overleftarrow{S[1 \cdots h-1]}$.

3. For each $i > k$ we find all approximate tandem repeats of length $2i$ whose first half contains h as follows. As in Item (4) of the previous algorithm we merge the two sequences $B_{\epsilon k}[h+i], \cdots, B_k[h+i]$ and $\overleftarrow{B}_{\epsilon k}[h+i-1], \cdots, \overleftarrow{B}_{k+1}[h+i-1]$. Let f be an arbitrary position between two consecutive positions of the merged list. There are integers j_1 and j_2 such that the number of mismatches between $S[h \cdots f]$ and $S[h+i \cdots f+i]$ is smaller than $j_1 \epsilon k + 1$, and the number of mismatches between $\overleftarrow{S[f+1 \cdots h+i-1]}$ and $\overleftarrow{S[f-i+1 \cdots h-1]}$ is smaller than $j_2 \epsilon k + 1$. If $j_1 + j_2 \leq \frac{1}{\epsilon} + 1$ then we report f as a center of an approximate tandem repeat.

It is easy to see that this algorithm produces all tandem repeats with at most k mismatches. The algorithm may also report tandem reports with at most $(1+\epsilon)k$-mismatches.

Items (1) and (2) that take $O(\frac{1}{\epsilon} n k^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k)$ time dominate the running time of each recursive call. Therefore the total time is $O(\frac{1}{\epsilon} n k^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z)$.

7.2 Approximating k-mismatch globally defined repeats

For two strings S, T we define $h(S,T)$ to be the Hamming distance between S and T. Kolpakov and Kucherov [57] defined a string $R[1 \cdots n]$ to be *a k-mismatch globally-defined*

[1] we use the version of the algorithms that can match suffixes of the text that are shorter than the pattern. See the end of Section 5.1.

repeat (gd-repeat) of period length $p \leq n/2$, if $h(R[1 \cdots n-p], R[p+1 \cdots n]) \leq k$. String R is a maximal gd-repeat of period length p if we cannot extend it to the left or to the right and get a longer k-mismatch gd-repeat of period length p. Kolpakov and Kucherov gave an algorithm for finding all maximal k-mismatch gd-repeats in a string of length n that runs in $O(nk \log k + z)$ time, where z is the number of these repeats. Note that if the same string is a repeat for more than one period length then we count it with multiplicity equals to the number of lengths of which it is a period. For example the string $c(ab)^{2r}d$ has the substring $(ab)^{2r}$ as a maximal gd-repeat with period length $2i$ for every $1 \leq i \leq r$.

We define the *approximate k-mismatch gd-repeat problem*. Given a string S and $\epsilon > 0$, we want to find a set of substrings of S which are maximal $(1+\epsilon)k$-mismatch gd-repeats, such that each maximal k-mismatch gd-repeat in S of period length p is contained in one of these repeats that also has period length p.

7.2.1 The algorithm of Kolpakov and Kucherov for finding maximal k-mismatch globally defined repeats

We first describe the algorithm of [57] for finding all maximal k-mismatch gd-repeats in S. Then we show how to modify it so that we can solve the approximate k-mismatch gd-repeat problem more efficiently.

Let $r = S[i \cdots j]$ be a k-mismatch gd-repeat with period length p in a string S. We define the substring $S[j-p+1 \cdots j]$ to be the *right root* of r. The *Lempel-Ziv factorization with copy overlap* of a string S is a partition of the string S into *factors* $f_1 \cdots f_u$ defined as follows: $f_1 = S[1]$, and f_i, for $i \geq 2$, is the shortest substring of S that starts immediately after $f_1 \cdots f_{i-1}$ which does not occur in $f_1 \cdots f_i$ other than as a suffix of $f_1 \cdots f_i$. See Figure 7.2. Kolpakov and Kucherov proved ([57], Lemma 3.3) the following.

Lemma 7.2.1 *The right root of a k-mismatch globally-defined repeat in S cannot contain as a substring $k+1$ consecutive Lempel-Ziv factors of S.*

Proof: Let p be the period of the k-mismatch. Each factor $f = S[i \cdots j]$ contained in the right root has a mismatch with the substring $S[i-p \cdots j-p]$. Otherwise, there is an earlier copy of that factor at position $i-p$, which contradicts the definition of a factor. ∎

We compute in linear time the Lempel-Ziv factorization $S = f_1 \cdots f_r$. This is easy to do using the suffix tree of S, see [41, 76]. Then we partition the string S into consecutive blocks $S = B_1 \cdots B_{r'}$. Each block (except possibly the last block), contains $k+2$ consecutive Lempel-Ziv factors. We first find gd-repeats spanning more than a single block, then those gd-repeats contained in a block but span more than one factor of the block, and last gd-repeats contained in a factor. For the first two steps we use a procedure to find gd-repeats containing a particular character.

7.2.1.1 k-mismatch gd-repeats that contain a specific character

In a way similar to the procedure of Landau *et al.* [64] k-mismatch tandem repeats, we can find k-mismatch gd-repeats containing a particular character $S[\ell]$. We assume that we have a suffix tree for S and a suffix tree for $\overleftarrow{S}$, both supporting constant time LCA queries. We show how the algorithm finds all gd-repeats of period length p whose right root starts to the right of (or at) $S[\ell]$. repeats whose right root starts to the left of $S[\ell]$ can be computed similarly.

The algorithm computes the positions of the first $k+1$ mismatches of $S[\ell \cdots |S|]$ and $S[\ell+p \cdots |S|]$ by performing $k+1$ successive LCA queries on the suffix tree of S similar to the kangaroo method described in Section 6.1. Similarly, we find the positions of the first $k+1$ mismatches of $\overleftarrow{S[1 \cdots \ell-1]}$ and $\overleftarrow{S[1 \cdots \ell+p-1]}$ by performing $k+1$ successive LCA queries on a suffix tree of $\overleftarrow{S}$. Using these positions we compute the following functions in $O(k)$ time.

$$\begin{aligned} LP_p(r) &= \max\{j \mid h(S(\ell, j), S(\ell+p, j)) < r\} && 1 \le r \le k+1 \\ LS_p(r) &= \max\{j \mid h(\overleftarrow{S}(\ell-1, j), \overleftarrow{S}(\ell+p-1, j)) < r\} && 1 \le r \le k+1 \end{aligned}$$

Since we are interested in maximal gd-repeats, each gd-repeat (unless we are at the end of the string) contains exactly k mismatches. Therefore the gd-repeats of period length p in S whose right root starts to the right of (or at) $S[\ell]$ correspond to indices $i \le k+1$ such that the following holds.

$$LP_p(i) + LS_p(k+2-i) \ge p\ . \tag{7.1}$$

In this case, the gd-repeat is the substring that starts at position $\ell - LS_p(k+2-i)$ and ends at position $\ell + p + LP_p(i) - 1$. It is easy to find all these repeats in $O(k)$ time by checking Condition (7.1) for each $i \le k+1$.

7.2.1.2 Finding all k-mismatch gd-repeats

As we mentioned the algorithm consists of three steps. In Step (1) we find gd-repeats that span a block boundary. In fact for technical reasons that would be clear when we describe Step (3), we find all k-mismatch gd-repeats spanning one of the positions $\text{head}(B_i) - 1$, $\text{head}(B_i)$, or $\text{head}(B_i) + 1$ for every block B_i, where $\text{head}(B_i)$ is the last character in B_i. In Step (2) we find gd-repeats inside each block B_i that span a factor boundary and were not found in Step (1). For the same technical reasons we in fact compute all k-mismatch gd-repeats inside a block B not found in Step (1), that span at least one of the positions $\text{head}(f) - 1$, $\text{head}(f)$, or $\text{head}(f) + 1$ where f is a factor in B, and $\text{head}(f)$ is the last character in f. In Step (3) we find gd-repeats that are contained inside a factor and were not found in Steps (1) and (2). All the gd-repeats that we find are maximal, even though we may not repeat this explicitly each time.

Step 1. For every $i < r'$, we compute all k-mismatch gd-repeats that contain one of the characters at positions $\text{head}(B_i) - 1$, $\text{head}(B_i)$, and $\text{head}(B_i) + 1$, and do not contain the character at position $\text{head}(B_{i+1})-1$. We also compute all k-mismatch gd-repeats containing either $S[head(B_{r'}) - 1]$ or $S[head(B_{r'})]$.

We find these repeats using the procedure of Section 7.2.1.1 to compute all k-mismatch gd-repeats of a specific period length p that contain a specific character $S[\ell]$, we apply it such that $S[\ell]$ is one of the characters at positions $\text{head}(B_i)-1$, and $\text{head}(B_i)+1$ for $i < r'$, and we also apply it with $S[\text{head}(B_{r'})-1]$.[2] Finally, we eliminate any duplicates and any of the gd-repeats containing $\text{head}(B_{i+1})-1$. The following lemma specifies the period lengths which we have to consider.

Lemma 7.2.2 *A gd-repeat containing head$(B_i)-1$, head(B_i), or head$(B_i)+1$, for $i < r'$ but not containing* $\text{head}(B_{i+1}) - 1$ *is of period length at most $|B_iB_{i+1}|$. A gd-repeat containing head$(B_{r'}) - 1$ or head$(B_{r'})$ is of period length at most $|B_{r'-1}B_{r'}|$.*

Proof: If the period of such a gd-repeat is of length $\geq |B_iB_{i+1}|$ then its right root must start before the first character of B_i. So its right root contains at least $k + 1$ of the $k + 2$ factors of B_i. This contradicts Lemma 7.2.1. The proof of the second part of the lemma is similar. ∎

Since $S[\text{head}(B_i)+1]$ is the first character of B_{i+1} for $i < r'$, we in fact compute also the gd-repeats containing the first character of each block B_i, for $i > 1$. For the special case of the first block, we also compute for each period length the (at most one) k-mismatch gd-repeat that contains $S[1]$ and does not contain $S[\text{head}(B_1) - 1]$. We do that using the same procedure from Section 7.2.1.1 as described above. Clearly such a gd-repeat is of period length at most $|B_1|/2$.

Since by Lemma 7.2.2 we apply the procedure of Section 7.2.1.1 to $S[\text{head}(B_i) - 1]$ and $S[\text{head}(B_i) + 1]$ for each period length not larger than $|B_iB_{i+1}|$, to $S[\text{head}(B_{r'}) - 1]$ for period length at most $|B_{r'-1}B_{r'}|$, and to $S[1]$ for period length at most $|B_1|/2$ then this step takes $O(nk)$ time.

Step 2. Let $f_i \cdots f_{i+r}$ be the factors of some block B (where $r = k + 2$ initially, but we are describing a recursive procedure that would work the same when applied to smaller blocks). At this step we compute every k-mismatch gd-repeat which satisfies the following conditions: 1) It is completely inside B, 2) It contains at least one of the characters at positions $\text{head}(f_j) - 1$, $\text{head}(f_j)$, and $\text{head}(f_j) + 1$ for some $i \leq j < i + r$, 3) It does not contain $S[\text{head}(B) - 1]$, and 4) It does not contain the first character of f_i.[3] We do this recursively as follows. We divide B into two subblocks $B' = f_i \cdots f_{i+\lfloor r/2 \rfloor}$, and $B'' = f_{i+\lfloor r/2 \rfloor+1} \cdots f_{i+r}$. Find all k-mismatch gd-repeats that contain at least one of the

[2]Note that a gd-repeat containing $S[\text{head}(B_i)]$ either contains $S[\text{head}(B_i) - 1]$ or $S[\text{head}(B_i) + 1]$, and a gd-repeat containing $S[\text{head}(B_{r'})]$ must contain $S[\text{head}(B_{r'}) - 1]$.

[3]Notice that we computed gd-repeats containing the first character and the last two characters of each block in Step (1).

characters at positions $\text{head}(f_{i+\lfloor r/2 \rfloor})-1$, $\text{head}(f_{i+\lfloor r/2 \rfloor})$, and $\text{head}(f_{i+\lfloor r/2 \rfloor})+1$ and discard those among them that span position $\text{head}(B)-1$, and those that contain the first character of f_i. Then we process B' and B'' recursively. The algorithm has $\log k$ levels of recursion.

In each level we use the procedure from Section 7.2.1.1 (with periods of length bounded by the current size of the block) to find gd-repeats that contain a particular character and remove duplicates. We repeat this for each block in S. It is easy to see that the overall running time is $O(nk \log k)$.

Step 3. Last, for each factor $f_h = S[i \cdots j]$, we want to find all maximal gd-repeats contained in f_h. Recall that $i = \text{head}(f_{h-1}) + 1$, and $j = \text{head}(f_h)$, and we have already found repeats containing characters $S[i]$, $S[j-1]$, and $S[j]$ in Steps (2) and (3). So we only have to find all maximal gd-repeats that are contained in $S' = S[i+1 \cdots j-2]$.

Since $S[i \cdots j-1]$ is properly contained in f_h it has another copy (possibly overlapping), in $S[1 \cdots j-2]$. In the preprocessing phase after computing the factors, we save for each factor f_h a pointer to its previous copy that excludes $\text{head}(f_h)$. We compute all these pointers in linear time using the suffix tree of S.

Let $S[i-\rho \cdots j-\rho-1]$ be the previous copy associated with f_h, and $S'_c = S[i-\rho+1 \cdots j-\rho-2]$. We use the following lemma to find the maximal gd-repeats in f_h after we found the maximal gd-repeats in f_j for $j < h$.

Lemma 7.2.3 *The string $S[a \cdots b]$ is a maximal gd-repeat in S'_c if and only if $S[a+\rho \cdots b+\rho]$ is a maximal gd-repeat in S'.*

Proof: Let $S[a \cdots b]$ be a maximal gd-repeat in S'_c. From the maximality of S'_c follows that $S[a-1] \neq S[a-1+p]$, and that $S[b+1-p] \neq S[b+1]$. Since $S[i \cdots j-1] = S[i-\rho \cdots j-\rho-1]$ we get that $S[a \cdots b] = S[a+\rho \cdots b+\rho]$ and $S[a-1+\rho] \neq S[a-1+\rho+p]$ and $S[b+1+\rho-p] \neq S[b+1+\rho]$. So $S[a+\rho \cdots b+\rho]$ is a maximal gd-repeat in S'. We prove the other direction similarly. ∎

Lemma 7.2.3 explains why we handled gd-repeats spanning either the first character, or any of the last two characters of a block separately. Any other maximal repeat contained in the factor can be copied from the previous copy of the prefix of the factor.

We construct an array A such that $A[j]$ is a linked list of all gd-repeats ending at position j, found so far, sorted in descending order of their start positions as follows. We bucket sort all maximal k-mismatch gd-repeats found in Steps (1) and (2) in descending order of their starting positions. Then we scan the sorted list and for each gd-repeat $S[i \cdots j]$ we add i at the end of the linked list associated with $A[j]$.

Now, to find all maximal gd-repeats contained in f_h, we scan A from $A[i-\rho+1]$ to $A[j-\rho-2]$. Let $A[q]$ be the next cell that we scan. For each repeat in $A[q]$, we report the corresponding gd-repeats at position $q+\rho$ of f_h until we reach a repeat whose starting position is smaller than $i-\rho+1$. We also update the list in $A[q+\rho]$ with the new gd-repeats that we find, so it may be used for the following factors or for the current factor if the copy of f_h overlaps f_h. Let $i' \geq i-\rho+1$ be the starting position of the first gd-repeat in $A[q]$

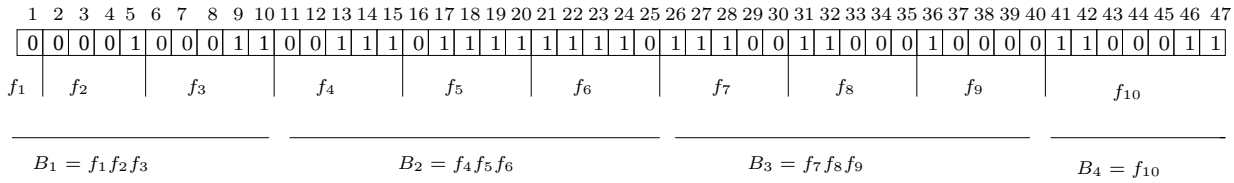

Figure 7.2: The string of the example of the algorithm for finding gd-repeats. We show the partition of this string into factors and blocks.

that has a corresponding repeat that starts at position $i' + \rho$ and ends at position $q + \rho$. Since all previous gd-repeats found at Step (1) and at Step (2), which end at position $q + \rho$ are not contained in f_h, position $i' + \rho$ is larger than all starting positions of gd-repeats ending at position $q + \rho$ that we had found so far. So we add $i' + \rho$ as the first element of the list associated with $A[q + \rho]$. All subsequent gd-repeats ending at position $q + \rho$, are inserted in the same order as they appear in $A[q]$ preceding all other repeats (that were found at Step (1) and at Step (2)), in $A[q + \rho]$.

Correctness follows by induction using the correspondence between repeats in $S' = S[i + 1 \cdots j - 2]$ and its previous copy $S'_c = S[i - \rho + 1 \cdots j - \rho - 2]$ established in Lemma 7.2.3. It is easy to verify that the total running time of this step is $O(n + z)$, where z is the number of gd-repeats.

An example of the algorithm for finding gd-repeats In this example we assume that $k = 1$, that is we look for gd-repeats with one mismatch. We also focus on gd-repeats of period length 2, but the algorithm finds gd-repeats of all period lengths simultaneously through the same steps. Figure 7.2 shows the string S and its partition into factors with copy overlap, and the partition of the factors into blocks.

In Step (1) the algorithm finds all gd-repeats that span a block boundary. We find $S[1 \cdots 6] = 000010$ that spans the first character of S. We find $S[6 \cdots 9] = 0001$ that spans head$(B_1) - 1$. Then we find $S[15 \cdots 24] = 1011111111$, $S[17 \cdots 26] = 1111111101$, and $S[24 \cdots 28] = 10111$ that span head$(B_2) - 1$, and $S[26 \cdots 29] = 1110$ that span head$(B_2) + 1$. We also find $S[35 \cdots 40] = 010000$, and $S[37 \cdots 41] = 00001$ that span head(B_3), and finally we find $S[43 \cdots 46] = 0001$ that spans head(B_4).

In Step (2) we find $S[4 \cdots 8] = 01000$, $S[12 \cdots 15] = 0111$, $S[13 \cdots 17] = 11101$, $S[32 \cdots 35] = 1000$, and $S[33 \cdots 37] = 00010$ that span a factor boundary.

In Step (3) we find $S[42 \cdots 45] = 1000$ that is contained in the factor f_{10}. We find this repeat by copying $S[32 \cdots 35] = 1000$ from the previous copy of the prefix of f_{10} at $S[31 \cdots 36]$.

7.2.2 Finding approximate k-mismatch gd-repeats

To solve the approximate k-mismatch gd-repeats problem, we use the algorithm of [57] that we just described with some modifications. The main modification is that we use a different procedure to compute a set of maximal $(1+\epsilon)k$-mismatch gd-repeats of period length p in a string W that contain the character $W[\ell]$. (Here W would be an appropriate substring of S). This modification is similar to the modification done in the algorithm for finding all approximate k-mismatch tandem repeats in Chapter 7.1.

We now describe this procedure, and then in Section 7.2.2.2, we describe other changes we made in the algorithm of [57]. In the rest of this section, unless specifically stated, all $(1+\epsilon)k$-mismatch gd-repeats we mention are maximal. Also, when we talk about a $(1+\epsilon)k$-mismatch repeat containing a k-mismatch repeat they would always be of the same period length, even when we do not mention this explicitly.

7.2.2.1 Finding $(1+\epsilon)k$-mismatch gd-repeats that contain $W[\ell]$

We compute a set of $(1+\epsilon)k$-mismatch gd-repeats each of which is maximal within W and contains $W[\ell]$. These repeats satisfy the additional property that if there is a $(1+\epsilon)k$-mismatch gd-repeat that spans $W[\ell]$ and contains a maximal k-mismatch gd-repeat u, then u is a substring of a repeat in the set. Specifically, the strings that we find contain the following maximal k-mismatch gd-repeats as substrings, (if such k-mismatch gd-repeats actually exist).

1. Every maximal k-mismatch gd-repeat of period length p that contains $W[\ell]$ is a substring of a repeat in the set.

2. Let $W[i \cdots j]$ be a maximal k-mismatch gd-repeat, such that $j < \ell$ and $W[i \cdots \ell]$ is a (not necessarily maximal) $(1+\epsilon)k$-mismatch gd-repeat, then $W[i \cdots j]$ is a substring of a repeat in the set.

3. Let $W[i \cdots j]$ be a maximal k-mismatch gd-repeat, such that $i > \ell$ and $W[\ell \cdots j]$ is a (not necessarily maximal) $(1+\epsilon)k$-mismatch gd-repeat, then $W[i \cdots j]$ is a substring of a repeat in the set.

We now describe how to find approximate k-mismatch gd-repeats whose right root starts to the right of (or at) $W[\ell]$. Gd-repeats whose right root starts to the left of $W[\ell]$ are computed similarly.

Let $P_\ell = W[\ell \cdots |W|]$, and let $T_\ell = W[\ell \cdots |W|]$. Let $\overleftarrow{P}_{\ell-1} = \overleftarrow{W[1 \cdots \ell-1]}$ and let $\overleftarrow{T} = \overleftarrow{W[1 \cdots |W|]}$. We compute the position of the $(i+1)$-mismatch between every suffix of T_ℓ and the pattern P_ℓ, for $i = 0, \epsilon k, 2\epsilon k, \cdots k - \epsilon k, k$, and $(1+\epsilon)k$, by running the k-mismatch algorithm of Chapter 6.2 once for every such i, (we assume that ϵk and $\frac{1}{\epsilon}$ are integers). Recall that the output of this computation for each is an array X, such that $X[r]$, $r \geq \ell$, contains the position of the $i+1$-mismatch between $W[r \cdots |W|]$ and $W[\ell \cdots |W|]$.

Similarly, we use the exact k-mismatch algorithm of Chapter 6.2 to compute the position of the $(i+1)$-mismatch between the text $\overleftarrow{T}$ and the pattern $\overleftarrow{P}_{\ell-1}$, for $i = 0, \epsilon k, 2\epsilon k, \cdots k - \epsilon k, k$, and $(1+\epsilon)k$. The result of this computation for such an i is an array $\overleftarrow{X}$, such that $\overleftarrow{X}[r]$ for $r \geq 1$, contains the position of the $i+1$-mismatch between $\overleftarrow{P}_{\ell-1} = \overleftarrow{W[1 \cdots \ell - 1]}$ and $\overleftarrow{W[1 \cdots |W| - r + 1]}$.

These computations are done once in $O\left(\frac{1}{\epsilon}|W|k^{\frac{2}{3}} \log^{\frac{1}{3}} |W| \log k\right)$ time, and are used to find gd-repeats of all period lengths.

To find gd-repeats of period length p we use the positions of these mismatches to compute the following functions for every $0 \leq r \leq \frac{1}{\epsilon} + 1$.

$$\begin{aligned} LP_p(r) &= \max\{j \mid h(W(\ell, j), W(\ell + p, j)) < r\epsilon k + 1\} \text{ and} \\ LS_p(r) &= \max\{j \mid h(\overleftarrow{W}(\ell - 1, j), \overleftarrow{W}(\ell + p - 1, j)) < r\epsilon k + 1\} \ . \end{aligned}$$

These functions can be computed in a straightforward way using the arrays X and $\overleftarrow{X}$.

For $r = 0$ if $h(W(\ell, 1), W(\ell + p, 1)) = 1$ then the set on the right hand side of the first equality is empty so we define $LP_p(0) = 0$, and similarly if $h(W(\ell-1, 1), W(\ell+p-1, 1)) = 1$ we define $LS_p(0) = 0$. The gd-repeats of period length p whose right root starts to the right of (or at) $W[\ell]$ that we report correspond to indices $0 \leq i \leq 1/\epsilon + 1$, such that

$$LP_p(i) + LS_p\left(\frac{1}{\epsilon} + 1 - i\right) \geq p \ .$$

In this case, the gd-repeat is the substring that starts at position $\ell - LS_p(\frac{1}{\epsilon} + 1 - i)$ and ends at position $\ell + p + LP_p(i) - 1$.[4]

This computation for a specific period length p takes $O(1/\epsilon)$ time (rather than $O(k)$ for the exact computation). Since the total number of possible period lengths is no more than $|W|$ we obtain that using the exact algorithm of Chapter 6 for the k-mismatch problem, the overall running time for all period lengths together is $O\left(\frac{1}{\epsilon}|W|k^{\frac{2}{3}} \log^{\frac{1}{3}} |W| \log k\right)$.

The next lemma shows that the set of repeats which we find contains all maximal k-mismatch gd-repeats that we claimed above.

Lemma 7.2.4 *Let $u = W[i \cdots j]$ be a maximal k-mismatch gd-repeat of period length p for which one of the following holds.*

1. *The string u contains $W[\ell]$.*
2. *$j < \ell$ and $W[i \cdots \ell]$ is a (not necessarily maximal) $(1+\epsilon)k$-mismatch gd-repeat.*
3. *$i > \ell$ and $W[\ell \cdots j]$ is a (not necessarily maximal) $(1+\epsilon)k$-mismatch gd-repeat.*

[4]Note that if either $\ell - LS_p(\frac{1}{\epsilon} + 1 - i) = 1$ or $\ell + p + LP_p(i) - 1 = |W|$, the repeat may not be maximal and we eliminate it if it is contained in a larger repeat of period length p which we find.

Then u is a substring of a $(1+\epsilon)k$-mismatch gd-repeat u' which we find.

Proof: Assume that u contains $W[\ell]$, and the right root of u starts to the right of (or at) $W[\ell]$, that is $j \geq \ell + p - 1$, (the case where the right root of u starts to the left of $W[\ell]$ is similar). Let q_1 be the number of mismatches between $W[\ell \cdots j-p]$ to $W[\ell+p \cdots j]$, and let q_2 be the number of mismatches between $W[i \cdots \ell-1]$ to $W[i+p \cdots \ell+p-1]$. Clearly, $q_1 + q_2 = k$. Let j' be the minimal number such that $q_1 \leq j'\epsilon k$. Then, $j'\epsilon k - q_1 < \epsilon k$, and $q_2 = k - q_1 < ((\frac{1}{\epsilon} + 1 - j')\epsilon k)$. Let $u' = W[\ell - LS_p(\frac{1}{\epsilon} + 1 - j') \cdots \ell + p + LP_p(j') - 1]$, then u' contains u, and since $|u| \geq 2p$, we have that $LS_p(\frac{1}{\epsilon} + 1 - j') + LP_p(j') \geq p$. Thus u' belongs to the set of $(1+\epsilon)k$-mismatch gd-repeats that we find, and u' contains u.

Assume that $i > \ell$ and $W[\ell \cdots j]$ is a (not necessarily maximal) $(1+\epsilon)k$-mismatch gd-repeat, (the case where $j < \ell$ and $W[i \cdots \ell]$ is a (not necessarily maximal) $(1+\epsilon)k$-mismatch gd-repeat is similar). Let $u' = W[\ell - LS_p(0) \cdots \ell + p + LP_p(\frac{1}{\epsilon} + 1) - 1]$. Then u' contains u, and $|u'| \geq 2p$. Thus u' belongs to the set of $(1+\epsilon)k$-mismatch gd-repeats that we find. ■

The reason for which we have to cover substrings u which satisfy Items 2 and 3 of Lemma 7.2.4 follows from the fact that we will run this procedure on substrings W of S, and a $(1+\epsilon)k$-mismatch gd-repeat which is maximal in W may not be maximal in S. This will become clearer in the next section, in particular when we prove Lemma 7.2.6 and Lemma 7.2.7.

7.2.2.2 Finding $(1+\epsilon)k$-mismatch gd-repeats containing all k-mismatch repeats

We make the following changes in Steps (1), (2), and (3) of the algorithm in Section 7.2.1.

Step 1. In this step we find a set of maximal $(1+\epsilon)k$-mismatch gd-repeats that contain at least one of the characters at positions $\text{head}(B_i) - 1$, $\text{head}(B_i)$, or $\text{head}(B_i) + 1$, and do not contain the character at position $\text{head}(B_{i+1}) - 1$ for every $1 < i < r'$ (The treatment of blocks B_1, and $B_{r'}$ is slightly different and described below). Let u be a (maximal) k-mismatch gd-repeat. This set is such that if u is a substring of a $(1+\epsilon)k$-mismatch gd-repeat that contains at least one of the characters at positions $\text{head}(B_i) - 1$, $\text{head}(B_i)$, or $\text{head}(B_i) + 1$, and u is not a substring of a $(1+\epsilon)k$-mismatch gd-repeat that contains $\text{head}(B_{i+1}) - 1$, then u is a substring of a $(1+\epsilon)k$-mismatch gd-repeat in the set.

From Lemma 7.2.1 we get that the right root of a $(1+\epsilon)k$-mismatch gd-repeat cannot contain as a substring $(1+\epsilon)k+1$ consecutive Lempel-Ziv factors of S. This implies that the right root of a $(1+\epsilon)k$-mismatch gd-repeat containing position $\text{head}(B_i) - 1$, $\text{head}(B_i)$, or $\text{head}(B_i) + 1$, for $1 < i < r'$ cannot start before the first character of B_{i-1}. Otherwise, it contains at least $k+2$ factors of block B_{i-1}, and $k+1$ factors of block B_i, and since $(2k+3) > (1+\epsilon)k+1$ we contradict Lemma 7.2.1. So we can limit ourselves to approximate gd-repeats of period length at most $|B_{i-1}B_iB_{i+1}|$. Similarly, for $i = r'$, we can limit ourselves to approximate gd-repeats of period length at most $|B_{r'-2}B_{r'-1}B_{r'}|$ (remember

that $B_{r'}$ may contain less than $k+2$ factors), and for $i=1$, we look at approximate gd-repeats of period length at most $|B_1B_2|/2$, We use the following lemma which is similar to Lemma 3.4 in [57].

Lemma 7.2.5 *Let w be a $(1+\epsilon)k$-mismatch gd-repeat of period length $p \leq |B_{i-1}B_iB_{i+1}|$ containing one of the characters at positions head$(B_i)-1$, head(B_i), or head$(B_i)+1$, and not containing the character at position head$(B_{i+1})-1$ for some $1<i<r'$. Let ℓ be the position of the first character of B_{i-1}. Then w cannot start at position $\leq \ell - |B_{i-1}B_iB_{i+1}|$. In particular $|w| < 2|B_{i-1}B_iB_{i+1}|$.*

For $i=r'$, any $(1+\epsilon)k$-mismatch gd-repeat w, of period length $p \leq |B_{r'-2}B_{r'-1}B_{r'}|$ containing the character head$(B'_r)-1$, cannot start at position $\leq \ell - |B_{r'-2}B_{r'-1}B_{r'}|$, where ℓ is the first character of $B_{r'-2}$. In particular $|w| < 2|B_{r'-2}B_{r'-1}B_{r'}|$.

Proof: Let $1<i<r'$. Assume that w starts at position $\leq \ell - |B_{i-1}B_iB_{i+1}|$. Consider the suffix w' of w that starts at position $\ell - p$, and the suffix w'' of w that starts at position ℓ. Note that w' is well defined since $p \leq |B_{i-1}B_iB_{i+1}|$. The string w'' contains at least $k+2$ factors of block B_{i-1} and $k+1$ factors of block B_i. Each such factor $f = S[a \cdots b]$ in w'' has a mismatch with the substring $S[a-p \cdots b-p]$ (which may overlap f). So we obtain a contradiction since w has more than $2k+3 > (1+\epsilon)k+1$ mismatches. The proof of the second part of the lemma is similar. ∎

For every $1<i<r'$, Lemma 7.2.5 shows that it suffices to search for a set of $(1+\epsilon)k$-mismatch gd-repeats that span at least one of the positions head$(B_i)-1$, head(B_i), or head$(B_i)+1$, and do not span head$(B_{i+1})-1$ only within the substring $W = S[\ell - |B_{i-1}B_iB_{i+1}| \cdots \text{head}(B_{i+1})-1]$ where ℓ is the position of the first character of B_{i-1}. We do that by applying the procedure of Section 7.2.2.1 twice to find a set of $(1+\epsilon)k$-mismatch gd-repeats of W containing all maximal k-mismatch gd-repeats of W that span $W[|B_{i-1}B_iB_{i+1}| + |B_{i-1}B_i| - 1] = S[\text{head}(B_i)-1]$, and $W[|B_{i-1}B_iB_{i+1}| + |B_{i-1}B_i| + 1] = S[\text{head}(B_i)+1]$.

For $i=1$, B_{i-1} is not defined so instead we set $W = S[1 \cdots \text{head}(B_2)-1]$ and apply the procedure of Section 7.2.2.1 twice to find a set of $(1+\epsilon)k$-mismatch gd-repeats of W containing all k-mismatch gd-repeats of W that span $W[|B_1|-1] = S[\text{head}(B_1)-1]$, and $W[|B_1|+1] = S[\text{head}(B_1)+1]$.

For $i=r'$, we set $W = S[\ell - |B_{r'-2}B_{r'-1}B_{r'}| \cdots \text{head}(B_{r'})]$ where ℓ is the position of the first character of $B_{r'-2}$, and apply the procedure of Section 7.2.2.1 to find the $(1+\epsilon)k$-mismatch gd-repeats of W that span $S[\text{head}(B'_r)-1]$.

When searching for repeats containing head$(B_i)+1$, if we find a repeat that also contains head$(B_{i+1})-1$ we discard it.

We also search for $(1+\epsilon)k$-mismatch gd-repeats that contain the first character of S and does not contain the character at position head$(B_1)-1$, by applying the procedure of Section 7.2.2.1 to the string $S[1 \cdots \text{head}(B_1)-1]$ to find approximate gd-repeats that span $S[1]$. We discard approximate repeats that contain head$(B_1)-1$.

The following lemma specifies the crucial property of the repeats which we find in this step.

Lemma 7.2.6 *Let u be a maximal k-mismatch gd-repeat of period length p such that there is a maximal $(1+\epsilon)k$-mismatch gd-repeat u' of period length p that contains u and spans at least one of the positions* $\text{head}(B_i)-1$, $\text{head}(B_i)$, *or* $\text{head}(B_i)+1$ *for some* $1 < i < r'$, $\text{head}(B_{r'})-1$, $\text{head}(B_{r'})$, *or* $S[1]$. *Then we find a maximal $(1+\epsilon)k$-mismatch of period length p containing u in Step (1). (This repeat is not necessarily u'.)*

Proof: Assume that u' is the maximal $(1+\epsilon)k$-mismatch gd-repeat of period length p containing u whose right endpoint is the largest.

Assume also that u' spans one of the positions $\text{head}(B_i)-1$, $\text{head}(B_i)$, or $\text{head}(B_i)+1$ but does not span $\text{head}(B_{i+1})-1$ for some $1 < i < r'$. Consider the applications of the procedure of Section 7.2.2.1 within the substring $W = S[\ell - |B_{i-1}B_iB_{i+1}| \cdots \text{head}(B_{i+1})-1]$ where ℓ is the position of the first character of B_{i-1}, to find repeats that span $\text{head}(B_i)-1$, $\text{head}(B_i)$, or $\text{head}(B_i)+1$. By Lemma 7.2.5, u' (and therefore u) is contained in W. Thus, it follows from Lemma 7.2.4 that the procedure of Section 7.2.2.1 when applied to W indeed finds some maximal $(1+\epsilon)k$-mismatch gd-repeat u'' of period length p that contains u. By our assumption about u', u'' does not contain $\text{head}(B_{i+1})-1$ and therefore is not discarded.

The cases where u' spans 1) position $\text{head}(B_{r'})-1$, 2) one of the positions $\text{head}(B_1)-1$, $\text{head}(B_1)$, and $\text{head}(B_1)+1$, but does not span position $\text{head}(B_2)-1$, or 3) contains $S[1]$ and does not span $\text{head}(B_1)-1$ are similar. ∎

The running time of the procedure for block $B_i, 1 < i < r'$ that uses the algorithm for the k-mismatch problem of Chapter 6.2 is $O\left(\frac{1}{\epsilon}|B_{i-1}B_iB_{i+1}|k^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k\right)$. The running time for block B_1 is $O\left(\frac{1}{\epsilon}|B_1B_2|k^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k\right)$. The running time for block $B_{r'}$ is $O\left(\frac{1}{\epsilon}|B_{r'-2}B_{r'-1}B_{r'}|k^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k\right)$. We find $(1+\epsilon)k$ gd-repeats (at most one per period length) that contain the first character of S and do not contain the character at position $\text{head}(B_1)-1$, in $O\left(\frac{1}{\epsilon}|B_1|k^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k\right)$ time. Summing up we obtain that the running time for all blocks is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k\right)$.

Step 2. Let $f_i \cdots f_{i+r}$ be the factors of B. At this step we compute a set of $(1+\epsilon)k$-mismatch gd-repeats inside block B that contain at least one of the characters at positions $\text{head}(f_j)-1$, $\text{head}(f_j)$, or $\text{head}(f_j)+1$ for $i \le j < i+r$, and do not contain $S[\text{head}(B)-1]$ and the first character of f_i. Let u be a (maximal) k-mismatch gd-repeat. Then if u is a substring of a $(1+\epsilon)k$-mismatch gd-repeat that contains at least one of the characters at positions $\text{head}(f_j)-1$, $\text{head}(f_j)$, or $\text{head}(f_j)+1$, and u is not a substring of $(1+\epsilon)k$-mismatch gd-repeat that contains $S[\text{head}(B)-1]$ or the first character of f_i, then u is a substring of a $(1+\epsilon)k$-mismatch gd-repeat in the set.

We use the same recursive algorithm as in Step (2) of Section 7.2.1.2, using the algorithm of Section 7.2.2.1 instead of the algorithm of Section 7.2.1.2. The recursive procedure has

$\log k$ levels of recursion. In each level we use the algorithm of Section 7.2.2.1 on strings of total length $|B|$. Thus the total running time for block B is $O\left(\frac{1}{\epsilon}|B|k^{\frac{2}{3}}\log^{\frac{1}{3}} n\log^2 k\right)$. The running time for all blocks is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log^2 k\right)$.

The following lemma specifies the property of the repeats that we have at the end of this stage.

Lemma 7.2.7 *At the end of Step (2) we have a set of maximal $(1+\epsilon)k$-mismatch gd-repeats each spanning position $head(f)-1$, $head(f)$, or $head(f)+1$ for some factor f, or contains $S[1]$, with the following property. For every maximal k-mismatch gd-repeat u of period length p, contained in a maximal $(1+\epsilon)k$-mismatch gd-repeat u' of period length p that spans $head(f)-1$, $head(f)$, or $head(f)+1$, for some factor f, or contains $S[1]$, we have a repeat of period length p in the set which contains u.*

Proof: Let u be a maximal k-mismatch gd-repeat of period length p satisfying the conditions of the lemma, such that we have not found a $(1+\epsilon)k$-mismatch gd-repeat of period length p containing u in Step (1). We show that we find a maximal $(1+\epsilon)k$-mismatch gd-repeat of period length p containing u in Step (2).

By Lemma 7.2.6 we know that u is contained within a block B. Furthermore Lemma 7.2.6 also implies that all $(1+\epsilon)k$-mismatch repeats that contain u are contained within B, do not span $B[1]$, and do not span $B[\text{head}(B)-1]$.

When we apply Step (2) to B, let $B' = f_i \cdots f_{i+r}$ be the largest sub-block of B such that there exists a $(1+\epsilon)k$-mismatch gd-repeat of period length p containing u which spans one of the characters at positions $\text{head}(f_j)-1$, $\text{head}(f_j)$, and $\text{head}(f_j)+1$ for $j = i+\lfloor r/2 \rfloor$, but there isn't a $(1+\epsilon)k$-mismatch gd-repeat of period length p containing u that spans either the first character of B' or one of the last two characters of B'. The existence of B' follows from the assumptions of the lemma and the definition of Step (2).

By Lemma 7.2.4 when we apply the procedure of Section 7.2.2.1 to B' we find a $(1+\epsilon)k$ mismatch gd-repeat of period length p containing u. Since this repeat does not contain the first character of B' or the next to last character we do not discard it. ■

Step 3. Let u be a k-mismatch gd-repeat of period length p which is not contained in a $(1+\epsilon)k$-mismatch gd-repeat of period length p which we found in Step (1) or Step (2). By Lemma 7.2.7 every $(1+\epsilon)k$-mismatch gd-repeat containing u of period length p is contained in $S[i+1\cdots j-2]$ for some factor $f = S[i\cdots j]$.

To find $(1+\epsilon)k$-mismatch repeats of period length p containing each such k-mismatch gd-repeat of period length p we run exactly the same procedure of Step (3) of the algorithm of Section 7.2.1.2. This takes $O(n+z)$ time, where z is the number of maximal $(1+\epsilon)k$-mismatch repeats which we find.

The next lemma establishes the correctness of this step.

Lemma 7.2.8 *For every period length p, we find in Step (3) maximal $(1+\epsilon)k$-mismatch*

gd-repeats of period length p containing all maximal k-mismatch gd-repeats of period length p which are not contained within gd-repeats of period length p found in Steps (1) and (2).

Proof: Assume the contrary. Then, let $u = S[a \cdots b]$ be the first maximal k-mismatch gd-repeat of period length p not contained in a repeat found by the algorithm. Since u is not contained in a gd-repeat of period length p found in Steps (1) and (2), then by Lemma 7.2.7 every maximal $(1+\epsilon)k$-mismatch repeat $u' = S[a' \cdots b']$ of period length p containing u is contained in the substring $S[i+1 \cdots j-2]$ of some factor $f = S[i \cdots j]$.

Let $S' = S[i-\rho \cdots j-1-\rho]$ be the previous copy of $S[i \cdots j-1]$ in S. By Lemma 7.2.3, $\hat{u} = S[a-\rho \cdots b-\rho]$ is a maximal k-mismatch gd-repeat of period length p contained in $S[i+1-\rho \cdots j-2-\rho]$, and every maximal $(1+\epsilon)k$-mismatch repeat $\hat{u}'$ of period length p containing $\hat{u}$ is contained in $S[i+1-\rho \cdots j-2-\rho]$. Since u is the first k-mismatch gd-repeat of period length p that is not contained in a repeat found by the algorithm, we have already found a repeat of period length p containing $\hat{u}$. Since this repeat is completely within $S[i+1-\rho \cdots j-2-\rho]$, we must have found its copy in $S[i+1 \cdots j-2]$ which gives a contradiction. ■

The running time of the algorithm is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n \log^2 k + z\right)$, where z is the number of maximal $(1+\epsilon)k$-mismatch gd-repeats in S.

7.2.3 Using the algorithm for approximate gd-repeats to find approximate tandem repeats

We now describe how to use the algorithm for approximate k-mismatch gd-repeats of Section 7.2.2 to solve the approximate k-mismatch tandem repeat problem in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n \log^2 k + z)$ time, where z is the number of $(1+\epsilon)k$-mismatch tandem repeats in S. This is in contrast with the algorithm in Chapter 7.1 whose running time is $O(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k \log(n/k) + z)$. That is we replace a $\log(n/k)$ factor by a $\log k$ factor.

Let u be one of the $(1+\epsilon)k$-mismatch gd-repeats of period length p that the algorithm of Section 7.2.2 finds. Then clearly, each substring of u of length $2p$ is a $(1+\epsilon)k$-mismatch tandem repeat. On the other hand, each k-mismatch tandem repeat of length $2p$ is contained in some maximal k-mismatch gd-repeat of period length p. Since any maximal k-mismatch gd-repeat of period length p is a substring of a repeat of period length p reported by the algorithm of Section 7.2.2, it follows that all k-mismatch tandem repeats of length $2p$ are substrings of gd-repeats of period length p found by the algorithm of Section 7.2.2.

Following this observation our algorithm runs first the algorithm for approximate k-mismatch gd-repeats of Section 7.2.2. Then we report every substring of length $2p$ of every gd-repeat of period length p that the algorithm of Section 7.2.2 finds. However, since the approximate gd-repeats found by the algorithm of Section 7.2.2 may overlap, we need to be careful not to report a $(1+\epsilon)k$-mismatch tandem repeat more than once.

To report each approximate tandem repeat exactly once, we use the array A that was

built in Step (3) of the algorithm of Section 7.2.2. Recall that at the end of that algorithm, we have an array A such that $A[j]$ is a linked list of all approximate k-mismatch gd-repeats ending at position j sorted in descending order of their starting positions. We use this array to report each approximate tandem repeat once as follows. We maintain for each period length p an index $x(p)$, which holds the position of the last character of the last approximate tandem repeat we reported for period length p. We initialize $x(p)$ to 0. We scan the array A from its first entry to its last entry. Suppose that $A[j]$ contains an approximate gd-repeat $u = S[i \cdots j]$ of period length p (since gd-repeats are maximal there is only one repeat per period, in each entry of the array). Then we report all substrings of u of length $2p$ that end between position $x(p) + 1$ and position j, and set $x(p) = j$. It is easy to verify that 1) we report every substring of length $2p$ of each gd-repeat of period length p and 2) each such substring is reported once.

To establish the claimed running time we have to show that the number of approximate gd-repeats that the algorithm of Section 7.2.2 finds, is no greater than the number of $(1 + \epsilon)k$-mismatch tandem repeats. Indeed, each gd-repeat that the algorithm of Section 7.2.2 finds is a maximal $(1 + \epsilon)k$-mismatch gd-repeat and therefore at most one of these gd-repeats of period length p starts at a particular position. It follows that mapping each gd-repeat to its prefix of length $2p$ is a one to one mapping from maximal gd-repeats to tandem repeats. So the number of approximate gd-repeats that the algorithm of Section 7.2.2 finds is no greater than the number of $(1 + \epsilon)k$-mismatch tandem repeats.

7.3 Another algorithm for the approximate k-mismatch gd-repeat problem

In fact we can also adapt the algorithm for k-mismatch tandem repeats of Landau, Schmidt, and Sokol [64] so that it finds maximal k-mismatch gd-repeats. In each iteration instead of finding tandem repeats of all possible lengths crossing one of the middle positions we find maximal k-mismatch gd-repeats of all possible period lengths crossing one of the middle positions. The running time would be $O(nk \log(n/k)+z)$ where z is the number of maximal k-mismatch gd-repeats.

In a way similar to what we did for approximate tandem repeats in Chapter 7.1, we adapt this algorithm to solve the approximate k-mismatch gd-repeat problem in $O(\frac{1}{\epsilon}nk^{\frac{2}{3}} \log^{\frac{1}{3}} n \log k \log(n/k) + z)$ time, where z is the number of $(1 + \epsilon)k$-mismatch gd-repeats in S. The algorithm consists of the following steps.

1. We find for each period length p the (maximal) $(1+\epsilon)k$-mismatch gd-repeat of period length p that spans $S[1]$, if it exists. We can do it by applying the algorithm of Section 7.2.2.1 for finding approximate gd-repeats that span a particular character, to the string S and the character $S[1]$.

2. We find for each period length p the (maximal) $(1+\epsilon)k$-mismatch gd-repeat of period length p that spans $S[n]$ and does not span $S[1]$, if it exists. We do that by applying the algorithm of Section 7.2.2.1 for finding approximate gd-repeats that span a particular character, to the string S and the character $S[n]$. Then we eliminate those gd-repeats that span $S[1]$.

3. We apply the recursive procedure *find-approx-repeats* to the string S. The procedure *find-approx-repeats* gets as input a string W, and finds a set of maximal $(1+\epsilon)k$-mismatch gd-repeats such that each maximal k-mismatch gd-repeat that is not contained in a $(1+\epsilon)k$-mismatch gd-repeat that spans $W[1]$ or $W[|W|]$ is a substring of a gd-repeat in the set. This set will not contain the gd-repeats that span $W[1]$ or $W[|W|]$.

The recursive procedure *find-approx-repeats* when applied to a string W works as follows. If $|W| \leq 2(1+\epsilon)k$, the procedure returns, since for each period length, W is a $(1+\epsilon)k$-mismatch gd-repeat. Assume that $|W| > 2(1+\epsilon)k$. Let $h = \lfloor |W|/2 \rfloor$. We apply the procedure of Section 7.2.2.1 to W and the character $W[h]$ and eliminate any approximate gd-repeats containing $W[1]$ or $W[|W|]$. We also apply the procedure of Section 7.2.2.1 to W and the character $W[h+1]$, this time eliminating from its output any approximate gd-repeats containing $W[h]$ or $W[|W|]$. We recursively apply find-approx-repeats to $W[1 \cdots h]$ and to $W[h+1 \cdots |W|]$. The next lemma which is similar to Lemma 7.2.7, establishes the correctness of the algorithm.

Lemma 7.3.1 *The algorithm described above finds a set A of maximal $(1+\epsilon)k$-mismatch gd-repeats such that each maximal k-mismatch gd-repeat is a substring of a repeat in A.*

Proof: It is easy to verify from the implementation of the algorithm described above, and from the implementation of the procedure of Section 7.2.2.1, that we find a set of maximal $(1+\epsilon)k$-mismatch gd-repeats. Let u be a maximal k-mismatch gd-repeat of period length p. If u is contained in a maximal $(1+\epsilon)k$-mismatch gd-repeat u' that spans $S[1]$ then we find u' in Step 1. Similarly, if u is contained in a maximal $(1+\epsilon)k$-mismatch gd-repeat u' that spans $S[n]$ and does not span $S[1]$, then we find u' in Step 2.

Assume now that all maximal $(1+\epsilon)k$-mismatch gd-repeats that contain u are contained in $S[2 \cdots n-1]$. Let W be the longest substring of S to which we apply the procedure find-approx-repeats and $W[2 \cdots |W|-1]$ contains a maximal $(1+\epsilon)k$-mismatch gd-repeat of period length p that contains u, and spans either position h or position $h+1$ of W where $h = \lfloor |W|/2 \rfloor$. It is easy to verify that such a string W exists and furthermore that there is no $(1+\epsilon)k$-mismatch gd-repeat of period length p that contains u, and spans either the first or the last position of W. By Lemma 7.2.4, when we apply the procedure of Section 7.2.2.1 to W and the character $W[h]$ or $W[h+1]$, we find a $(1+\epsilon)k$-mismatch gd-repeat of period length p containing u. Since this repeat does not contain $W[1]$ or $W[|W|]$, we do not discard it. ■

The running time of Step 1 and Step 2 is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log k\right)$. The procedure *find-approx-repeats* has $\log(n/k)$ levels of recursion. In each level we use the algorithm of Section 7.2.2.1 on strings of total length n. The running time of the procedure *find-approx-repeats* on the string S is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n\log k\log(n/k)+z\right)$, where z is the number of $(1+\epsilon)k$-mismatch gd-repeats in S, and this is also the running time of the algorithm.

7.4 Approximating runs of k-mismatch tandem repeats

We recall from Chapter 5 that Kolpakov and Kucherov [57] also defined another type of repeat called a *run of k-mismatch tandem repeats*. A string $R[1\cdots n]$ is a *run of k-mismatch tandem repeats of period length $p \leq n/2$*, if every substring of R of length $2p$ is a k-mismatch tandem repeat. String R is a maximal such run if it cannot be extended to the left or to the right and remain a run of k-mismatch tandem repeats of period length p. They gave an algorithm for finding all maximal runs of k-mismatch tandem repeats in a string of length n that runs in $O(nk\log k+z)$ time where z is the number of such runs.

We define the *approximate runs of k-mismatch tandem repeats problem* as follows. Given a string S and $\epsilon > 0$, we want to find a set A of substrings of S which are (not necessarily maximal) runs of $(1+2\epsilon)k$-mismatch tandem repeats, such that:
1) Each maximal run of k-mismatch tandem repeats in S of period length p is contained in one of these repeats of period length p, and
2) The size of the set A that we find is bounded by the number of maximal runs of $(1+\epsilon)k$-mismatch tandem repeats in S.

In Section 7.4.1 we describe an algorithm similar to the algorithm for finding k-mismatch tandem repeats of Landau, Schmidt, and Sokol [64] that finds all maximal runs of k-mismatch tandem repeats in $O(nk\log(n/k)+z)$ time where z is the number of runs of k-mismatch tandem repeats in S. In Section 7.4.2, we show how to modify this algorithm so that we solve the approximate runs of k-mismatch tandem repeats problem.

7.4.1 Algorithm for finding runs of k-mismatch tandem repeats

We use some ideas from the algorithm of [57] for finding maximal runs of k-mismatch tandem repeats. We refer to a maximal run of k-mismatch tandem repeats as *run of repeats* for short, and by *subrun of repeats* or simply a *subrun*, we refer to a run of k-mismatch tandem repeats which is not necessarily maximal.

The algorithm finds in each step subruns of k-mismatch tandem repeats. The main difficulty of the algorithm is in assembling these subruns into runs. To get the correct time bound we have to ensure that we never accumulate more than $O(n+z)$ subruns, where z is the number of runs of k-mismatch tandem repeats.

It is easy to see that each run of repeats is a union (not necessarily disjoint) of k-mismatch gd-repeats. The main idea is to find k-mismatch gd-repeats, then to combine them into subruns, and finally to combine the subruns into runs of k-mismatch repeats.

We use a procedure that given period length p and a character $S[\ell]$, finds subruns of k-mismatch tandem repeats of period length p that are contained in $S[\ell-2p+1\cdots\ell+2p-1]$.

7.4.1.1 Finding subruns of k-mismatch tandem repeats of period length p contained in the substring $S[\ell-2p+1\cdots\ell+2p-1]$

For a period length p and a character $S[\ell]$, we compute subruns of period length p that are contained in the substring $S[\ell-2p+1\cdots\ell+2p-1]$ and are maximal within this substring. (To be precise we in fact use $S[\max\{1,\ell-2p+1\}\cdots\min\{\ell+2p-1,|S|\}]$ but we omit the max and the min here and in the rest of this section to simplify the notation.) Specifically, we find all subruns of period length p in the substring $S[\ell-2p+1\cdots\ell+2p-1]$, which are maximal within this substring and their last character is in the substring $S[\ell\cdots\ell+2p-1]$.

We first find subruns whose right root starts to the right of (or at) $S[\ell]$ by applying the procedure of Section 7.2.1.1 to $S[\ell]$. Using the same definitions of Section 7.2.1.1, the gd-repeats of period length p in S whose right root starts to the right of (or at) $S[\ell]$ correspond to indices $1\leq i\leq k+1$ such that the following holds

$$LP_p(i)+LS_p(k+2-i)\geq p. \tag{7.2}$$

If indeed $LP_p(i)+LS_p(k+2-i)\geq p$ we take as our subrun the part of the gd-repeat that starts at position $\max\{\ell-LS_p(k+2-i),\ell-2p+1\}$ and ends at position $\min\{\ell+p+LP_p(i)-1,\ell+2p-1\}$.

We define a subrun v and a subrun u of period length p to be *mergeable* if either u or v contains the other, or if the length of the overlap between u and v is at least $2p-1$. It is clear that two mergeable subruns of period length p are not maximal as their union is a subrun of period length p. By *merging* u and v we mean replacing them by their union.

For every $i>1$ if we find a subrun associated with the index i (that is if $LP_p(i)+LS_p(k+2-i)\geq p$) we merge it, if possible, with the last subrun that we found (which is associated with some index $j<i$). The subruns which we find are organized in doubly linked list sorted by their start position. We keep pointers to the first subrun and to the last subrun in the list.

Subruns of period length p in S whose right root starts to the left of $S[\ell]$ are found similarly. We concatenate the list of subruns whose right root starts to the left of $S[\ell]$ with the list of subruns whose right root starts to the right of (or at) $S[\ell]$. We merge the last subrun in the list of subruns whose right root starts to the left of $S[\ell]$ with the first subrun in the list of subruns whose right root starts to the right of (or at) $S[\ell]$, if they are mergeable.

Since our initial subruns are maximal gd-repeats and we merge subruns whenever possible it is straightforward to prove that if a subrun which we found ends before $S[\ell+2p-1]$

then it cannot be extended to a longer subrun by adding characters to its right. Similarly, if a subrun starts after $S[\ell - 2p + 1]$ then it cannot be extended to a longer subrun by adding characters to its left. It follows that each subrun in our list other than possibly the first and the last is in fact a run.

Since the running time of the procedure of Section 7.2.1.1 is $O(k)$ then it takes $O(k)$ time to compute this list of subruns for the particular period length p.

7.4.1.2 Finding all runs of k-mismatch repeats

We maintain subruns in explored intervals. An *explored interval* I_p is a substring, say $S[i \cdots j]$, with a period length p associated with it. An explored interval also has subruns and runs associated with it which are defined as follows. The first subrun z associated with I_p is a suffix of the first run of period p that ends in $S[i \cdots j]$. If this run starts at or after character $S[i - 2p + 1]$ then z is the entire run and otherwise z is the suffix of this run which starts with the character $S[i - 2p + 1]$. The explored interval I_p is also associated with all runs that follow z and end in $S[i \cdots j]$. The last subrun associated with I_p is the prefix (up to position j) of the first run that ends following position j, if that run does not start at or following position $j - (2p - 2)$. Note that if I_p contains just one subrun which is both the first and the last then this subrun may be neither a prefix nor a suffix of a run, but some substring of it. We maintain the subruns associated with I_p in a doubly linked list sorted by their starting positions. We keep pointers to the first subrun and to the last subrun in the list.

Let $I^1 = S[i \cdots j]$ and $I^2 = S[j+1 \cdots t]$ be two explored intervals of period length p. We *concatenate* I^1 and I^2 to one interval $I = S[i \cdots t]$ of period length p by 1) concatenating the list of subruns of I^1 with the list of subruns of I^2, and 2) merging the last subrun in I^1 with the first subrun of I^2 if they are mergeable.

By definition, for a period length p, $p \leq k$, $S[1 \cdots n]$ is a run of k-mismatch tandem repeats. Therefore, we will only consider periods of length greater than k. The algorithm consists of the following steps.

1. We check for each period length p (such that $2k + 2 \leq 2p \leq n$) whether the string $S[1 \cdots 2p]$ is a k-mismatch tandem repeat. This is easily done using the suffix tree of S in $O(k)$ time per period length. If the string $S[1 \cdots 2p]$ is indeed a subrun we associate it with the explored interval $S[2p \cdots 2p]$ of period length p.

2. Similarly, we check for each period length p (such that $2k + 2 \leq 2p < n$) whether the string $S[n-2p+1 \cdots n]$ is a k-mismatch tandem repeat. If so we associate this subrun with the explored interval $S[n \cdots n]$ of period length p. For the period length p such that $n = 2p + 1$, we concatenate the explored intervals $S[2p \cdots 2p]$ and $S[n \cdots n]$.

3. If $n > 2k + 3$, we call a recursive procedure *find-subruns* giving it as parameters the string S and for each period length $p > k$ such that $n > 2p+1$, the explored intervals

$S[2p \cdots 2p]$ and $S[n \cdots n]$ of period length p.

In general, the procedure find-subruns gets as input a substring $S[i \cdots j]$ of S of length $\ell = j - i + 1$, and for each period length p, such that $2p + 1 < \ell$ an explored interval $I_i(p)$ that ends at $S[i + 2p - 1]$, and an explored interval $I_j(p)$ that starts at $S[j]$. For each period length p such that $2p + 1 < \ell$ it extends $I_i(p)$ to the right and $I_j(p)$ to the left until they are eventually concatenated. Recall that since we only handle periods of length $p \geq k + 1$, then $2k + 3 \leq 2p + 1 < \ell$. So the procedure works on substrings of length $\ell > 2k + 3$

We now describe the implementation of the procedure find-subruns when operating on the substring $S[i \cdots j]$. We refer only to period lengths p such that $2k + 3 \leq 2p + 1 < \ell$ and recall that we have for each such period length an explored interval $I_i(p)$ that ends in position $S[i + 2p - 1]$, and an explored interval $I_j(p)$ that starts at $S[j]$.

Let $h = \lfloor \ell/2 \rfloor$ and let $m = i + h - 1$. For each such period length p, we create the explored interval $I_m(p) = S[\max\{i+2p, m\} \cdots \min\{m+2p-1, j-1\}]$. We use the algorithm of Section 7.4.1.1 to find subruns in $S[\max\{i, m-2p+1\} \cdots \min\{m+2p-1, j\}]$, ending in $S[m \cdots m+2p-1]$, that are maximal within this substring. Then we truncate (if necessary) each of the subruns which we found so that the truncated subrun does not end after $S[j-1]$ and does not start before $S[i+1]$, and remove it if its length is smaller than $2p$ after the truncation. For each p, such that $I_m(p)$ starts at $S[i+2p]$ we concatenate $I_i(p)$ with $I_m(p)$. Similarly, for each p, such that $I_m(p)$ ends at $S[j-1]$ we concatenate $I_m(p)$ with $I_j(p)$. Let $I'_m(p)$ be the explored interval containing $I_m(p)$ after these steps. If $m - i + 1 > 2k + 3$ we recursively apply find-subruns to $S[i \cdots m]$ with the explored intervals $I_i(p)$ and $I'_m(p)$ for each p such that $2p + 1 < m - i + 1$. Let $I'_m(p)$ be the resulting explored interval. If $j - m + 1 > 2k + 3$ we recursively apply find-subruns to $S[m \cdots j]$ with the explored intervals $I'_m(p)$ and $I_j(p)$ for each p such that $2p + 1 < j - m + 1$.

It is easy to see that since we merge subruns whenever possible, the algorithm outputs maximal runs of k-mismatch tandem repeats. To prove that the running time is indeed $O(nk \log(n/k) + z)$, we now show that we never accumulate more than $O(n + z)$ subruns of k-mismatch tandem repeats, where z is the number of runs of k-mismatch tandem repeats in S.

Lemma 7.4.1 *At any point of the algorithm we never have more than $O(n)$ subruns that are not maximal.*

Proof: Notice that the recursion has a structure of a complete binary tree of depth $O(\log(n/k))$, (since we stop the recursion when we reach a string of length at most $2k+3$), where each vertex corresponds to a recursive call to find-subruns and the algorithm visits the vertices of the tree in some depth first order. We associate each explored interval with the node corresponding to the recursive call by which it was created. Consider a node v of depth i where find-subruns is applied to a substring $S[a \cdots b]$ of S of length $\ell \leq \lceil n/2^i \rceil$.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	0	0	0	1	0	0	0	1	1	1	0	1	1	0	1	1

Figure 7.3: An example of the algorithm for finding runs of repeats. We assume that $k = 1$.

The relevant period lengths for which this call creates an explored interval are p such that $2p + 1 < \ell$. In each of these explored intervals only the first and last subruns may not be maximal. Therefore it follows that at most $\lceil n/2^i \rceil$ subruns that are associated with v are not maximal.

It is easy to see that the subruns that may not be maximal at the time we visit the node v are those that are associated with v and its ancestors. So we have at most $\sum_{j=0}^{i} \lceil n/2^i \rceil = O(n)$ subruns that are not maximal when we visit v. ∎

Using similar arguments as in Section 7.3, the total running time is $O(nk \log(n/k) + z)$.

An example of the algorithm for finding runs of k-mismatch tandem repeats We assume that $k = 1$. Figure 7.3 shows the string S . Our algorithm runs simultaneously on all period lengths, but we demonstrate it only for period length $p = 2$. In Step (1) we create the explored interval $S[4 \cdots 4]$ with the subrun $S[1 \cdots 4] = 0000$. In Step (2) we create the explored interval $S[17 \cdots 17]$ with the subrun $S[14 \cdots 17] = 1011$.

Then we apply the procedure find-subruns to S. The procedure creates the explored interval $S[8 \cdots 11]$ and finds the subruns that are contained in $S[5 \cdots 11]$. It finds the subrun $S[5 \cdots 9]$ and the subrun $S[8 \cdots 11]$. We now apply the procedure find-subruns to $S[1 \cdots 8]$ giving it as parameters the explored intervals $S[4 \cdots 4]$ and $S[8 \cdots 11]$. The procedure searches for subruns that are contained in $S[2 \cdots 7]$ and end in the explored interval $S[5 \cdots 7]$. It finds the subrun $S[2 \cdots 7]$. We then concatenate the interval $S[4 \cdots 4]$ with the interval $S[5 \cdots 7]$ and get the explored interval $S[4 \cdots 7]$ that contains the subrun $S[1 \cdots 7]$. We also concatenate $S[4 \cdots 7]$ with $S[8 \cdots 11]$ and end up with the interval $S[4 \cdots 11]$ that contains the subruns $S[1 \cdots 9]$ and $S[8 \cdots 11]$. Notice that we merged the subruns $S[1 \cdots 4]$ with $S[2 \cdots 7]$ and then merged the resulting subrun $S[1 \cdots 7]$ with $S[5 \cdots 9]$ to get the subrun $S[1 \cdots 9]$.

We now handle the second half of S, and apply find-subruns to the string $S[8 \cdots 17]$ giving it the explored intervals $S[4 \cdots 11]$ and $S[17 \cdots 17]$. The procedure finds the subrun $S[9 \cdots 14]$ that ends in the explored interval $S[12 \cdots 15]$. We concatenate $S[4 \cdots 11]$ with $S[12 \cdots 15]$ and get the explored interval $S[4 \cdots 15]$ that contains the subruns $S[1 \cdots 9]$ and $S[8 \cdots 14]$. (Notice that we merged $S[8 \cdots 11]$ with $S[9 \cdots 14]$.)

Last, we make another recursive call to find-subruns with $S[12 \cdots 17]$ giving it the explored intervals $S[4 \cdots 15]$ and $S[17 \cdots 17]$. The procedure finds the subrun $S[13 \cdots 16]$ that ends in the explored interval $S[16 \cdots 16]$. We merge the explored interval $S[4 \cdots 15]$

with $S[16 \cdots 16]$ to create the explored interval $S[4 \cdots 16]$, and last we merge $S[4 \cdots 16]$ with $S[17 \cdots 17]$, creating the explored interval $S[4 \cdots 17]$ that contains the runs $S[1 \cdots 9]$, $S[8 \cdots 14]$ and $S[13 \cdots 17]$.

7.4.2 Finding approximate runs of k-mismatch tandem repeats

We give an algorithm for finding approximate runs of k-mismatch tandem repeats. The running time of the algorithm is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k \log(n/k) + z\right)$, where z is the number of runs of $(1+\epsilon)k$-mismatch tandem repeats in S. The algorithm finds a set A of subruns of $(1+2\epsilon)k$-mismatch tandem repeats such that every run of k-mismatch tandem repeats is a substring of a string in A.

The algorithm has the same structure of the algorithm of Section 7.4.1. We first describe a procedure that given a substring W of S and an index ℓ finds approximate runs of repeats contained in $W[\max\{\ell - 2p + 1, 1\} \cdots \min\{|W|, \ell + 2p - 1\}]$ for all period lengths p.

7.4.2.1 Finding approximate runs of $(1+2\epsilon)k$-mismatch tandem repeats contained in a string W that span the character $W[\ell]$ for all period lengths p

Given a string W and a position ℓ, we describe a procedure that finds for each period length p such that $2p + 1 < |W|$,[5] a set of subruns of $(1+2\epsilon)k$-mismatch repeats that are contained in the substring $W_p = W[\ell - 2p + 1 \cdots \ell + 2p - 1]$. (To be precise we in fact use $W_p = W[\max\{\ell - 2p + 1, 1\} \cdots \min\{|W|, \ell + 2p - 1\}]$.) Each run of k-mismatch tandem repeats that is contained in W_p is a substring of one of the subruns which we find. Moreover, each subrun that we find except for the first and the last overlaps a unique maximal run of $(1+\epsilon)k$-mismatch tandem repeats. The first and last subruns that we find also overlap maximal runs of $(1+\epsilon)k$-tandem repeat which are unique within W_p, but are not necessarily unique within S.

Since we use this procedure only for $\ell = \lfloor |W|/2 \rfloor$, (see Section 7.4.2.2), to simplify the presentation we assume in the sequel that $\ell = \lfloor |W|/2 \rfloor$. To be able to bound the number of subruns that our algorithm finds we also guarantee the following.

1. If our procedure didn't find any subruns in W_p, then W_p contains a substring of length $2p$ which is not a $(1+\epsilon)k$-mismatch tandem repeat.

2. Let $Y = W_p[i \cdots j]$ be the last subrun that our procedure finds. If Y does not stretch to the end of W_p, then the string $W_p[j - 2p + 1 \cdots |W_p|]$ contains a substring of length $2p$ which is not a $(1+\epsilon)k$-mismatch tandem repeat.

[5] We assume in the rest of this section that indeed $2p + 1 < |W|$.

3. Let $Z = W_p[a \cdots b]$ be the first subrun that our procedure finds. If Z starts after the first position of W_p, then the string $W_p[1 \cdots a + 2p - 1]$ contains a substring of length $2p$ which is not a $(1+\epsilon)k$-mismatch tandem repeat.

To provide some intuition to the rather strange constraints that we satisfy, consider first the following algorithm which does not give a good result. Suppose we apply the algorithm of Section 7.2.2.1 for finding approximate gd-repeats that contain a particular character to $W[\ell]$. For period length p we truncate each gd-repeat that we find to W_p, and merge gd-repeats whose overlap is at least $2p-1$ into longer subruns when possible. Since all k-mismatch tandem repeats are contained in the gd-repeats which we found, and we merge gd-repeats whose overlap is at least $2p-1$, this algorithm indeed produces a list of subruns of $(1+\epsilon)k$-mismatch tandem repeats containing all runs of k-mismatch repeats in W_p as substrings. However, we do not know whether the subruns that we found actually contain any k-mismatch tandem repeats, and therefore the number of subruns we find may be greater than the number of runs of k-mismatch tandem repeats. On the other hand, two consecutive subruns that we find may belong to the same run of $(1+\epsilon)k$-mismatch tandem repeats (as we do not find all $(1+\epsilon)k$-mismatch gd-repeats). So we cannot bound the number of runs we find this way either.

To be able to bound the number of subruns that we find, we allow the subrun to contain $(1+2\epsilon)k$-mismatch tandem repeats. But we require that each subrun that we find (except for the first and the last) for a specific period length, will overlap a distinct (maximal) run of $(1+\epsilon)k$-mismatch tandem repeats.

We apply the algorithm of Section 7.2.2.1 to W and ℓ. (See that section for the definitions of $LS_p()$ and $LP_p()$.) We first show how to handle subruns with right root to the right of (or starts at) $W[\ell]$. For each $0 \le i \le \frac{1}{\epsilon} + 1$ let X_i be the part of the substring $W[\ell - LS_p\left(\frac{1}{\epsilon} + 1 - i\right) \cdots \ell + p + LP_p(i) - 1]$ which is within W_p. Recall that X_i is a $(1+\epsilon)k$-mismatch gd-repeat (maximal within W_p) if and only if $|X_i| \ge 2p$. We discard X_i if it is a substring of X_{i+1} or a substring of X_{i-1}. This may happen because we truncate the strings to within W_p. It is easy to verify that we are left with a consecutive sequence of indices i such that X_i is not contained in X_{i+1} or in X_{i-1}.

For each maximal sequence E of consecutive indices such that X_i is a maximal $(1+\epsilon)k$-mismatch gd-repeat in W_p for all $i \in E$, we create a subrun X which is the union of all strings X_i, $i \in E$, and the string $X_{i'}$ where i' is the index following the indices in E if there is such an index.

Notice that if i' exists and $X_{i'}$ does not stretch to the end of W_p then there isn't a $(1+\epsilon)k$-mismatch tandem repeat of length $2p$ in S containing $X_{i'}$. On the other hand if $X_{i'}$ does stretch to the end of W_p, the suffix of W_p of length $2p$ may be a $(1+\epsilon)k$-mismatch tandem repeat. This could happen when the number of mismatches between $W_p[\ell \cdots |W_p| - p]$ to $W_p[\ell + p \cdots |W_p|]$ is between $(i'-1)\epsilon k$ to $i'\epsilon k$. (Notice that this suffix of W_p cannot be a k-mismatch tandem repeat.) This is the reason for combining $X_{i'}$ into the subrun which we create.

In addition, for each subrun X that we create, if the overlap between X, and the previous subrun which we created is at least $2p-1$ we merge them into one subrun.

We generate subruns whose right root starts to the left of $W[\ell]$ analogously, by computing the following (as in Section 7.2.2.1),

$$\begin{aligned} LP'_p(r) &= \max\{j \mid h(W(\ell-p+1,j), W(\ell+1,j)) < r\epsilon k+1\} \text{ and} \\ LS'_p(r) &= \max\{j \mid h(\overleftarrow{W}(\ell-p,j), \overleftarrow{W}(\ell,j)) < r\epsilon k+1\} \ . \end{aligned}$$

for each $0 \le i \le 1/\epsilon + 1$. Let X'_i be the intersection of the substring $W[\ell - p - LS'_p(i) + 1 \cdots \ell + LP'_p\left(\frac{1}{\epsilon}+1-i\right)]$ with W_p.

We generate subruns whose right root is to the left of $W[\ell]$ as we generated subruns with right roots to the right of $W[\ell]$ using the substrings X'_i's instead of the X_i's. (We in fact may modify X'_0 as indicated below, before combining the X'_i's into subruns.) For each maximal sequence E of consecutive indices such that X'_i is a maximal $(1+\epsilon)k$-mismatch gd-repeat in W_p for all $i \in E$, we create a subrun X which is the union of all strings X'_i, $i \in E$, and the string $X_{i'}$ where i' is the index succeeding the indices in E if there is such an index. (Note that our indices are defined such that if $j > j'$ then X'_j starts to the left of $X'_{j'}$ whereas X_j starts to the right of $X_{j'}$.)

We also merge the last subrun whose right root is to the left of $W[\ell]$ with the first subrun whose right root is to the right of (or starts at) $W[\ell]$ if both the strings $V = W[\ell - p \cdots \ell + p - 1]$ (which is the string of length $2p$ whose right root starts at $W[\ell]$) and $U = W[\ell - p - 1 \cdots \ell + p - 2]$ (which is the last string of length $2p$ whose right root starts to the left of $W[\ell]$) are $(1+\epsilon)k$-mismatch tandem repeats.

Notice that if V is a $(1+\epsilon)k$-mismatch tandem repeat then the string X_0 contains it and therefore the first subrun among subruns with right root to the right of $W[\ell]$ which we create contains it. This, however, is not necessarily the case with U and X'_0.

So in order to merge subruns correctly we may have to change the definition of X'_0 in case V is a $(1+\epsilon)k$-mismatch tandem repeat as follows. It is easy to verify that if X'_0 starts at position no greater than $\ell - p - 1$ then X'_0 contains U if and only if U is a $(1+\epsilon)k$-mismatch tandem repeat. So in this case, we leave X'_0 as is. The difficulty is that U may be a $(1+\epsilon)k$-mismatch tandem repeat even when X'_0 starts after position $\ell - p - 1$. If X'_0 starts at position $\ell - p$ or at position $\ell - p + 1$ and V is a $(1+\epsilon)k$-mismatch tandem repeat, we use V to check in constant time if U is a $(1+\epsilon)k$-mismatch tandem repeat as follows. If X_0 contains U then clearly, U is a $(1+\epsilon)k$-mismatch tandem repeat. If X_0 does not contain U, then by definition the number of mismatches between $W[\ell - p \cdots \ell - 1]$ to $W[\ell \cdots \ell + p - 1]$ is $(1+\epsilon)k$, and $W[\ell - p - 1] \neq W[\ell - 1]$. In this case, U is a $(1+\epsilon)k$-mismatch tandem repeat if $W[\ell - 1] \neq W[\ell + p - 1]$. We can check, using the information that we have already computed, in $O(1)$ time, whether either of these conditions is satisfied. If indeed we discover that U is a $(1+\epsilon)k$-mismatch tandem repeat, we set X'_0 to be the string U.

This change to X'_0 guarantees that it contains U if both U and V are $(1+\epsilon)k$-mismatch tandem repeats, and as a result we will merge the last subrun whose right root is to the

left of $W[\ell]$ with the first subrun whose right root is to the right of (or starts at) $W[\ell]$, if and only if both U and V are $(1+\epsilon)k$-mismatch tandem repeats.

The next lemma shows that we find subruns of $(1+2\epsilon)k$-mismatch tandem repeats, and that each subrun overlaps a unique maximal run of $(1+\epsilon)k$-mismatch tandem repeats within W_p. Moreover, by the observation above, all subruns that start after the first position of W_p and end before the last position of W_p, overlap a unique maximal run of $(1+\epsilon)k$-mismatch tandem repeats within S.

Lemma 7.4.2 *The subruns that we find by the algorithm described above satisfy the following properties.*

1. *Each run of k-mismatch tandem repeats which is contained in W_p, is a substring of a subrun that we find.*
2. *Each subrun is a subrun of $(1+2\epsilon)k$-mismatch tandem repeats.*
3. *Each subrun overlaps a unique maximal run of $(1+\epsilon)k$-mismatch tandem repeats within W_p.*
4. *If we didn't find any subruns for period length p in W_p, then W_p contains a substring of length $2p$ which is not a $(1+\epsilon)k$-mismatch tandem repeat.*
5. *If the last string that we find ends at position $j < |W_p|$ then $W_p[j-2p+1 \cdots |W_p|]$ contains a substring of length $2p$ that is not a $(1+\epsilon)k$-mismatch tandem repeat.*
6. *If the first string that we find starts at position $i > 1$ then $W_p[1 \cdots i+2p-1]$ contains a substring of length $2p$ that is not a $(1+\epsilon)k$-mismatch tandem repeat.*

Proof: Every k-mismatch tandem repeat of period length p in W_p contains $W[\ell]$ and therefore must be contained in one of the strings X_i or X'_i (defined above) which is of length at least $2p$. Since each such X_i and X'_i is contained in a subrun which we find then every k-mismatch tandem repeat is contained in one of the subruns that we find. By merging subruns whose overlap is at least $2p-1$, we ensure that each run of k-mismatch tandem repeats within W_p is a substring of a single subrun generated by the algorithm so Property (1) holds.

We now prove Property (2). Let X be a subrun which is the union of the $(1+\epsilon)k$-mismatch gd-repeats $X_i, \cdots, X_j$, (we discuss other types of subruns found by the algorithm below). If $i = j$ then clearly X is a subrun of $(1+\epsilon)k$-mismatch tandem repeats. So assume that $j > i$. First notice that since each $X_{i'}$ contains $W[\ell]$, then all the strings $X_i, \cdots, X_j$ overlap and therefore X is indeed well defined. Each substring of X of length $2p$ which is fully contained in $X_{i'}, i \leq i' < j$ is clearly a $(1+\epsilon)k$-mismatch tandem repeat. It remains to argue that each substring of X of length $2p$ which is not contained in some $X_{i'}$, for $i \leq i' \leq j$, is also $(1+\epsilon)k$-mismatch tandem repeat. (Notice that by construction, if X_j

is not a subrun of $(1+\epsilon)k$-mismatch tandem repeats then $|X_j| < 2p$. So there isn't any substring of length $2p$ that is contained in X_j.)

Let X' be a substring of X of length $2p$ which is not fully contained in any $X_{i'}$, for $i \le i' \le j$. Let $j' < j$ be the largest index such that X' starts in $X_{j'}$. Since j' is largest it follows that X' starts before $X_{j'+1}$. If $j'+1 = j$ then clearly X' ends within $X_{j'+1}$. Otherwise, $|X_{j'+1}| \ge 2p = |X'|$ and since X' starts before $X_{j'+1}$ it also must end within $X_{j'+1}$.

Let $Y = X_{j'}U$ where U is the suffix of $X_{j'+1}$ that is not contained in $X_{j'}$. Then the argument in the preceding paragraph shows that X' is contained in Y. By the definition of the strings X_i, the number of mismatches between $U = S[a \cdots b]$ and $S[a-p \cdots b-p]$, is at most ϵk. Therefore since $X_{j'}$ is a $(1+\epsilon)k$-mismatch gd-repeat it follows that $Y = X_{j'}U$ is a $(1+2\epsilon)k$-mismatch gd-repeat, and so X' is a $(1+2\epsilon)k$-mismatch tandem repeat. The proof for subruns X whose right root is to the left of $W[\ell]$ is similar. The proof for a subrun which is the union of a subrun Y whose right root is to the left of $W[\ell]$, with a subrun X whose right root is to the right of (or start at) $W[\ell]$, follows since both X and Y are subruns of $(1+2\epsilon)k$-mismatch tandem repeats and their overlap is at least $2p-1$.

We now prove Property (3). Consider first a subrun X which is the union of the strings $X_i, \cdots, X_j$. Clearly X contains $(1+\epsilon)k$-mismatch tandem repeats. Let V be the last $(1+\epsilon)k$-mismatch tandem repeat that X contains. We associate with X the maximal run of $(1+\epsilon)k$-mismatch tandem repeats that contains V. Notice that if X is not the last subrun, then the fact that $|X_j| < 2p$ implies that the suffix of X of length $2p$ is not a $(1+\epsilon)k$-mismatch tandem repeat. Thus the subrun following X overlaps a different maximal run of $(1+\epsilon)k$-mismatch tandem repeats. Similarly, let X' be a subrun which is the union of the $(1+\epsilon)k$-mismatch gd-repeats $X'_i, \cdots, X'_j$. We map X' to a maximal run of $(1+\epsilon)k$-mismatch tandem repeats that contains the last $(1+\epsilon)k$-mismatch tandem repeat in X'. If X' is not the first subrun in W_p, then $|X'_{j'}| < 2p$ and it follows that the prefix of X' of length $2p$ is not a $(1+\epsilon)k$-mismatch tandem repeat, and so the run of $(1+\epsilon)k$-mismatch tandem repeats associated with X' is different from the one associated with the subrun preceding it.

If we did not merge the last subrun whose right root is to the left of $W[\ell]$ with the first subrun whose right root is to the right of (or starts at) $W[\ell]$ then either $V = W[\ell-p \cdots \ell+p-1]$ or $U = W[\ell-p-1 \cdots \ell+p-2]$ are not $(1+\epsilon)k$-mismatch tandem repeats. So the maximal runs associated with the last subrun whose right root is to the left of $W[\ell]$ and the first subrun whose right root is to the right of (or starts at) $W[\ell]$ must be different. If we did merge these subruns then we can associate with the resulting subrun either of the runs associated with the subruns which we merged. An argument as before shows that this run is different from the one associated with the adjacent subruns.

We now prove Property (4). Since $\ell = \lfloor |W|/2 \rfloor$, and $|W| > 2p+1$ then W_p always contains the substring $W[\ell-p \cdots \ell+p-1]$ which is the string of length $2p$ whose right root starts at $W[\ell]$. As mentioned above, if $W[\ell-p \cdots \ell+p-1]$ is a $(1+\epsilon)k$-mismatch

tandem repeat then the string X_0 contains it. Thus if we did not find any subruns in W_p, then $W[\ell - p \cdots \ell + p - 1]$ is not a $(1+\epsilon)k$-mismatch tandem repeat.

We now prove Property (5). Let X be the last subrun which we find and assume that X ends at position $j < |W_p|$. If the right root of X starts to the left of $W[\ell]$, then by the arguments of Property (4), the substring $W[\ell - p \cdots \ell + p - 1]$ is not a $(1+\epsilon)k$-mismatch tandem repeat. Assume that the right root of X is to the right (or starts at) $W[\ell]$. If the last string X_i contained in X is such that $i < \frac{1}{\epsilon} + 1$ then $|X_i| < 2p$ as otherwise we would have added X_{i+1} to X. (Since X_i does not stretch to the end of W_p it cannot contain X_{i+1} so X_{i+1} has not been discarded.) Thereby the suffix of X of length $2p$ is not a $(1+\epsilon)k$-mismatch tandem repeat. If $i = \frac{1}{\epsilon} + 1$ then since $X_{\frac{1}{\epsilon}+1}$ does not stretch to the end of W_p and the last character of W_p does not follow $W[\ell + 2p - 1]$, we must have that the suffix of length $2p$ of W_p is not a $(1+\epsilon)k$-mismatch tandem repeat. The proof of Property (6) is symmetric. ■

It is easy to verify that the running time of the algorithm described in this section is dominated by the running time of the algorithm in Section 7.2.2.1; that is $O\left(\frac{1}{\epsilon}|W|k^{\frac{2}{3}}\log^{\frac{1}{3}}|W|\log k\right)$.

7.4.2.2 Finding all approximate subruns of $(1+2\epsilon)k$-mismatch tandem repeats in S

The algorithm has the same steps as the algorithm of Section 7.4.1. By definition, for a period length p, $p \leq (1+\epsilon)k$, $S[1 \cdots n]$ is a run of $(1+\epsilon)k$-mismatch tandem repeats. Therefore, we will only consider periods of length greater than $(1+\epsilon)k$.

As in Section 7.4.1.2, here we also maintain subruns in explored intervals. An *explored interval* I_p is a substring, say $S[i \cdots j]$, with a period length p associated with it. We associate with I_p subruns of $(1+2\epsilon)k$-mismatch tandem repeats that are contained in the substring $S[\max\{1, i-2p+1\} \cdots j]$. We concatenate two adjacent explored intervals, as described in Section 7.4.1.2.

The algorithm consists of the following steps.

1. We check for each period length p (such that $2(1+\epsilon)k + 2 \leq 2p \leq n$) whether the string $S[1 \cdots 2p]$ is a $(1+\epsilon)k$-mismatch tandem repeat. We can do it by applying the procedure of Section 7.2.2.1 for finding approximate gd-repeats that span a certain character, to the string S with $\ell = 1$. If the string $S[1 \cdots 2p]$ is indeed a subrun we associate it with the explored interval $S[2p \cdots 2p]$ of period length p.

2. Similarly, we check for each period length p (such that $2(1+\epsilon)k+2 \leq 2p < n$) whether the string $S[n-2p+1 \cdots n]$ is a $(1+\epsilon)k$-mismatch tandem repeat. If so we associate this subrun with the explored interval $S[n \cdots n]$ of period length p. For the period length p such that $n = 2p+1$, we concatenate the explored intervals $S[2p \cdots 2p]$ and $S[n \cdots n]$.

3. If $n > 2(1+\epsilon)k+3$, we call a recursive procedure *find-subruns* giving it as a parameter the string S and for each period length $p > (1+\epsilon)k$ such that $n > 2p+1$, the explored intervals $S[2p \cdots 2p]$ and $S[n \cdots n]$ of period length p. The procedure find-subruns described below is similar to the procedure of Section 7.4.1, using the procedure of Section 7.4.2.1, instead of that of Section 7.4.1.1.

We now describe the implementation of the procedure find-subruns when operating on the substring $S[i \cdots j]$. Let $\ell = j - i + 1$ be the length of $S[i \cdots j]$. The procedure find-subruns considers only period lengths p such that $2p + 1 < \ell$. It gets for each such period length an explored interval $I_i(p)$ that ends in position $S[i+2p-1]$, and an explored interval $I_j(p)$ that starts at $S[j]$.

Let $h = \lfloor \ell/2 \rfloor$ and let $m = i + h - 1$. For each relevant period length p, we create the explored interval $I_m(p) = S[\max\{i+2p, m\} \cdots \min\{m+2p-1, j-1\}]$. We use the algorithm of Section 7.4.2.1 to find subruns in $S[\max\{i, m - 2p + 1\} \cdots \min\{m + 2p - 1, j\}]$ that are maximal within this substring.[6] Then we truncate (if necessary) each of the subruns which we find so that the truncated subrun does not end after $S[j - 1]$ and does not start before $S[i+1]$, and remove it if its length is smaller than $2p$ after the truncation. For each p, such that $I_m(p)$ starts at $S[i + 2p]$ we concatenate $I_i(p)$ with $I_m(p)$. Similarly, for each p, such that $I_m(p)$ ends at $S[j - 1]$ we concatenate $I_m(p)$ with $I_j(p)$. Let $I'_m(p)$ be the explored interval containing $I_m(p)$ after these steps. If $m-i+1 > 2(1+\epsilon)k+3$ we recursively apply find-subruns to $S[i \cdots m]$ with the explored intervals $I_i(p)$ and $I'_m(p)$ for each p such that $2p+1 < m-i+1$. Let $I'_m(p)$ be the resulting explored interval. If $j-m+1 > 2(1+\epsilon)k+3$ we recursively apply find-subruns to $S[m \cdots j]$ with the explored intervals $I'_m(p)$ and $I_j(p)$ for each p such that $2p + 1 < j - m + 1$.

The next lemma establishes the correctness of the algorithm.

Lemma 7.4.3 *The algorithm finds a set A of subruns of $(1+2\epsilon)k$-mismatch tandem repeats such that each maximal run of k-mismatch tandem repeats is a substring of a string in the set. The size of the set is bounded by the number of runs of $(1 + \epsilon)k$-mismatch tandem repeats in S.*

Proof: By Lemma 7.4.2(1) and the definition of the algorithm each k-mismatch tandem repeat is contained in a subrun which we find. Since we merge subruns of period length p if their overlap is at least $2p - 1$, each maximal run of k-mismatch tandem repeats is a substring of a single subrun which we report.

By Lemma 7.4.2(2), the procedure of Section 7.4.2.1 finds subruns of $(1+2\epsilon)k$-mismatch tandem repeats. Since the algorithm of Section 7.4.2.2 merges subruns of period length p only if their overlap is at least $2p - 1$, it is easy to see that each string found by our algorithm is a subrun of $(1 + 2\epsilon)k$-mismatch tandem repeats.

[6] We use the procedure of Section 7.4.2.1 with $W = S[i \cdots j]$ and $\ell = m$.

By Lemma 7.4.2(3), we can associate a run of $(1+\epsilon)k$-mismatch tandem repeats with each subrun that we find such that the runs associated with subruns in the same explored interval are different.

By Lemma 7.4.2(4,5,6) runs associated with subruns in different explored intervals must also be different unless one of the subruns is last in its explored interval and ends at the last character of the explored interval, and the other subrun is the first in an adjacent explored interval and contains the first character of that interval.

Using Lemma 7.4.2 again, it is easy to verify that if two such adjacent subruns are associated with the same run of $(1+\epsilon)k$-mismatch tandem repeats then they must overlap in $2p-1$ characters and therefore we merge them when we concatenate the explored interval which they belong to.

■

Assume that when we merge subruns while concatenating explored intervals then we associate with the resulting subrun one of the runs of $(1+\epsilon)k$-mismatch tandem repeats associated with the subruns which we merge arbitrarily (note that these two runs may be identical). The next lemma shows that we never accumulate more than $O(n)$ subruns that are not associated with a unique run of $(1+\epsilon)k$-mismatch tandem repeats. The proof of the lemma is similar to the proof of Lemma 7.4.1 and hence omitted.

Lemma 7.4.4 *At any time during a run of the algorithm we have $O(n)$ subruns that are not associated with a unique run of $(1+\epsilon)k$-mismatch tandem repeats.*

Lemma 7.4.4 implies that we never accumulate more than $O(n+z)$ subruns, where z is the number of maximal runs of $(1+\epsilon)k$-mismatch tandem repeats in S. It is easy to see that the running time of the algorithm is $O\left(\frac{1}{\epsilon}nk^{\frac{2}{3}}\log^{\frac{1}{3}} n \log k \log(n/k) + z\right)$.

7.5 Concluding Remarks

We gave algorithms that find approximate tandem repeats and approximate repeats. It would be interesting to improve the time bounds for these algorithms. Another interesting problem is to use these techniques to get algorithms that find approximate repeats for other types of repeats, such as the ones mentioned in Chapter 1.1.3.2.

Bibliography

[1] Karl Abrahamson. Generalized string matching. *SIAM J. Comput.*, 16(6):1039–1051, 1987.

[2] Noga Alon. A simple algorithm for edge-coloring bipartite multigraphs. *Inf. Process. Lett.*, 85(6):301–302, 2003.

[3] Amihood Amir, Yonatan Aumann, Gad M. Landau, Moshe Lewenstein, and Noa Lewenstein. Pattern matching with swaps. *J. Algorithms*, 37(2):247–266, 2000.

[4] Amihood Amir, Martin Farach, and S. Muthukrishnan. Alphabet dependence in parameterized matching. *Inf. Process. Lett.*, 49(3):111–115, 1994.

[5] Amihood Amir, Moshe Lewenstein, and Ely Porat. Approximate swapped matching. In *FST TCS 2000: Proceedings of the 20th Conference on Foundations of Software Technology and Theoretical Computer Science*, pages 302–311, London, UK, 2000. Springer-Verlag.

[6] Amihood Amir, Moshe Lewenstein, and Ely Porat. Faster algorithms for string matching with k mismatches. *J. Algorithms*, 50(2):257–275, 2004.

[7] Amihood Amir, Ely Porat, and Moshe Lewenstein. Approximate subset matching with don't cares. In *SODA '01: Proceedings of the twelfth annual ACM-SIAM symposium on Discrete algorithms*, pages 305–306, Philadelphia, PA, USA, 2001. Society for Industrial and Applied Mathematics.

[8] Chris Armen and Clifford Stein. Improved length bounds for the shortest superstring problem (extended abstract). In *WADS '95: Proceedings of the 4th International Workshop on Algorithms and Data Structures*, pages 494–505, London, UK, 1995. Springer-Verlag.

[9] Chris Armen and Clifford Stein. A 2 2/3-approximation algorithm for the shortest superstring problem. In *CPM '96: Proceedings of the 7th Annual Symposium on Combinatorial Pattern Matching*, pages 87–101, London, UK, 1996. Springer-Verlag.

[10] Brenda S. Baker. Parameterized pattern matching by boyer-moore-type algorithms. In *SODA '95: Proceedings of the sixth annual ACM-SIAM symposium on Discrete algorithms*, pages 541–550, Philadelphia, PA, USA, 1995. Society for Industrial and Applied Mathematics.

[11] A. Barvinok, E. Kh. Gimadi, and A.I. Serdyukov. *The maximum traveling salesman problem In The Traveling Salesman problem and its variations, G. Gutin and A. Punnen, Eds. Kluwer*, pages 585–607. 2002.

[12] Michael A. Bender and Martin Farach-Colton. The lca problem revisited. In *LATIN*, pages 88–94, 2000.

[13] Markus Bläser. A new approximation algorithm for the asymmetric tsp with triangle inequality. In *SODA '03: Proceedings of the fourteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 638–645, Philadelphia, PA, USA, 2003. Society for Industrial and Applied Mathematics.

[14] Markus Bläser. A 3/4-approximation algorithm for maximum atsp with weights zero and one. In *APPROX-RANDOM*, pages 61–71, 2004.

[15] Markus Bläser and Bodo Manthey. Two approximation algorithms for 3-cycle covers. In *APPROX '02: Proceedings of the 5th International Workshop on Approximation Algorithms for Combinatorial Optimization*, pages 40–50, London, UK, 2002. Springer-Verlag.

[16] Markus Bläser, L. Shankar Ram, and Maxim Sviridenko. Improved approximation algorithms for metric maximum atsp and maximum 3-cycle cover problems. In *WADS*, pages 350–359, 2005.

[17] Markus Bläser and Bodo Siebert. Computing cycle covers without short cycles. In *ESA '01: Proceedings of the 9th Annual European Symposium on Algorithms*, pages 368–379, London, UK, 2001. Springer-Verlag.

[18] Avrim Blum, Tao Jiang, Ming Li, John Tromp, and Mihalis Yannakakis. Linear approximation of shortest superstrings. *Journal of the ACM*, 41(4):630–647, 1994.

[19] Robert S. Boyer and J. Strother Moore. A fast string searching algorithm. *Commun. ACM*, 20(10):762–772, 1977.

[20] Dany Breslauer, Tao Jiang, and Zhigen Jiang. Rotations of periodic strings and short superstrings. *J. Algorithms*, 24(2):340–353, 1997.

[21] Zhi-Zhong Chen and Takayuki Nagoya. Improved approximation algorithms for metric max tsp. In *ESA*, pages 179–190, 2005.

[22] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem, 1976. report 388.

[23] Marek Chrobak, Petr Kolman, and Jiří Sgall. The greedy algorithm for the minimum common string partition problem. *ACM Trans. Algorithms*, 1(2):350–366, 2005.

[24] Peter Clifford and Raphaël Clifford. Simple deterministic wildcard matching. *Inf. Process. Lett.*, 101(2):53–54, 2007.

[25] Raphaël Clifford, Klim Efremenko, Ely Porat, and Amir Rothschild. -mismatch with don't cares. In *ESA*, pages 151–162, 2007.

[26] Richard Cole and Ramesh Hariharan. Tree pattern matching and subset matching in randomized o(nlog3m) time. In *STOC '97: Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 66–75, New York, NY, USA, 1997. ACM.

[27] Richard Cole and Ramesh Hariharan. Approximate string matching: A simpler faster algorithm. *SIAM J. Comput.*, 31(6):1761–1782, 2002.

[28] Richard Cole, Ramesh Hariharan, and Piotr Indyk. Tree pattern matching and subset matching in deterministic o(n log3 n)-time. In *SODA '99: Proceedings of the tenth annual ACM-SIAM symposium on Discrete algorithms*, pages 245–254, Philadelphia, PA, USA, 1999. Society for Industrial and Applied Mathematics.

[29] Graham Cormode and S. Muthukrishnan. The string edit distance matching problem with moves. In *Proceedings of the thirteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 667–676. Society for Industrial and Applied Mathematics, 2002.

[30] Maxime Crochemore, Leszek Gasieniec, Wojciech Plandowski, and Wojciech Rytter. Two-dimensional pattern matching in linear time and small space. In *STACS*, pages 181–192, 1995.

[31] Maxime Crochemore and Lucian Ilie. Maximal repetitions in strings. *J. Comput. Syst. Sci.*, 74(5):796–807, 2008.

[32] Maxime Crochemore, Lucian Ilie, and Liviu Tinta. Towards a solution to the "runs" conjecture. In *CPM*, pages 290–302, 2008.

[33] Maxime Crochemore and Wojciech Rytter. *Text Algorithms*. Oxford Univ. Press, New-York, 1994. pp. 27-31.

[34] Artur Czumaj, Leszek Gasieniec, Marek Piotrów, and Wojciech Rytter. Sequential and parallel approximation of shortest superstrings. *J. Algorithms*, 23(1):74–100, 1997.

[35] Lars Engebretsen. An explicit lower bound for tsp with distances one and two. *Algorithmica*, 35(4):301–318, 2003.

[36] Lars Engebretsen and Marek Karpinski. Approximation hardness of tsp with bounded metrics. In *ICALP '01: Proceedings of the 28th International Colloquium on Automata, Languages and Programming,*, pages 201–212, London, UK, 2001. Springer-Verlag.

[37] Uriel Feige and Mohit Singh. Improved approximation ratios for traveling salesperson tours and paths in directed graphs. In *APPROX-RANDOM*, pages 104–118, 2007.

[38] Alan M. Frieze, Giulia Galbiati, and Francesco Maffioli. On the worst-case performance of some algorithms for the asymmetric traveling salesman problem. *Networks*, 12:23–39, 1982.

[39] Zvi Galil and Raffaele Giancarlo. Parallel string matching with k mismatches. *Theor. Comput. Sci.*, 51:341–348, 1987.

[40] Avraham Goldstein, Petr Kolman, and Jie Zheng. Minimum common string partition problem: Hardness and approximations. *Electr. J. Comb.*, 12, 2005.

[41] Dan Gusfield. *Algorithms on strings, trees and sequences: computer science and computational biology.* Cambridge Univ. Press, 1997. Gusfield.

[42] Dan Gusfield and Jens Stoye. Linear time algorithms for finding and representing all the tandem repeats in a string. *J. Comput. Syst. Sci.*, 69(4):525–546, 2004.

[43] Dov Harel and Robert Endre Tarjan. Fast algorithms for finding nearest common ancestors. *SIAM J. Comput.*, 13(2):338–355, 1984.

[44] Refael Hassin and Shlomi Rubinstein. Better approximations for max tsp. *Inf. Process. Lett.*, 75(4):181–186, 2000.

[45] Refael Hassin and Shlomi Rubinstein. A 7/8-approximation algorithm for metric max tsp. *Inf. Process. Lett.*, 81(5):247–251, 2002.

[46] Carmit Hazay, Moshe Lewenstein, and Dina Sokol. Approximate parameterized matching. *ACM Trans. Algorithms*, 3(3):29, 2007.

[47] Michael Held and Richard M. Karp. The traveling-salesman problem and minimum spanning trees. *Operations Res.*, 18:1138–1162, 1970.

[48] Michael Held and Richard M. Karp. The traveling-salesman problem and minimum spanning trees: Part ii. *Math. Programming*, 1:6–25, 1971.

[49] Tao Jiang and Ming Li. On the approximation of shortest common supersequences and longest common subsequences. *SIAM J. Comput.*, 24(5):1122–1139, 1995.

[50] Haim Kaplan, Moshe Lewenstein, Nira Shafrir, and Maxim Sviridenko. Approximation algorithms for asymmetric tsp by decomposing directed regular multigraphs. *J. ACM*, 52(4):602–626, 2005.

[51] Haim Kaplan, Ely Porat, and Nira Shafrir. Finding the position of the -mismatch and approximate tandem repeats. In *SWAT*, pages 90–101, 2006.

[52] Haim Kaplan and Nira Shafrir. The greedy algorithm for shortest superstrings. *Inf. Process. Lett.*, 93(1):13–17, 2005.

[53] Haim Kaplan and Nira Shafrir. The greedy algorithm for edit distance with moves. *Inf. Process. Lett.*, 97(1):23–27, 2006.

[54] Howard J. Karloff. Fast algorithms for approximately counting mismatches. *Inf. Process. Lett.*, 48(2):53–60, 1993.

[55] Howard J. Karloff and David B. Shmoys. Efficient parallel algorithms for edge coloring problems. *J. Algorithms*, 8(1):39–52, 1987.

[56] Donald E. Knuth, James H. Morris, and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.

[57] Roman Kolpakov and Gregory Kucherov. Finding approximate repetitions under hamming distance. *Theor. Comput. Sci.*, 303(1):135–156, 2003.

[58] Roman M. Kolpakov and Gregory Kucherov. Finding maximal repetitions in a word in linear time. In *FOCS*, pages 596–604, 1999.

[59] Roman M. Kolpakov and Gregory Kucherov. On maximal repetitions in words. In *FCT*, pages 374–385, 1999.

[60] S. Rao Kosaraju. Computation of squares in a string (preliminary version). In *CPM '94: Proceedings of the 5th Annual Symposium on Combinatorial Pattern Matching*, pages 146–150, London, UK, 1994. Springer-Verlag.

[61] S. Rao Kosaraju, James K. Park, and Clifford Stein. Long tours and short superstrings (preliminary version). In *FOCS*, pages 166–177, 1994.

[62] A. V. Kostochka and A. I. Serdyukov. Polynomial algorithms with the estimates 3/4 and 5/6 for the traveling salesman problem of the maximum. (russian). *Upravlyaemye Sistemy*, 26:55–59, 1985.

[63] Lukasz Kowalik and Marcin Mucha. 35/44-approximation for asymmetric maximum tsp with triangle inequality. In *WADS*, pages 589–600, 2007.

[64] Gad M. Landau, Jeanette P. Schmidt, and Dina Sokol. An algorithm for approximate tandem repeats. *Journal of Computational Biology*, 8(1):1–18, 2001.

[65] Gad M. Landau and Uzi Vishkin. Efficient string matching in the presence of errors. In *Proc. 26th IEEE Symposium on Foundations of Computer Science*, pages 126–136, Los Alamitos CA, USA, 1985. IEEE Computer Society.

[66] Gad M. Landau and Uzi Vishkin. Fast parallel and serial approximate string matching. *J. Algorithms*, 10(2):157–169, 1989.

[67] Moshe Lewenstein and Maxim Sviridenko. Approximating assymetric maximum tsp. *SIAM Journal of Discrete Mathematics*, 17:237–248, 2003.

[68] Ming Li, Bin Ma, and Lusheng Wang. On the closest string and substring problems. *J. ACM*, 49(2):157–171, 2002.

[69] Daniel Lopresti and Andrew Tomkins. Block edit models for approximate string matching. *Theor. Comput. Sci.*, 181(1):159–179, 1997.

[70] Bin Ma. Why greed works for shortest common superstring problem. *Theor. Comput. Sci.*, 410(51):5374–5381, 2009.

[71] Michael G. Main and Richard J. Lorentz. An o(n log n) algorithm for finding all repetitions in a string. *J. Algorithms*, 5(3):422–432, 1984.

[72] Michael G. Main and Richard J. Lorentz. *Linear time recognition of square free strings. In Alberto Apostolico and Zvi Galil, editors, Combinatorial Algorithms on Words*, pages 272–278. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 1985.

[73] Edward M. McCreight. A space-economical suffix tree construction algorithm. *J. ACM*, 23(2):262–272, 1976.

[74] S. Muthukrishnan and Süleyman Cenk Sahinalp. Approximate nearest neighbors and sequence comparison with block operations. In *Proceedings of the thirty-second annual ACM symposium on Theory of computing*, pages 416–424. ACM Press, 2000.

[75] Christos H. Papadimitriou and Mihalis Yannakakis. The traveling salesman problem with distances one and two. *Math. Oper. Res.*, 18(1):1–11, 1993.

[76] Michael Rodeh, Vaughan R. Pratt, and Shimon Even. Linear algorithm for data compression via string matching. *J. ACM*, 28(1):16–24, 1981.

[77] Baruch Schieber and Uzi Vishkin. On finding lowest common ancestors: Simplification and parallelization. In *AWOC*, pages 111–123, 1988.

[78] A. I. Serdyukov. the traveling salesman problem of the maximum (russian). *Upravlyaemye Sistemy*, 25:80–86, 1984.

[79] Dana Shapira and James A. Storer. Edit distance with move operations. In *Proceedings of the 13th Annual Symposium on Combinatorial Pattern Matching*, pages 85–98. Springer-Verlag, 2002.

[80] Z. Sweedyk. a $\mathbf{2\frac{1}{2}}$-approximation algorithm for shortest superstring. *SIAM J. Comput.*, 29(3):954–986, 2000.

[81] Jorma Tarhio and Esko Ukkonen. A greedy approximation algorithm for constructing shortest common superstrings. *Theor. Comput. Sci.*, 57:131–145, 1988.

[82] Shang-Hua Teng and Frances F. Yao. Approximating shortest superstrings. *SIAM J. Comput.*, 26(2):410–417, 1997.

[83] Walter F. Tichy. The string-to-string correction problem with block moves. *ACM Trans. Comput. Syst.*, 2(4):309–321, 1984.

[84] Jonathan S. Turner. Approximation algorithms for the shortest common superstring problem. *Inf. Comput.*, 83(1):1–20, 1989.

[85] Esko Ukkonen. On-line construction of suffix trees. *Algorithmica*, 14(3):249–260, 1995.

[86] Sundar Vishwanathan. An approximation algorithm for the asymmetric travelling salesman problem with distances one and two. *Inf. Process. Lett.*, 44(6):297–302, 1992.

[87] Peter Weiner. Linear pattern matching algorithms. In *SWAT '73: Proceedings of the 14th Annual Symposium on Switching and Automata Theory (swat 1973)*, pages 1–11, Washington, DC, USA, 1973. IEEE Computer Society.

[88] Assaf Zaritsky and Moshe Sipper. The preservation of favored building blocks in the struggle for fitness: the puzzle algorithm. *IEEE Trans. Evolutionary Computation*, 8(5):443–455, 2004.